

Extending Intuitionistic Logic to a Probability Calculus (in Lean 4)

Anonymous Author(s)

Abstract

Kolmogorov's probability fixes the event algebra to be Boolean, so every event is decidable. This is a poor fit for the semidecidable properties, termination, reachability, that verification cares about. Following the Cox–Jaynes interpretation of probability as extended logic, we define a construct for extending a logic (treated as a parameter in our approach) to a calculus of probability, and specify criteria for when this works: (1) when the resulting calculus can reconstruct the original logic in the limiting case of certain (0– or 1– probability) events, and (2) when the construction, evaluated at classical logic, agrees with classical probability theory. Instantiating the construction at intuitionistic logic gives valuations on frames, complete Heyting algebras of open sets, the algebras of verifiable propositions. In this setting, we define slack, $1 - v(A) - v(\neg A)$, the classical complement rule's deficit and boundary mass in the measure model. Slack pinpoints exactly when Bayesian conditioning, which treats the evidence as fully decided, gives an incorrect update, disagreeing with Dempster's rule. A separate obstruction, a five-element counterexample, is why Cox's sum rule cannot be derived and must instead be posited. Classical probability credits the slack of a semidecidable property to its failure, an event no finite observation can confirm. The whole development is machine-checked in Lean 4 over mathlib, in roughly 5,000 lines with no sorry.

Keywords: intuitionistic logic, probability, Cox's theorem, belief functions, locales, Lean, mathlib, formalized mathematics

1 Introduction

Kolmogorov's probability [Kolmogorov 1933] assigns numerical values representing uncertainty to events. The event logic is classical, forming a Boolean σ -algebra. The assignments satisfy non-negativity, normalization, and σ -additivity. This commits the framework to the axioms of classical logic, including the law of excluded middle. Many properties of programs (e.g. "does this computation halt?", "does this process reach a given state?") are only *semidecidable*: a procedure confirms the property when it holds, and no procedure, in general, confirms its failure. The mismatch that results is not a contradiction. It is a failure of discrimination. A probability calculus built on a particular logic inherits that logic's identifications, so it cannot discriminate events the logic proves equal.

In the case of classical logic, this identification is the *law of excluded middle*: $\text{halts} \sqcup \neg \text{halts} = \top$, so the Boolean event algebra contains $\neg \text{halts}$. The complement rule $P(A^c) = 1 - P(A)$ then forces the remainder $1 - P(\text{halts})$ onto $\neg \text{halts}$, an event no finite observation can confirm. The events "not yet confirmed" and "refuted" are identified. For a program whose non-termination is independent of ZFC [Yedidia and Aaronson 2016], the classical value $P(\text{halts}) = 0$ is correct, yet neither running the program nor any ZFC proof ever establishes it. Uniformly over programs, computing these sharp values is exactly as hard as the halting problem.

Another failure of discrimination appears in *linear logic*, whose tensor $\otimes$ distinguishes a from $a \otimes a$, one use of a resource from two. Modularity with $\otimes$ in place of $\sqcap$, $v(a) + v(b) = v(a \sqcup b) + v(a \otimes b)$, inherits idempotence of disjunction, $a \sqcup a = a$, and at $b = a$ forces $v(a \otimes a) = v(a)$. So, no assignment satisfying this axiom distinguishes one use of a from two.

Probability is, as the Cox–Jaynes tradition has it, *extended logic* [Cox 1946; Jaynes 2003], and different logics can be extended through the same construction [Paris 2001; Williams 2012]. The logic of events, in its algebraic form the event algebra, is a *parameter* of the construction, and the choice of event algebra determines the probability calculus. An event algebra consists of a set of events together with operations interpreting the logic's connectives and structural rules, while a probability calculus is differentiated from an arbitrary grading of events by satisfying certain axioms. We specify how to construct such extensions, and propose two criteria for when the extension of a logic is a probability calculus:

- (1) the valuations taking only the values 0 and 1 (the two degrees of complete certainty) must reconstruct the underlying event logic. This is what we take "extended logic" to mean.
- (2) the construction evaluated at classical logic must return the classical theory, ensuring it is a genuine generalization of classical probability.

We propose two fixes for the identifications above. Both extend a logic to a probability calculus, and on finite frames the second reduces to the first. The first fix we propose is one that accepts the standard axioms imposed on a grading to make it a probability calculus, while choosing an event algebra that makes the required distinctions. The extension of the event algebra to a probability calculus is then given by the *valuations*, i.e. the maps from events to numbers satisfying monotonicity, normalization, and modularity [Paris 2001; Weatherson 2003]. The resulting extension does not

guarantee defining a probability calculus, and we specify the conditions under which it does: the $\{0, 1\}$ -valued valuations are the points of the event algebra, and they reconstruct its order when it is distributive. Intuitionistic logic, in particular, is a case for which this approach yields an extension, and we work it out in detail. Its event algebras are the Heyting algebras, and we work with frames (the complete Heyting algebras). The open sets of any topological space form a frame. On a frame the pseudocomplement $\neg a$ need not satisfy $a \sqcup \neg a = \top$, so the complement rule weakens to $v(a) + v(\neg a) \leq 1$. The gap $\text{slack}(a) = 1 - v(a) - v(\neg a)$ is the mass of the part of a neither confirmed nor refuted. For the valuation that a measure μ induces on the open sets of a space, the slack of an open set is the μ -measure of its boundary.

The second fix also applies to logics whose connectives are not lattice operations, such as linear logic. It generalizes the axioms of the probability calculus in addition to varying the event algebra. It applies to any logic presented by a set of *certain states*, its $\{0, 1\}$ -valued assignments. The calculus is the set of finite mixtures of certain states, the construction of de Finetti that generalized probabilism applies to non-classical logics [Paris 2001; Williams 2012]. A mixture that takes only the values 0 and 1 is one of the certain states it mixes, because every certain state with positive weight must agree with it at every event. So the $\{0, 1\}$ -valued elements of the calculus are exactly the certain states, which present the logic, and criterion (1) holds. For finitely many events, the axioms of the calculus are exactly the affine laws that hold on every certain state. When the certain states respect $\wedge$ and $\vee$, Paris shows that these laws are monotonicity, normalization, and modularity [Paris 2001], so on finite frames and finite Boolean algebras the calculus is the set of valuations.

For linear logic, the event algebra is a quantale, a complete lattice with an associative tensor $\otimes$ that need not be idempotent. The first fix fails there because of the probability axioms, not the event algebra: the modularity axiom identifies a with $a \otimes a$. Probabilistic coherence spaces [Danos and Ehrhard 2011] avoid this by assigning weights to the points of each formula's web instead of to formulas, with $a \otimes a$ interpreted on a product web. Their elements are not valuations in the sense of the modularity axiom. For every multiplicative-additive formula over finite data types, the calculus of the second fix is contained in the probabilistic coherence space of the formula, and the certain elements of that space are cliques of Girard's coherence space [Girard 1987].

The calculus and the probabilistic coherence space coincide at first-order linear logic. At higher order they can differ: for a program with two functional arguments the calculus is the set of randomized sequential programs, a strict subset of the probabilistic coherence space (Section 11). The probabilistic coherence space, the standard model of probabilistic programs at this type, therefore admits behaviours

that no sequential program flipping coins can produce in a linear probabilistic language for this type. Reasoning in it can certify only weaker guarantees than the programs satisfy. The gap has the same numerical form as the gap between classical and non-classical correlations in quantum foundations [Cabello et al. 2014]. At $! \text{Bool}$ the calculus is also a strict subset.

Quantum logic, whose event algebras are orthomodular lattices [Birkhoff and von Neumann 1936], is a logic to which neither fix applies. Both rely on $\{0, 1\}$ -valued assignments: the first on $\{0, 1\}$ -valued valuations, which need not exist on non-distributive lattices and do not exist on M_3 , and the second relies on having certain states. A certain state of quantum logic would assign 1 to exactly one ray of every orthogonal basis, and by the Kochen–Specker theorem [Kochen and Specker 1967] there is none in dimension at least 3. We prove this for an 18-ray configuration in $\mathbb{R}^4$ [Cabello et al. 1996], so the calculus of the second fix is empty there. Quantum states, which by Gleason's theorem [Gleason 1957] are given by density matrices, nevertheless exist, and they fail criterion (1).

Contributions.

1. **Characterizations** (Sections 2, 3 and 5). The complement rule holds for every valuation iff the frame satisfies excluded middle, Dempster's rule agrees with Bayesian conditioning iff the evidence has zero slack, and slack splits exactly into a double-negation part and a De Morgan part.
2. **Structure theorems** (Sections 4 and 5). Every valuation on the lower sets of a poset splits into an atomic part carried by points and a diffuse remainder, and on its Boolean core it is a belief function.
3. **Cox's theorem** (Section 6). The sum-rule half admits no derivation over a frame, even with weak excluded middle, since on a five-element frame a disjunction's value is not a function of its disjuncts' values.
4. **Grounding models** (Section 7). For a classical measure read on the opens, slack is boundary mass, and a classical posterior overstates refutation by exactly the conditional boundary mass $\mu(\partial A \cap B)/\mu(B)$.
5. **Products** (Section 8). Product valuations are unique on $\text{Low}(\mathbb{N} \times \mathbb{N})$, and the countably-presented valuations compose associatively and commutatively.
6. **Beyond frames** (Section 9). Lattice valuations are the normalized monotone homomorphisms out of a valuation monoid in which every element is a sum over a chain, and no modular grading of linear logic's $\otimes$ distinguishes a from $a \otimes a$.
7. **Calculi from certain states** (Section 10). The completeness theorem of generalized probabilism has a proof without choice for rational points, and on arbitrary sets of events the functions satisfying its laws

are the barycenters of measures concentrated on the certain states.

8. **Linear logic** (Sections 10.2 and 10.3). The calculus of a multiplicative–additive formula is contained in its probabilistic coherence space, whose certain elements are cliques, with equality at implications between data types and strict inclusion at !Bool .
9. **Second-order linear types** (Section 11). For a program with two functional arguments over data types with at least two elements, the calculus is the stable-set polytope of a graph whose clique-constrained polytope is the probabilistic coherence space, and the resulting strict inclusion transfers to every formula in which this type occurs positively.
10. **Quantum event logic** (Section 12). On Cabello’s Kochen–Specker set the calculus is empty, and with divergence allowed every mixture of certain states has total weight at most 4, against $\frac{9}{2}$ for every quantum state.
11. **A first formal treatment.** Every result above is machine-checked in Lean 4 over mathlib, except cited theorems the text marks as not formalized, which to our knowledge is the first mechanization of non-additive probability over intuitionistic logic.

Mechanization. The development is carried out in Lean 4 [de Moura and Ullrich 2021] over mathlib [mathlib Community 2020], in roughly 12,000 lines with no sorry, and is included in full as anonymous supplementary material.¹ Two points are specific to a proof assistant. We separate *object-level* classicality, excluded middle in the frame Ω , from the *meta-level* classicality mathlib inherits through `Classical.choice`, identify where the meta-level dependence is essential (Table 1), and give proofs without choice of the finite results that admit them. The product-rule half of Cox’s theorem needs an Aczél/Hölder regratuation absent from mathlib, which we build from scratch.

2 Valuations on Frames

2.1 The Event Logic

Definition 1 (frame). A *frame* is a complete lattice Ω in which finite meets distribute over arbitrary joins,

$$a \sqcap \left(\bigsqcup_{i \in I} b_i \right) = \bigsqcup_{i \in I} (a \sqcap b_i)$$

for every $a \in \Omega$ and every family $\{b_i\}_{i \in I} \subseteq \Omega$. Equivalently, it is a complete Heyting algebra. By $x \Rightarrow y$ we denote the Heyting implication in a frame, and $\neg a := a \Rightarrow \perp$ is the pseudocomplement of $a \in \Omega$. Also, a^c denotes a complement of a , if it exists, i.e. an element such that $a \sqcap a^c = \perp$ and $a \sqcup a^c = \top$.

¹Claude (Anthropic) was used to assist with drafting and debugging the Lean formalization.

Frames are the algebraic semantics of intuitionistic logic and the point-free abstraction of topological spaces. The same objects with morphisms reversed form the category of *locales*, so the distinction is invisible at the level of a single algebra. We say *frame* throughout and reserve *locale* for the spatial and categorical statements (points of a locale, classifying locales) where that word is the standard one. The motivating example throughout is $\Omega = \mathcal{O}(X)$, the opens of a space, with $\sqcap, \sqcup$ as intersection and union and the pseudocomplement $\neg a$ of an open given by the *interior* of its set complement. In mathlib this is the typeclass `Order.Frame`, and the Heyting operations come with the key asymmetry already in place: $a \sqcap \neg a = \perp$ holds in any Heyting algebra (`inf_compl_eq_bot`), while $a \sqcup \neg a = \top$ is available only in Boolean algebras. An element with $a \sqcup \neg a = \top$ is *complemented*: it satisfies its own instance of excluded middle.

2.2 The Valuation Structure

The central object assigns a degree of certainty to each element of the frame, with inclusion–exclusion as the only interaction axiom:

Definition 2 (valuation^{Lean}). A *valuation* on a frame Ω is a *monotone* function $v : \Omega \rightarrow [0, \infty]$ with $v(\perp) = 0$, $v(\top) = 1$ (*normalization*), and, for all $a, b \in \Omega$, *modularity*:

$$v(a) + v(b) = v(a \sqcup b) + v(a \sqcap b).$$

```
structure Valuation (Ω : Type*) [Order.
  Frame Ω] where
  toFun : Ω → ℝ≥0∞
  map_bot' : toFun ⊥ = 0
  map_top' : toFun ⊤ = 1
  mono' : Monotone toFun
  modular' : ∀ a b, toFun a + toFun b
    = toFun (a ⊔ b) + toFun (a ⊓ b)
```

This axiom set is not ours: Weatherston’s axioms for intuitionistic probability are exactly these [Weatherston 2003], they are Paris’s axioms for logics whose two-valued valuations respect $\wedge$ and $\vee$ [Paris 2001], and the valuations of constructive measure theory [Coquand and Spitters 2009; Vickers 2008] are their Scott-continuous instances (Section 13).

Values live in the extended non-negative reals $[0, \infty]$, and normalization confines them to $[0, 1]^{\text{Lean}}$. The conventions this choice carries, unconditional summation and truncated subtraction, are an artifact of the mechanization and are recorded in Appendix A.

2.3 Slack

Modularity applied to the disjoint pair $(a, \neg a)$ collapses to $v(a) + v(\neg a) = v(a \sqcup \neg a)^{\text{Lean}}$. So, the join need not be $\top$:

Theorem 3 (the complement bound^{Lean}). *For every valuation v on a frame Ω and every $a \in \Omega$, $v(a) + v(\neg a) \leq 1$.*

theorem `Valuation.add_compl_le_one` ($v :$
 $\text{Valuation } \Omega$) ($a : \Omega$) :
 $v\ a + v\ a^c \leq 1$

This is a *derived* Dempster–Shafer sub-additivity inequality. The deficit

$$\text{slack}(a) = 1 - (v(a) + v(\neg a)) = 1 - v(a \sqcup \neg a)$$

is the valuation of the undecided region. It decomposes canonically. The pair $(\neg\neg a, \neg a)$, double negation against negation, is always disjoint. However, it is a partition only under *weak excluded middle* ($\neg a \sqcup \neg\neg a = \top$). Applying modularity to this disjoint pair splits the slack into two logically independent gaps:

Definition 4 (regular element). An element $a \in \Omega$ is *regular* (or $\neg\neg$ -stable) if $a = \neg\neg a$.

Theorem 5 (slack decomposition^{Lean}). *For every valuation v on a frame Ω and every $a \in \Omega$, $\text{slack}(a) = \text{dnGap}(a) + \text{deMorganGap}(a)$, where $\text{dnGap}(a) = v(\neg\neg a) - v(a)$ is 0 if a is $\neg\neg$ -stable (regular), and $\text{deMorganGap}(a) = 1 - v(\neg\neg a \sqcup \neg a)$ is 0 if weak excluded middle holds at a .*

The single Dempster–Shafer “ignorance” number is thus resolved into a regularity defect and a De Morgan defect: the first measures failure of double-negation stability, the second corresponds to a named intermediate logic (weak excluded middle). Constructively, the contrapositive of a universal hypothesis returns only its double-negated form, so for open sets, this specification of probability does not fall victim to Hempel’s raven paradox [Hempel 1945]: evidence of the white sock indoors confirms at most $\neg\neg a$, not a itself, and dnGap is exactly the gap between confirming $\neg\neg a$ and confirming a itself.

Because dnGap and deMorganGap are themselves defined from v , the apparent three unknowns $v(a)$, $v(\neg a)$, and $v(\neg\neg a)$ are not independent: they are part of a three-constraint system, yielding the following result:

Theorem 6 (slack in closed form^{Lean}). *For every valuation v on a frame Ω and every $a \in \Omega$,*

$$\text{slack}(a) = (v(\neg\neg a) - v(a)) + (1 - v(\neg\neg a) - v(\neg a)),$$

where $v(a) \leq v(\neg\neg a)$.

The inequality follows from monotonicity applied to $a \leq \neg\neg a$, a fact of every Heyting algebra, and shows the first summand is exactly the difference, not the floor produced by $[0, \infty]$ ’s truncated subtraction. The split into a regularity defect and a De Morgan defect follows from monotonicity and modularity alone, with no further axiom about ignorance required. This contrasts with the usual Dempster–Shafer picture, where the mass assigned to non-singleton, ambiguous sets is an extra primitive.

Theorem 6 expresses slack pointwise in terms of v . The converse, whether slack in turn determines v , is more delicate. One constraint is automatic:

Theorem 7 (slack is submodular^{Lean}). *For every valuation v on a frame Ω and every $a, b \in \Omega$,*

$$\text{slack}(a \sqcup b) + \text{slack}(a \sqcap b) \leq \text{slack}(a) + \text{slack}(b).$$

The proof runs v ’s own modularity twice, once at (a, b) and once at $(\neg a, \neg b)$, and combines the two results with the De Morgan laws: the equality $\neg(a \sqcup b) = \neg a \sqcap \neg b$, which holds in every Heyting algebra, and the inequality $\neg a \sqcup \neg b \leq \neg(a \sqcap b)$, monotone under v but not in general an equality. Any function proposed as the slack of some valuation must therefore be submodular.

Remark 8 (non-recovery^{Lean}). Submodularity is necessary, not sufficient: Section 7.2 exhibits two valuations on the same frame with identical slack functions.

Remark 9 (slack under weak excluded middle^{Lean}). The two gaps, dnGap and deMorganGap , computed at a single element, are logically independent. However, if weak excluded middle is imposed on the frame for every a ($\neg\neg a \sqcup \neg a = \top$ ^{Lean}), then deMorganGap vanishes identically and $\text{slack}(a) = \text{dnGap}(a)$ for every valuation and every a . This sits strictly between plain intuitionistic logic, where both gaps can be positive, and full excluded middle, where both vanish^{Lean}. A concrete instance appears in Section 7.

2.4 What Survives without Excluded Middle

Three classical pillars survive intact, and one fails in a precisely delimited way. *Disjoint additivity* is unconditional: $a \sqcap b = \perp$ implies $v(a \sqcup b) = v(a) + v(b)$. *Conditioning* is native: for $v(b) \neq 0$ the posterior $v(a \mid b) = v(a \sqcap b) / v(b)$ is again a valuation (the proof is frame distributivity), and the product rule $v(a \mid b) \cdot v(b) = v(a \sqcap b)$ and Bayes symmetry (and therefore Bayesian updating) $v(a \mid b) v(b) = v(b \mid a) v(a)$ hold^{Lean}. *Total probability* holds over any genuine partition, meaning $a \sqcap a' = \perp$ and $a \sqcup a' = \top$: $v(b) = v(a \sqcap b) + v(a' \sqcap b)$. What fails is total probability over $\{a, \neg a\}$. That family need not join to $\top$, and the conditional masses fall short of $v(b)$ by a conditional slack^{Lean}.

The classical fragment. We characterize the elements of Ω on which the classical complement rule holds. The $\neg\neg$ -stable (regular) elements, which form a Boolean algebra (by Glivenko’s theorem), are a natural candidate, but they do not suffice. Modularity constrains the frame join $\sqcup$, whereas the Boolean join of regular elements is the regularized $\neg\neg(a \sqcup b)$, for which only the inequality $v(a) + v(b) \leq v(\neg\neg(a \sqcup b)) + v(a \sqcap b)$ survives (i.e. the 2-monotonicity of Section 5). A regular but uncomplemented element, which is an open ray $(-\infty, 0)$ in the opens of $\mathbb{R}$, under an atomic measure at 0, still carries slack. The correct classical fragment is the *complemented* elements, where $v(a) + v(a^c) = 1$ holds exactly^{Lean}. On a connected locale that fragment can be as small as $\{\perp, \top\}$. In the Boolean limit, every element is complemented, the slack vanishes identically, and classical Kolmogorov axioms are recovered^{Lean}.

Mixtures. Valuations are closed under finite convex combination^{Lean}, with every axiom inherited termwise, because modularity is linear. This innocuous fact becomes a characterization in Section 4.

3 Classicality in Theory and Metatheory

Classicality enters this work twice: in the metatheory, the logic of the proof assistant itself, and in the object theory, the frame’s own logic.

The metatheory. Lean’s kernel is constructive, and its core library declares `Classical.choice` as an axiom, from which excluded middle follows. A theorem is constructive when its proof does not use this axiom, which `#print` axioms reports. `mathlib` uses the axiom throughout. The order on $\mathbb{R}$ and $[0, \infty]$ has no computable Decidable instance, so every library lemma that case-splits on $x \leq y$ for such numbers uses `Classical.propDecidable`, and so do the library’s finite sums and finite sets. A theorem that never invokes excluded middle for frame elements $a \in \Omega$ can therefore still depend on the axiom through the library. Almost every theorem of this development does. This dependence is incidental: it comes from the library, not from the argument. Table 1 lists the classical principles the arguments themselves use, and what a constructive version would require.

The finite combinatorial results (M_3 , the pentagon, the perfect-graph criterion, the Kochen–Specker set, and the multiplicative–additive formulas) need none of these principles. The Cox model of Section 6 is given an explicit computable Decidable instance.

Object-level classicality: the R3 axiom. Van Horn’s repaired Cox theorem [Horn 2003] axiomatizes plausibility over classical propositions. Its negation axiom, called *R3*, posits that the plausibility of $\neg A$ is a fixed non-increasing function S of the plausibility of A . Consistency forces the involution $S \circ S = \text{id}$, and therefore the complement rule $p \mapsto 1 - p$. We show that the involutive negation is double-negation elimination, so R3 is where classical logic enters every known Cox-style derivation. To do this, we first isolate the exact content of R3’s conclusion as a predicate on valuations, then prove equivalence to excluded middle.

That predicate is *classical negation*: a valuation v has classical negation if $v(a) + v(\neg a) = 1$ for every $a \in \Omega$.

Theorem 10 (the R3 characterization^{Lean}). *For every frame Ω , every valuation on Ω has a classical negation iff $a \sqcup \neg a = \top$ for every $a \in \Omega$.*

The forward direction is a calculation. The content is the converse: wherever excluded middle fails, a valuation with genuine slack must be *constructed*. Suppose $a \sqcup \neg a \neq \top$.

Definition 11 (ideal, prime ideal). An *ideal* $J \subseteq \Omega$ is a downward closed set which is also closed under finite joins.

Result	Classical principle	Constructive version	
Converse of Theorem 10, separation (Sec. 9)	prime ideal theorem	finite lattices: proved without choice ^{Lean} . In finite frames: only for frames with enough points	496 497 498 499 500 501
Completeness (Thm. 68)	Hahn–Banach over $\mathbb{R}$	for rational points and integer certain states, proved without choice by Fourier–Motzkin elimination over $\mathbb{Z}^{\text{Lean}}$	502 503 504 505 506
Barycenters (Thm. 70)	compactness of probability measures	finite E and rational values: the completeness row. In general the fan theorem is needed	507 508 509 510
Sharp readout (Sec. 7.3)	excluded middle for halting	not definable, replaced by a synthetic undecidability statement [Forster et al. 2019]	511 512 513 514
Non-uniqueness of products (Thm. 49)	free ultrafilter on $\mathbb{N}$	not shown on $\text{Set}(\mathbb{N} \times \mathbb{N})$. Proved without choice on the algebra of sets eventually determined by the order of the coordinates ^{Lean}	515 516 517 518 519 520
Sierpiński model (Sec. 7.3)	bivalence of propositions	an explicit two-point space, proved without choice ^{Lean}	521 522 523
Slack is boundary mass (Thm. 32)	classical measures and set complements	finite spaces, proved without choice ^{Lean} . In general constructive valuations [Coquand and Spitters 2009]	524 525 526 527 528
Kraft bound for Ω (Sec. 7.3)	product measure on Cantor space	finite Kraft inequality, proved without choice ^{Lean} . Ω is the supremum of its partial sums, a lower real	529 530 531 532
Stonean characterization (Sec. 7.1) Values in $[0, \infty]$, infinite sums	Stone duality for spaces classical extended reals	none, the localic statement is the definition rationals for finite results, lower reals in general, in which slack is an upper real	533 534 535 536 537 538
Aczél regratuation (Sec. 6)	classical real analysis	not attempted	539 540

Table 1. Classical principles used by the arguments. “Proved without choice” means that `#print` axioms lists at most `propext` and `Quot.sound`. These proofs use core Lean, basic `mathlib` order files, and `mathlib`’s ring tactic on $\mathbb{Z}$, and avoid `mathlib`’s $\mathbb{Q}$ and finite sums, which depend on `Classical.choice`.

An ideal J is *prime* if $\top \notin J$ and $a \sqcap b \in J$ implies $a \in J$ or $b \in J$.

The prime ideal separation theorem (mathlib's `DistribLattice.prime_ideal_of_disjoint_filter_ideal`) yields a prime ideal J containing $a \sqcup \neg a$ but not $\top$. The indicator of J 's complement,

$$v(x) = \begin{cases} 0 & x \in J \\ 1 & x \notin J, \end{cases}$$

is a valuation. Modularity in the only nontrivial case ($x, y \notin J$) is *exactly* primeness, $x \sqcap y \in J \Rightarrow x \in J \vee y \in J$. And this valuation assigns $v(a) + v(\neg a) = 0 \neq 1$.

Sharp valuations are the points of the locale. The construction above is one half of an exact correspondence. Call v *sharp* if it is $\{0, 1\}$ -valued, a state of full certainty. Sharpness characterizes exactly the prime-ideal indicators:

Theorem 12 (sharp valuations are points^{Lean}). *For every valuation v on a frame Ω , v is sharp iff v is the complement-indicator of a prime ideal.*

The correspondence is given by two mutually inverse maps: $v \mapsto Z(v) = \{x \in \Omega : v(x) = 0\}$, which computes the zero-set of a sharp valuation, and the map taking J to its complement-indicator (the valuation of Theorem 10's construction). $Z(v)$ is prime exactly because v is sharp, and the complement-indicator of $Z(v)$ recovers v itself. In locale-theoretic terms, sharp valuations recover the *points* of the locale. A prime ideal is *completely prime* when it is also closed under arbitrary (not just finite) suprema of its members. The finitely prime points of Theorem 12 sharpen to the completely prime points exactly where Scott continuity is added, recovering points of the locale in the sense used by [Coquand and Spitters 2009; Vickers 2008].

Definition 13 (directed set, Scott continuity). A subset $D \subseteq \Omega$ is *directed* if, for any finite subset $F \subseteq D$, there exists an element $d \in D$ that is an upper bound of F (taking $F = \emptyset$ shows a directed set is nonempty). A valuation v is *Scott-continuous* if $v(\bigsqcup D) = \sup_{a \in D} v(a)$ for every directed D .

Theorem 14 (Scott-continuous sharp valuations^{Lean}). *For every valuation v on a frame Ω , v is sharp and Scott-continuous iff v is the complement-indicator of a completely prime ideal.*

The characterization and the two correspondences together locate the object-level classicality precisely: whenever excluded middle fails at some a , the failure occurs at a point of the locale.

Points and the adjunction with spaces. Write $[1] = \{0 < 1\}$, the initial frame. For a lattice P , monotone maps $P \rightarrow [1]$ correspond to upper sets (the preimage of 1), bounded lattice homomorphisms to prime filters, and frame homomorphisms to completely prime filters, those F with $\bigsqcup S \in F$ only if $S \cap F \neq \emptyset$. The complements are the lower

sets, prime ideals, and completely prime ideals of Theorem 12 and the Scott-continuous characterization above. The points of a frame P , $\text{pt}(P) = \text{Frm}(P, [1])$, carry the topology of the sets $\Sigma_a = \{f \mid f(a) = 1\}$, and Σ preserves finite meets and arbitrary joins. With O sending a space to its frame of opens, this gives the adjunction $O \dashv \text{pt}$ between spaces and locales [Johnstone 1982], in mathlib as `Locale.adjunctionTopToLocalePT`. Its units send a point x to $U \mapsto [x \in U]$ and an element a to Σ_a . A prime filter need not be completely prime: in $[0, \infty]$, the filter $\{\infty\}$ is prime but contains $\bigsqcup_n n = \infty$ and no n . The corresponding sharp valuation is not Scott-continuous, as for the $\top$ -indicator on $\text{Low}(\mathbb{N})$ of Section 4^{Lean}.

Finite frames without choice. The prime ideal theorem is a choice principle. Without it, a frame can have no points at all [Johnstone 1982], and then it has no sharp valuations, so the converse of Theorem 10 and the statement that sharp valuations reconstruct the logic (the first criterion of Section 1) hold only in a classical metatheory. On a finite distributive lattice with decidable order the needed prime filter is found by search: an element below a and not below b that is minimal for the size of its downset is join-prime, and the indicator of its principal filter is a sharp valuation. This proves the finite separation theorem and the finite form of the converse of Theorem 10 without `Classical.choice`^{LeanLean}, for $\mathbb{N}$ -valued valuations. In a constructive metatheory the first criterion therefore holds for finite event algebras and for frames with enough points, and not in general.

A development in Rocq. Rocq's kernel, like Lean's, does not assume excluded middle or choice, so the two differ in their libraries more than in their logic. The differences would be the following.

- **Finite results.** The Mathematical Components library [Mahboubi and Tassi 2021] works with types with decidable equality and finite types, and avoids classical axioms in its algebraic core. The finite theorems of this paper (the finite representation, the frame recipe, M_3 , the pentagon, the perfect-graph criterion, and the Kochen–Specker set) would be proved there without classical axioms and could be extracted to programs.
- **Values.** $[0, \infty]$ is a classical object. A constructive development would use rational values for finite results and lower reals, approximable from below, for valuations in general [Coquand and Spitters 2009; Vickers 2008]. The value of the halting event, Chaitin's Ω , is a lower real of this kind: it is approximated from below by the halting programs found so far, and no better.
- **Analysis.** MathComp-Analysis [Affeldt et al. 2018] assumes classical axioms, as mathlib does. The completeness theorem for rational points is the Fourier–Motzkin argument above, which carries over directly.

Completeness over real points would require the constructive analysis of Bishop's school, with locatedness hypotheses for hyperplane separation. The barycenter theorem for infinitely many events requires compactness of $\{0, 1\}^{\mathbb{N}}$, which constructively is the fan theorem, a principle that would have to be added as an axiom.

- *Computability.* Without excluded middle the sharp classical readout cannot be defined. The uncomputability statement would be phrased synthetically [Forster et al. 2019]: no function from codes to Booleans decides halting, which requires an axiom such as Church's thesis to state as a negative result about all functions.
- *Points.* The separation results would be stated for frames with enough points, or for finite lattices as above.

Lean was chosen for mathlib's measure theory, order theory and topology, which the infinite results use. The cost is that constructive content is a property to be checked theorem by theorem with `#print axioms`, rather than a default.

A natural direction for future work is to develop the extension of logics to probability calculi in a constructive metatheory throughout. This would give algorithms in place of existence proofs, for instance the weights of a convex combination and the separating laws of Theorem 68, and would match the semidecidable reading of events with value types such as lower reals. It would also restrict the results that depend on points existing to frames with enough points.

4 Valuation Representation

Every valuation on a finite frame is a probability mass function on points. On infinite frames, this can fail, and Scott continuity marks the boundary between where it still holds and where it does not.

4.1 Finite Frames Representation

Theorem 15 (Birkhoff duality [Birkhoff 1937]). *Every finite distributive lattice is isomorphic to the lattice of lower sets of a finite poset: its sup-irreducible elements form a poset P whose lower sets reconstruct the lattice, and every finite poset arises this way from its own lattice of lower sets.*

Let P be the finite poset of sup-irreducible elements of a finite frame. Then, by Birkhoff duality (`OrderIso.lowerSetSupIrred`), every finite frame is the lattice of lower sets of P . We fix the following notation, used throughout the rest of the paper. A lower set $L \subseteq P$ satisfies $p \in L, q \leq p \Rightarrow q \in L$, and an upper set is defined dually. Lower sets are always denoted L , and upper sets U (opens of a topological space are still U, V, W , as in traditional usage). The principal lower set at p is $\downarrow p = \{q : q \leq p\}$, the strict principal lower set at p is $\Downarrow p = \{q : q < p\}$, and the principal upper set at p is $\uparrow p = \{q : q \geq p\}$. We prove the

finite representation theorem below on the lower sets of P . Transporting it across the isomorphism above then gives the theorem for every finite frame, not just lower sets^{Lean}. Let us define the *mass* of a point as the discrete derivative of v^{Lean} :

$$\text{mass}(p) = v(\downarrow p) - v(\Downarrow p).$$

The masses recover v and sum to 1:

Theorem 16 (finite representation^{Lean}). *For a finite P , any valuation v on $\text{Low}(P)$, and any $L \in \text{Low}(P)$, $v(L) = \sum_{p \in L} \text{mass}(p)$, and $\sum_p \text{mass}(p) = 1$.*

The proof works by induction on $|L|$: repeatedly remove a maximal point p from L , and use modularity applied to $(L \setminus \uparrow p, \downarrow p)$ to show each removal changes v by exactly $\text{mass}(p)$. The mass function is a classical probability mass function on P^{Lean} . And since lower sets are exactly the opens of the Alexandrov topology, where interior is the identity, the theorem also reads $v = \mu \circ \text{int}$, i.e. every valuation on a finite frame is the classical measure μ applied after taking the interior. In the finite case, this is the converse of the measure-theoretic bridge of Section 7. The intuitionistic mass also agrees with the classical notion of the mass of a point: reading $v = \mu \circ \text{int}$,

$$\text{mass}(p) = \mu(\downarrow p) - \mu(\Downarrow p) = \mu(\{p\}),$$

since $\downarrow p$ is the disjoint union of $\Downarrow p$ and $\{p\}$ and μ is defined on all subsets of P .

Theorem 16 represented v as a sum of point masses. The dual view represents v as a mixture of sharp point-valuations. For each $p \in P$, let δ_p be the valuation with $\delta_p(L) = 1$ if $p \in L$, and $\delta_p(L) = 0$ otherwise:

Theorem 17 (mixture characterization^{Lean}). *On a finite frame, every valuation v satisfies $v = \sum_p \text{mass}(p) \cdot \delta_p$.*

Together with the fact that any mixture of valuations is again a valuation^{Lean}, this theorem characterizes the valuations on a finite frame exactly: they are the finite convex mixtures of point-valuations, closed under mixing because modularity is linear. Section 6 shows modularity cannot instead be derived from more primitive axioms. It must be posited outright, and doing so is exactly what makes every valuation a mixture of sharp, all-or-nothing certainties. The characterization is an instance of a theorem of Paris: for a propositional logic whose two-valued valuations respect $\wedge$ and $\vee$, such as intuitionistic logic, the functions that give tautologies 1 and contradictions 0, are monotone for the logic's consequence relation, and are modular are, on every finite set of sentences, exactly the mixtures of valuations [Paris 2001].

4.2 Infinite Boundary Representation via Scott Continuity

Both halves of the finite theorem fail for infinite frames:

Example 18 (obstruction^{Lean}). On $\text{Low}(\mathbb{N})$, the indicator v of $\top$ is a valuation with $\text{mass}(p) = 0$ for every point p , and $v(\top) = 1$.

No point carries any of the mass. The failure is one of approximation: $v(\top)$ is not the supremum of $v(\downarrow n)$ over the finite stages. The $\top$ -indicator of Example 18 is not Scott-continuous^{Lean}, and Scott continuity is exactly what restores the representation:

Example 19 (recovery^{Lean}). Let v be a valuation on $\text{Low}(\mathbb{N})$. If $v(\top) = \sup_n v(\downarrow n)$ then $v(\top) = \sum_n \text{mass}(n)$.

In general, on any poset all of whose principal lower sets are finite, Scott continuity implies zero diffuse part without additional preconditions. However, subtraction in $[0, \infty]$ is truncated at 0, so if $\sum_p \text{mass}(p)$ were greater than $v(\top)$, the expression $v(\top) - \sum_p \text{mass}(p)$ would still evaluate to 0. Theorem 20 rules out $\sum_p \text{mass}(p) > v(\top)$ unconditionally, so the subtraction below is never truncated:

Theorem 20 (general decomposition^{Lean}). *For every valuation v on $\text{Low}(P)$ and an arbitrary poset P , $\sum_p \text{mass}(p) \leq v(\top)$.*

By Theorem 20 the remainder $v(\top) - \sum_p \text{mass}(p)$ is a genuine difference. Every valuation thus splits unconditionally into an *atomic* part carried by points ($\sum_p \text{mass}(p)$, which is always a sub-probability), and a *diffuse* (nonnegative) remainder $v(\top) - \sum_p \text{mass}(p)$. This is a constructive Lebesgue-style decomposition. The finite and Scott cases are the purely atomic extremes, and the $\top$ -indicator is the purely diffuse one. The proof of Theorem 20 re-runs the maximal-point induction on each finite subset of P , bounding its partial sum by $v(\top)$, then takes the supremum over all finite subsets to bound $\sum_p \text{mass}(p)$ itself. Its proof does not directly invoke an excluded-middle case split. However, it relies on the (classical) decidability of the ordering of the reals (a default in mathlib). Representing the diffuse part on non-spatial frames remains the open frontier. The obstruction there is again that the frame has *too few points*, the same one as behind the undecidability of Section 7.

5 The Dempster–Shafer Reading

Sections 2–4 independently arrive at the same structure as Dempster–Shafer (DS) theory [Dempster 1967; Shafer 1976], including sub-additive complements, a belief/plausibility interval, and mass functions. This section makes the identification exact in both statics and dynamics. In each case, it locates the precise point where excluded middle makes a difference.

5.1 Total Monotonicity via Constructive Inclusion–Exclusion

The $\neg\neg$ -stable (*regular*) elements of a frame form a Boolean algebra with meet $\sqcap$ and join $\neg\neg(a \sqcup b)$ (Glivenko). Let us call the resulting algebra the *Booleanization* of this set.

Definition 21 (belief function, plausibility function^{Lean}). Let B be a Boolean algebra. A DS *belief function* on B is a monotone map $\text{Bel} : B \rightarrow [0, 1]$ with $\text{Bel}(\perp) = 0$ and $\text{Bel}(\top) = 1$

that is *n-monotone* for every $n \geq 2$: for all $A_1, \dots, A_n \in B$,

$$\text{Bel}\left(\bigvee_{i=1}^n A_i\right) \geq \sum_{\emptyset \neq T \subseteq \{1, \dots, n\}} (-1)^{|T|+1} \text{Bel}\left(\bigwedge_{i \in T} A_i\right).$$

Its *plausibility function* is the map $\text{Pl} : B \rightarrow [0, 1]$ given by $\text{Pl}(A) = 1 - \text{Bel}(\neg A)$.

We show v , restricted to its Booleanization, is such a belief function. The 2-monotone case is immediate from modularity^{Lean}. The full tower follows from the following result:

Theorem 22 (inclusion–exclusion^{Lean}). *For every valuation on every frame Ω , every finite family $a : \iota \rightarrow \Omega$, and every finite $s \subseteq \iota$, inclusion–exclusion holds with equality for the frame join:*

$$v\left(\bigsqcup_{i \in s} a_i\right) = \sum_{\emptyset \neq T \subseteq s} (-1)^{|T|+1} v\left(\bigsqcap_{i \in T} a_i\right).$$

We state this in Lean additively, with the odd-cardinality half of the signed sum set against the even-cardinality half, so that no $[0, \infty]$ subtraction occurs. The proof is by induction with modularity and frame distributivity.

Replacing the frame join $\bigsqcup_i a_i$ on the equality’s other side with its double negation $\neg\neg(\bigsqcup_i a_i)$, the Booleanization’s own join, yields the following inequality:

Theorem 23 (total monotonicity^{Lean}). *For every valuation on every frame Ω , every finite family $a : \iota \rightarrow \Omega$, and every finite $s \subseteq \iota$:*

$$\sum_{\emptyset \neq T \subseteq s} (-1)^{|T|+1} v\left(\bigsqcap_{i \in T} a_i\right) \leq v\left(\neg\neg\left(\bigsqcup_{i \in s} a_i\right)\right).$$

The Booleanization’s join is given by $\neg\neg \circ \sqcup$, and its meets of regular elements are again regular. Interpreted in this way, Theorem 23 states exactly the ∞ -monotone tower: v restricted to its Booleanization is a belief function.

The proof of Theorem 23 from Theorem 22 follows from the monotonicity along $a \leq \neg\neg a$. Here, inclusion–exclusion does *not* fail constructively. It is exact for the intuitionistic disjunction. Applying the double-negation nucleus (a closure operator preserving finite meets) $\neg\neg$ to the join on the equality’s right-hand side can only enlarge it. This follows from the fact that $a \leq \neg\neg a$ and that v is monotone, which turns the equality into the inequality of Theorem 23. The amount by which the right-hand side is enlarged is exactly the double-negation gap of Section 2.

Definition 21 is the classical notion, stated on an abstract Boolean algebra. In our setting each of its ingredients is already present: the Boolean algebra is the Booleanization, the belief function is v itself, restricted to the regular elements and evaluated against the Boolean join $\neg\neg \circ \sqcup$, and the plausibility function is $\text{Pl}(a) = 1 - v(\neg a)$. Additive probability does not become a belief function by weakening an axiom, since modularity for v is never relaxed. What changes is only the join used to state the tower. The belief–plausibility interval is governed by the slack:

Theorem 24 (interval width^{Lean}). *For every valuation v on a frame Ω and every $a \in \Omega$, $\text{Pl}(a) - v(a) = \text{slack}(a)$.*

5.2 The Conditioning Characterization

DS theory carries two update rules that coincide classically. *Geometric* conditioning renormalizes belief, and under the DS dictionary it is the localic posterior $v(a \sqcap b)/v(b)$ of Section 2.

Definition 25 (Dempster's rule of conditioning^{Lean}). For a valuation v on a frame Ω , elements $a, b \in \Omega$, and evidence of nonzero plausibility, $v(b) \neq 0$, *Dempster's rule of conditioning* [Shafer 1976] states that

$$v(a \parallel b) = \frac{v(a \sqcup \neg b) - v(\neg b)}{1 - v(\neg b)}.$$

The hypothesis $v(b) \neq 0$ makes the denominator nonzero, since $v(b) + v(\neg b) \leq 1$ gives $v(b) \leq 1 - v(\neg b)$.

That is, the DS conditioning rule discards the mass incompatible with the evidence and renormalizes by its plausibility. Both the geometric posterior of Section 2 and Dempster's rule of Definition 25 are again valuations. Their modularity is the frame distributivity $(a \sqcup \neg b) \sqcap (a' \sqcup \neg b) = (a \sqcap a') \sqcup \neg b$. Their disagreement is given by the *excluded-middle gap* $\text{emGap}(a, b) = v(a) - v(a \sqcap (b \sqcup \neg b))$, i.e. the mass of a outside b 's instance of excluded middle:

Lemma 26 (the join-complement identity^{Lean}). *For every valuation v on every frame Ω and every $a, b \in \Omega$, $v(a \sqcup \neg b) = v(\neg b) + v(a \sqcap b) + \text{emGap}(a, b)$.*

Dempster's numerator therefore exceeds Bayes's by exactly $\text{emGap}(a, b)$. This gap depends on a , but whether it vanishes at every a depends only on the slack at the evidence b :

Theorem 27 (the conditioning characterization^{Lean}). *For every valuation v on every frame Ω and every $b \in \Omega$ with $v(b) \neq 0$, $(v(a \parallel b) = v(a \sqcap b)/v(b) \text{ for every } a \in \Omega) \text{ iff } \text{slack}(b) = 0$.*

The hypothesis $\text{slack}(b) = 0$ eliminates the gap $\text{emGap}(a, b)$ for every a and makes the two normalizers coincide, since $1 - v(\neg b) = v(b)$. Conversely, testing at $a := b$, geometric conditioning gives certainty 1, while Dempster gives $v(b)/(1 - v(\neg b))$, which is 1 only if the slack vanishes. Theorem 27 is the valuation-relative analogue of Theorem 10: both are iff statements about the same proposition, $b \sqcup \neg b = \top$.

We call the frame-level fact *static* and the valuation-level fact *dynamic*: the former is an equation in the frame itself, between $b \sqcup \neg b$ and $\top$, with no valuation involved, while the latter tests the same proposition through v , since $\text{slack}(b) = 0$ is exactly $v(b \sqcup \neg b) = v(\top)$. Dempster and Bayes therefore coincide at b whenever v cannot distinguish $b \sqcup \neg b$ from $\top$, even if the two differ in the frame itself. In the measure

model, Dempster and Bayes disagree precisely when the evidence's boundary carries mass.

6 Cox's Theorem

Cox's theorem [Cox 1946; Jaynes 2003] derives the probability calculus from conditions on plausibility. After Halpern's counterexample [Halpern 1999], its repaired forms [Arnborg and Sjödin 2000; Horn 2003] fix the calculus up to monotone regreduation. Every known derivation is classical, and Section 3 showed why: the negation axiom R3 is excluded middle. Dropping R3 leaves the product rule unaffected, since it is logic-free, and requires the sum rule to become an axiom rather than a theorem. Separately, the naive constructive statement, demanding a regreduation strictly monotone on all of $\mathbb{R}$, is unsatisfiable (Section 6.3).

6.1 A Cox Structure without Negation

Definition 28 (Cox model^{Lean}). A *Cox model* on a frame Ω is a plausibility assignment $\text{pl} : \Omega \times \Omega \rightarrow \mathbb{R}$, written $\text{pl}(a, b)$ for the plausibility of a given b , together with a *conjunction functional* $F : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$, such that $\text{pl}(\cdot, c)$ is monotone for every c , the boundary conditions $\text{pl}(\perp, c) = 0$ and $\text{pl}(\top, c) = 1$ hold for every c , the product rule $\text{pl}(a \sqcap b, c) = F(\text{pl}(a, b \sqcap c), \text{pl}(b, c))$ holds, F is associative, and $x \mapsto F(x, y)$ is strictly monotone for every $y > 0$.

The missing R3 axiom can be observed in the omission of any constraint on the relation between $\text{pl}(\neg a, b)$ and $\text{pl}(a, b)$. The definition restricts the monotonicity of F to $0 < y$ to avoid a contradiction with the fact that the boundary axioms force $F(x, 0) = 0$ for all x . An explicit model on the non-classical chain $[0, \infty]$ satisfies this constraint^{Lean}, as $a = \top$ is provably decidable.

6.2 The Sum Rule Is Irreducible

Classically, additivity is *derived*: R3 turns disjunction into negated conjunction via De Morgan, and the product rule does the rest. Constructively that route is closed, since $a \sqcup b \neq \neg(\neg a \sqcap \neg b)$ in general:

Example 29 (the sum rule is irreducible^{Lean}). Let Ω be the five-element frame of lower sets of the poset $o < a, o < b$ with a, b incomparable. Then there is a monotone, normalized assignment q on Ω that is additive on disjoint joins yet not modular, and no binary operation S on values satisfies $q(x \sqcup y) = S(q(x), q(y))$ for all x, y . Moreover, the pairs $(\downarrow a, \downarrow a)$ and $(\downarrow a, \downarrow b)$ have identical value-pairs $(\frac{1}{2}, \frac{1}{2})$ but joins valued $\frac{1}{2}$ and 1.

Unlike for a conjunction, a disjunction's plausibility cannot generally be expressed as its disjuncts' plausibilities via conditioning. So, a constructive Cox theorem *must* posit the sum-rule half. Modularity, which says that the calculus is closed under mixing certainties, is what posits this, justified by Theorem 17. The failure is not repaired by strengthening

the logic short of Boolean: the counterexample frame satisfies weak excluded middle^{Lean}, since every nonempty lower set contains the bottom point o , so the irreducibility persists under that intermediate assumption.

6.3 Cox Model Theorem

A regratuation is a map $g : \mathbb{R} \rightarrow [0, \infty]$ with $g(0) = 0$. No such map is strictly monotone on all of $\mathbb{R}$, since strictness would force $g(-1) < g(0) = 0$, impossible in $[0, \infty]$. The repair (strictness only on $[0, 1]$, where plausibilities live) is exactly the same domain restriction that Halpern's critique of the classical theorem turns on. The corrected statement is:

Theorem 30 (the constructive Cox theorem^{Lean}). *For every frame Ω , every Cox model on Ω whose unconditional plausibility is modular regratuates to a valuation: there exist a valuation v on Ω and a strictly monotone function g on $[0, 1]$ with $g(0) = 0$, $g(1) = 1$, and for all $a \in \Omega$, $v(a) = g(\text{pl}(a, \top))$.*

The regratuation for the sum-rule half is straightforward ($g = \text{ENNReal.ofReal}$). The product-rule half, showing F regratuates to multiplication, rests on Aczél's associativity theorem [Aczél 1966], a statement of pure analysis whose type never mentions Ω . `mathlib` has no Aczél theorem, no Hölder-style embedding for ordered semigroups on intervals, and no Cauchy functional equation under monotonicity, so we build the analytic core from scratch, obtaining a normalized, strictly monotone multiplicative generator on the positive cone^{Lean}. The construction also weakens Hölder's classical hypotheses [Hölder 1901]: the Archimedean property is *derived* from continuity and positivity rather than assumed, and the interval form of divisibility follows from the intermediate value theorem^{Lean} applied to a plain divisibility hypothesis^{Lean}. The constructive/classical divide is therefore *entirely* in the sum rule: nothing in the product-rule apparatus mentions Ω . Appendix C gives the construction. The residual analytic gap is discussed in Section 14.

7 Two Grounding Models

Two concrete models demonstrate that our abstract theory is non-degenerate. One shows every classical measure *contains* an intuitionistic probability. The other shows the intuitionistic theory cannot collapse back without solving the halting problem.

7.1 Measures through the Interior Operator

Gödel–McKinsey–Tarski interpret intuitionistic logic in modal S4: propositions are *open* sets, and the modal necessity operator $\Box$ of S4 is interpreted as the topological interior, written $\text{int}(\cdot)$ below [Gödel 1933; McKinsey and Tarski 1944]. Our valuations quantify that same translation:

Theorem 31 (measures as valuations^{Lean}). *Let μ be a probability measure on a space X , $A \subseteq X$, and define $v(A) = \mu(\text{int } A)$.*

Since $\text{int } U = U$ for open U , v restricts on the frame $O(X)$ to $U \mapsto \mu(U)$, and this restriction is a valuation.

Modularity is $\mu(U \cup V) + \mu(U \cap V) = \mu(U) + \mu(V)$. The slack of an open U is then $1 - \mu(U) - \mu(\text{int } U^c)$, which is the measure of the boundary ∂U . We can now prove that, in general, slack is exactly the boundary mass:

Theorem 32 (slack is boundary mass^{Lean}). *For any probability measure μ on a topological space X and any open U , $\text{slack}(U) = \mu(\partial U)$.*

The Heyting complement of U in $O(X)$ is the interior of its set complement. So, the undecided region, the complement of $U \sqcup \neg U$, is exactly $\overline{U} \setminus U = \partial U$. The general sub-additivity inequality of Section 2 still holds here^{Lean}. This identification of slack with boundary mass also exhibits the modal identification $\text{Bel} = P(\Box \cdot)$ of the DS literature [Ruspini 1986] arising from the logic itself. Theorem 16 is the converse in the finite case (in the general case, the converse is the open problem, see Section 4).

The same decomposition locates a failure of classical probability calculus in conditioning, that is, in updating a belief about an open A on evidence given by an open B . The Boolean complement of A splits into its Heyting complement and its boundary, $A^c = \text{int } A^c \sqcup \partial A$, so a classical posterior that treats A as confirmed-or-refuted overstates the refutation by the boundary mass the evidence retains:

Theorem 33 (classical overconfidence in refutation^{Lean}). *Let μ be a probability measure on a topological space X , let v be its restriction to $O(X)$, and let A, B be opens with $\mu(B) \neq 0$. Then*

$$\frac{\mu(A \cap B)}{\mu(B)} = v(A \mid B) \quad \text{and} \quad \frac{\mu(A^c \cap B)}{\mu(B)} = v(\neg A \mid B) + \frac{\mu(\partial A \cap B)}{\mu(B)}.$$

The excess term is the conditional slack of A given B , the mass that the evidence leaves neither confirmed nor refuted. The classical posterior does not discard this mass. It credits all of it to A^c , an event no finite observation confirms. For the halting event of Section 7.3 with $B = \top$, the classical credence in non-termination is $1 - p$ while the constructive one is 0, and the difference $1 - p$ is exactly the slack.

To characterize weak excluded middle spaces, we give the (standard) definitions required:

Definition 34 (extremally disconnected). A topological space X is *extremally disconnected* if the closure of every open set is again open.

Definition 35 (Stone space, Stonean space). The *Stone space* of a Boolean algebra B is the space of ultrafilters on B , topologized by the clopen basis $\{U : b \in U\}$ for $b \in B$. *Stone duality* [Johnstone 1982] identifies Boolean algebras with exactly the compact, Hausdorff, totally disconnected spaces arising this way. A *Stonean space* is a compact Hausdorff space that is also extremally disconnected. Equivalently, it is the Stone space of a *complete* Boolean algebra.

Remark 36 (weak excluded middle on opens^{Lean}). Section 2 noted that imposing weak excluded middle on the whole frame collapses slack to pure double-negation instability. For $\Omega = O(X)$, this global condition holds exactly when X is extremally disconnected [Johnstone 1980].

An immediate corollary gives the characterization:

Corollary 37 (Stonean characterization^{Lean}). *Given a compact Hausdorff space X , weak excluded middle on $O(X)$ holds exactly when X is a Stonean space.*

7.2 Slack Does Not Determine the Valuation

Submodularity (Theorem 7) is a necessary condition on slack. It is not sufficient: distinct valuations, each satisfying every axiom a valuation must satisfy, can share a slack function.

Example 38 (boundaries in the three-point space^{Lean}). Let $\text{ThreePt} = \{p_A, p_B, p_Z\}$ carry the topology generated by the two singletons $\{p_A\}$ and $\{p_B\}$, so its opens are exactly $\emptyset$, $\{p_A\}$, $\{p_B\}$, $\{p_A, p_B\}$, and the whole space. For every open U , $\partial U \subseteq \{p_Z\}$.

This is a fact about the topology alone, independent of any measure: p_Z has no proper open neighbourhood, so it is the only point any open's frontier can contain.

Example 39 (non-recovery^{Lean}). There exist valuations $v \neq v'$ on $O(\text{ThreePt})$ whose slack functions are identical: $\text{slack}_v = \text{slack}_{v'}$ as functions.

For weights $a, b, c \geq 0$ with $a + b + c = 1$, the point measure $\mu_{a,b,c} = a \delta_{p_A} + b \delta_{p_B} + c \delta_{p_Z}$, read through Section 7.1's bridge, gives a valuation $v_{a,b,c}$ on $O(\text{ThreePt})$. By Example 38 and Theorem 32, its slack at an open U is $\mu_{a,b,c}(\partial U)$, equal to c when $p_Z \in \partial U$ and to 0 otherwise: a quantity depending only on c , not on how the remaining mass splits between a and b . So $v_{1/4,1/4,1/2}$ and $v_{1/8,3/8,1/2}$ have identical slack functions while disagreeing at $\{p_A\}$, where they take the values $1/4$ and $1/8$. Both are fully modular, monotone, and normalized: nothing in Theorem 7 or its proof distinguishes them.

Weak excluded middle does not rescue recoverability. If anything, the fully classical case is the worst one for it. When X is extremally disconnected, the De Morgan gap vanishes (Section 7.1) and slack degenerates to the double-negation gap alone, but on a space where excluded middle holds outright that gap vanishes too, so slack is identically 0 for every valuation. The zero slack function is then shared by every probability measure on X , which is the most extreme failure of recovery available.

7.3 The Computability Guard

The theory developed here is non-degenerate only if there are spaces whose slack does not vanish identically. Computability supplies them:

Definition 40 (Sierpiński topology, semidecidable proposition). The *Sierpiński space* $\mathbb{S}$ is the type Prop of propositions

carrying the topology with exactly three opens: $\emptyset$, the set $\text{halts} = \{q \mid q\}$ of true propositions, and $\mathbb{S}$ itself [Escardó 2004; Rosolini 1986]. A proposition is *semidecidable* if it is an open in this topology.

The topology encodes verifiability. A point of $\mathbb{S}$ is a proposition, say “this computation halts.” The one nontrivial observation available is membership in halts , established by producing a proof (running the computation to termination). No open captures the complementary observation, and on the frame $O(\mathbb{S}) = \{\perp, \text{halts}, \top\}$ that Σ_1 asymmetry is a theorem:

Theorem 41 (the Σ_1 asymmetry^{Lean}). $\neg \text{halts} = \perp$.

Every nonempty open of $\mathbb{S}$ contains True, so $\perp$ is the only open disjoint from halts . Thus, no nonempty open of $\mathbb{S}$ is evidence that the computation does not halt.

We now instantiate the measure-to-valuation construction of Theorem 31 at $X = \mathbb{S}$, allowing us to compute the slack:

Example 42 (halting slack^{Lean}). Let $\mu = p \cdot \delta_{\text{True}} + (1-p) \cdot \delta_{\text{False}}$, and let v be the resulting valuation. Then

$$\text{slack}(\text{halts}) = 1 - p = \mu(\partial \{q \mid q\}) = \mu\{\text{False}\}.$$

Since halts is already open, $v(\text{halts}) = \mu(\text{halts}) = p$. The DS “ignorance” of Section 5 is here the mass of the single point False, the one outcome no finite observation can ever resolve. This probability is a fact about the model, not about anyone's ignorance.

Chaitin's Ω . The same bridge scales to algorithmic information theory. Fix a universal prefix-free machine U , *prefix-free* in that no halting program of U is a proper prefix of another, and *universal* in that it simulates every other prefix-free machine. Give 2^ω , the space of infinite bit strings, the fair-coin measure μ . The event H of having a halting program of U as a prefix is open, since halting is established by a finite prefix (Definition 40), and prefix-freeness makes H a disjoint union of cylinders, so $v(H) = \mu(H) = \sum_{p \text{ halts}} 2^{-|p|}$: exactly Chaitin's halting probability Ω [Chaitin 1975]. Its slack $\mu(\partial H)$ is the mass of strings approximated arbitrarily well by halting strings while never having a halting program as a prefix, and Ω 's classical uncomputability is the aggregate form of the pointwise obstruction above. The measure-theoretic core is mechanized on Cantor space with mathlib's infinite product measure: basic cylinders are clopen with measure $2^{-|w|}$ ^{Lean}, and a *finite* prefix-free set of words has pairwise disjoint cylinders of total measure $\sum_w 2^{-|w|}$ ^{Lean}, the Kraft-inequality content underlying Ω . The universal machine itself, and the countable extension of the finite Kraft sum, are computability theory beyond our scope.

Definition 43 (halting problem). For a code c and input n , the *halting problem* is the predicate $H(c, n) =$ “the machine with code c halts on input n ,” (ComputablePred).

Slack cannot be eliminated by simply declaring a classical answer for each code. The family of valuations defined below attempts to do this:

Definition 44 (sharp classical readout^{Lean}). For a given n , the *sharp classical readout* is the family of $\{0, 1\}$ -valued valuations $\rho_n(c) \in \text{Valuation}(O(\mathbb{S}))$, with $\rho_n(c)(\text{halts}) = 1$ if $H(c, n)$ holds, and $\rho_n(c)(\text{halts}) = 0$ otherwise.

The readout is sharp but not slack-free. Since $\neg \text{halts} = \perp$ (Theorem 41), a code that does not halt receives slack $1 - 0 = 1$, the maximum. Sharp here means that the classical answer is known, not that the complement rule holds on $O(\mathbb{S})$.

However, this family is provably not computable:

Theorem 45 (the sharp readout is uncomputable^{Lean}). For every n , the predicate $c \mapsto (\rho_n(c)(\text{halts}) = 1)$ is not computable.

The proof identifies the predicate with $H(\cdot, n)$ and applies the undecidability of the halting problem (ComputablePred.halting_problem).

No route back to a classical, decidable treatment survives contact with computability. Excluded middle guarantees that $H(c, n)$ is true or false for any fixed c , but that guarantee is free: it names no algorithm and exhibits no witness for which disjunct holds. Demanding an actual witness, uniformly in c , is exactly ρ_n 's sharp readout, and Theorem 45 shows no algorithm computes it. Aggregating over the whole family of halting programs, rather than reading off one instance, does not help either: Ω is a perfectly well-defined classical probability, yet Chaitin showed it uncomputable by an independent argument turning on algorithmic randomness [Chaitin 1975]. Neither the pointwise nor the aggregate route gives classical probability a way to compute a solution to the halting problem, and each is blocked by its own uncomputability result.

We can conclude that slack is not a defect of the constructive theory. On $O(\mathbb{S})$, forcing the complement rule at *halts* leaves the single value $v(\text{halts}) = 1$, so every code is assigned certain termination. That assignment is computable but false of every non-halting code. Any assignment that avoids this error, giving $v(\text{halts}) < 1$ to each non-halting code, has positive slack at every such code, and the one that is moreover sharp is the uncomputable readout of Theorem 45. For the sum rule, this guard plays the role the explicit Cox-model instance plays for the product rule (Section 6): each axiom set in the paper comes with non-trivial models satisfying it.

8 Product Valuations and Kernel Composition

The structure through which classical probability supports sequential statistical reasoning is the Giry monad:

Definition 46 (Giry monad [Giry 1982]). The *Giry monad* G sends a measurable space X to the space $G(X)$ of probability

measures on X . Its unit sends a point $x \in X$ to the point mass at x , and its multiplication sends a measure M on $G(X)$ to the measure $A \mapsto \int m(A) dM(m)$.

Probabilistic reasoning then composes in stages, and, when the monad is commutative, independent stages compose in either order. The question is whether the intuitionistic calculus supports the same structure.

A *frame homomorphism* is a map preserving finite meets and arbitrary joins. Pushforward along a frame homomorphism $f : \Omega \rightarrow \Omega'$ sends a valuation v on Ω' to $a \mapsto v(f(a))$ on Ω , and this preserves the valuation axioms, making $\text{Val}(-)$ a functor on locales, the localic counterpart of the functor part of the Giry monad [Giry 1982]. A frame homomorphism is the opposite of a locale map, so the functor laws are checked against identity and composition of frame homomorphisms^{Lean}.

Definition 47 (valued frames^{Lean}). The *valuation functor* $\text{Val} : \text{Frm}^{\text{op}} \rightarrow \text{Set}$ sends a frame to its set of valuations and a frame homomorphism $\varphi : P \rightarrow Q$ to $w \mapsto w \circ \varphi$. The category SpVal of *valued frames* is its category of elements: objects are pairs (P, v) with v a valuation on P , and a morphism $(P, v) \rightarrow (Q, w)$ is a frame homomorphism $\varphi : P \rightarrow Q$ with $w \circ \varphi = v$.

A morphism of valued frames is a measure-preserving map of locales in the opposite direction. The point valuations δ_p (Section 4) provide the candidate (monad) unit. Theorem 16 shows that, on finite frames, Val agrees (on objects) with the classical finite-distribution monad on points. Flattening a valuation of valuations is integration, and integration in two variables requires product valuations. A *kernel* is a map assigning a valuation to each point of a locale, giving a conditional distribution. By Kock's identification of commutative monads with Fubini theorems [Kock 2012], kernels compose independently of order exactly when Val is a commutative monad.

A valuation u on a product frame is a *product* of valuations v and w when $u(A \times B) = v(A) \cdot w(B)$ for all rectangles $A \times B$. Let $\text{Low}(\mathbb{N} \times \mathbb{N})$ be the frame of lower sets of the grid $\mathbb{N} \times \mathbb{N}$, ordered component-wise. On this frame, products are unique:

Theorem 48 (product uniqueness^{Lean}). Let v, w be valuations on $\text{Low}(\mathbb{N})$. Any two valuations on $\text{Low}(\mathbb{N} \times \mathbb{N})$ that are products of v and w are equal.

By the inclusion-exclusion equality, two valuations agreeing on a $\sqcap$ -closed family agree on all finite joins of its members^{Lean}. The rectangles are closed under $\sqcap$, and in $\text{Low}(\mathbb{N} \times \mathbb{N})$ every lower set is a *finite* union of rectangles^{Lean}, so agreement on rectangles is agreement everywhere. Valuations carry no continuity axiom (Section 4), so the σ -additivity driving the classical uniqueness argument for product measures has no analogue here. The order structure of the frame does that work instead.

Theorem 49 (product non-uniqueness^{Lean}). *On the powerset frame $\text{Set}(\mathbb{N} \times \mathbb{N})$ there exist valuations v, w and two distinct products of v and w .*

An *ultrafilter* on $\mathbb{N}$ is a collection of subsets, closed under intersection and superset, containing exactly one of S and its complement for every $S \subseteq \mathbb{N}$. It is *free* when it contains no finite set. The map assigning 1 to the members of an ultrafilter and 0 to every other subset is a valuation^{Lean}. A rectangle $S \times T$ belongs to an ultrafilter on the product exactly when S and T belong to its images under the two coordinate projections^{Lean}. A free ultrafilter $\mathcal{U}$ on $\mathbb{N}$ yields two ultrafilters on $\mathbb{N} \times \mathbb{N}$. A set belongs to the former when, for $\mathcal{U}$ -most first coordinates, its slice contains $\mathcal{U}$ -most second coordinates. A set belongs to the latter under the same test with the coordinates in the opposite order.

The two ultrafilters' valuations agree on every rectangle yet differ on the upper triangle $\{(m, m') \mid m < m'\}$ ^{Lean}. Integrating the same marginal data in the two orders thus disagrees. Fubini fails for valuations, as it does for non-additive integration [Denneberg 1994; Ghirardato 1997]. By Kock's correspondence, no canonical commutative multiplication exists for unrestricted valuations. This half of the dichotomy is not specific to intuitionism, since powerset-frame valuations are exactly the classical finitely additive probability measures [Rao and Rao 1983]. Dropping continuity is what admits the non-uniqueness. A monad therefore requires restricting either the frames or the valuations.

A first step in the second direction is to restrict Val to the valuations given as countable mixtures. For these, we can demonstrate that the kernels compose independently of order, the property that Theorem 49 refutes for unrestricted valuations:

- Proposition 50** (countable kernel composition^{Lean}). 1. *A countable mixture of valuations is a valuation^{Lean}.*
 2. *A mixture of mixtures flattens to a mixture with product weights, which is discrete Chapman–Kolmogorov^{Lean}.*
 3. *The one-point mixture is the identity^{Lean}.*
 4. *Iterated countable mixing is order-independent^{Lean}.*

Restricted to countable-mixture presentations, the data of a monad are all in place: the category of locales, Val restricted to countable mixtures as the functor, the point valuations as the unit, and countable mixing as the multiplication. Proposition 50 verifies exactly the laws this needs: item 1 keeps the multiplication inside the fragment, item 2 is its associativity, item 3 its unit law, and item 4 its commutativity. What is not yet settled is whether this descends to valuations themselves, since a valuation can admit more than one countable-mixture presentation, and the diffuse part of Section 4 is not captured by any countable presentation at all.

9 Valuations Beyond Frames

Definition 1 fixes the event algebra to be a frame: a complete lattice in which finite meets distribute over arbitrary joins $\bigsqcup_{i \in I} a_i$, the operation Heyting implication $\Rightarrow$ (the right adjoint to $a \sqcap (-)$) is built from Definition 2's four axioms, normalization at $\perp$ and $\top$, monotonicity, and modularity $v(a) + v(b) = v(a \sqcup b) + v(a \sqcap b)$, use only the *binary* join $\sqcup$. None of the four mentions the *arbitrary* join $\bigsqcup_{i \in I}$. Completeness is used only where later sections evaluate a valuation on an infinite join, as in the Scott-continuity results of Section 4, or build $\Rightarrow$ and $\neg$ from it, as in Section 3's slack. Section 14 leaves open how much of the construction is tied to frames specifically, and whether some ambient category sends an event algebra to a probability calculus in general. This section settles both questions for one fragment of the construction, the sharp/point correspondence of Section 3: it isolates the exact algebraic hypothesis that correspondence needs and shows the hypothesis cannot be dropped. It does not recover $\Rightarrow$ or $\neg$ without completeness: the adjoint defining $\Rightarrow$ is built from an arbitrary join of witnesses, so it is unavailable once completeness goes, even if only finite joins are missing. The valuations studied below are negation-free by construction, not frame valuations with completeness quietly removed.

The functor without completeness. A *bounded lattice* is a lattice with a least element $\perp$ and a greatest element $\top$. A *bounded lattice homomorphism* preserves $\perp$, $\top$, $\sqcup$, and $\sqcap$. Definition 2's axioms make sense over any bounded lattice Ω , complete or not:

Definition 51 (lattice valuation^{Lean}). A *lattice valuation* on a bounded lattice Ω is a monotone function $v : \Omega \rightarrow [0, \infty]$ with $v(\perp) = 0$, $v(\top) = 1$, and, for all $a, b \in \Omega$, $v(a) + v(b) = v(a \sqcup b) + v(a \sqcap b)$.

```
structure LValuation (Ω : Type*) [Lattice
  Ω] [BoundedOrder Ω] where
  toFun : Ω → ℝ≥0∞
  map_bot' : toFun ⊥ = 0
  map_top' : toFun ⊤ = 1
  mono' : Monotone toFun
  modular' : ∀ a b, toFun a + toFun b
              = toFun (a ⊔ b) + toFun (a ⊓ b)
```

A lattice valuation on a frame is exactly a valuation in the sense of Definition 2. The two notions differ only in how much structure the ambient Ω carries.

Section 8 pulls a valuation back along a frame homomorphism and checks the assignment is functorial (identity, composition). That argument pattern-matches on $\sqcup$, $\sqcap$, $\perp$, $\top$ alone and never touches $\sqcup$:

Proposition 52 (pullback without completeness^{Lean}). *For bounded lattices Γ, Ω , a bounded lattice homomorphism $f :$*

$\Gamma \rightarrow \Omega$, and a lattice valuation v on Ω , $v \circ f$ is a lattice valuation on Γ . This assignment sends the identity homomorphism to the identity and composition of homomorphisms to composition of the induced maps.

$\text{Val}(-)$ is therefore a functor on bounded lattices with bounded lattice homomorphisms, of which the frame-level functor of Section 8 is the restriction to the subcategory of frames and frame homomorphisms. Completeness contributes nothing to functoriality.

The valuation monoid. The modularity axiom can be moved from the valuation into its domain.

Definition 53 (valuation monoid^{Lean}). For a lattice P with least element $\perp$, $V(P)$ is the free commutative monoid on P modulo the congruence generated by $[a] + [b] \sim [a \sqcup b] + [a \sqcap b]$ for all a, b and $[\perp] \sim 0$, where $[a]$ is the class of the generator a .

Theorem 54 (universal property^{Lean}). For every commutative monoid M , the maps $f : P \rightarrow M$ with $f(\perp) = 0$ and $f(a) + f(b) = f(a \sqcup b) + f(a \sqcap b)$ are exactly the composites of $a \mapsto [a]$ with monoid homomorphisms $V(P) \rightarrow M$, and the homomorphism is unique. On a bounded lattice, the lattice valuations are therefore the monoid homomorphisms $h : V(P) \rightarrow [0, \infty]$ with $h([\top]) = 1$ and $a \mapsto h([a])$ monotone.

So modularity becomes a property of the domain $V(P)$, and normalization and monotonicity are conditions on the homomorphism.

Theorem 55 (flags^{Lean}). In every lattice with a least element, every element of $V(P)$ equals $[\alpha_1] + \dots + [\alpha_n]$ for some chain $\alpha_1 \leq \dots \leq \alpha_n$.

The proof inserts one generator at a time into a chain $c_1 \leq \dots \leq c_n$: $[x] + [c_1] = [x \sqcap c_1] + [x \sqcup c_1]$, and $x \sqcup c_1$ is inserted into $c_2 \leq \dots \leq c_n$. Every common lower bound of x and the chain lies below every element of the result, which keeps the result a chain. Distributivity is not used. On a finite distributive lattice $\text{Low}(Q)$ the chain is unique up to copies of $\perp$, and $V(P)$ is cancellative: evaluating against the point valuations δ_q gives the number $k(q)$ of elements of the chain containing q , the j -th largest element of the chain is $\{q \mid k(q) \geq j\}$, and $x \mapsto k$ is an injective homomorphism into $\mathbb{N}^Q$. This argument is not formalized. Without distributivity both can fail.

Proposition 56 (flags at M_3 ^{Lean}). In $V(M_3)$, for the lattice of Definition 59, $[a] + [o] = [c] + [o]$, and $[a] + [b] = [a] + [c]$ while $[b] \neq [c]$. So the chain of Theorem 55 is not unique, and $V(M_3)$ is not cancellative.

Both equations follow from $[x] + [y] = [o]$ for distinct atoms x, y . The map sending z and c to 0 and a, b, o to ∞ satisfies the hypotheses of Theorem 54 and separates $[b]$ from $[c]$.

Separation needs only distributivity. Theorem 12 characterizes what a sharp valuation is, a complement-indicator of a prime ideal, but not whether there are enough of them to distinguish elements of Ω . That is a separate fact:

Theorem 57 (sharp valuations separate points^{Lean}). For every frame Ω and $a, b \in \Omega$ with $a \not\leq b$, there is a sharp valuation v with $v(a) = 1$ and $v(b) = 0$.

The proof runs the same prime-ideal separation theorem that built the counterexample of Theorem 10 (`DistribLattice.prime_ideal_of_disjoint_filter_ideal`) on the principal filter of a and the principal ideal of b rather than on $a \sqcup \neg a$ and $\top$. As a corollary, any two distinct $a, b \in \Omega$ are distinguished by some sharp valuation: Ω need not be linear, so $a \neq b$ alone does not hand back $a \leq b$ or $b \leq a$, but it forces at least one failure of $\leq$, and Theorem 57 applies in whichever direction fails. This is the cash-out of the first qualifying condition of Section 14: the certain valuations do not merely characterize a single point of the locale in isolation, together they reconstruct the order on Ω . Reconstructing the order is not the same as reconstructing the full Heyting structure: $\neg$ and $\Rightarrow$, as just noted, are adjoints built from an arbitrary join and carry no meaning over a plain distributive lattice. What this theorem recovers is the order-theoretic core of the first qualifying condition, not the negation fragment of intuitionistic logic, which stays frame-specific.

Tracing the proof of Theorem 57 shows it never invokes `⊔`. The only place frame structure enters is `DistribLattice.prime_ideal_of_disjoint_filter_ideal` itself, which needs distributivity and nothing beyond it:

Theorem 58 (separation over distributive lattices^{Lean}). For every distributive bounded lattice Ω and $a, b \in \Omega$ with $a \not\leq b$, there is a sharp lattice valuation v on Ω with $v(a) = 1$ and $v(b) = 0$.

Theorem 58 reproves Theorem 57 with the hypothesis `Order.Frame` weakened to `DistribLattice`, and nothing else about the argument changes. The condition doing the work in the sharp/point correspondence is distributivity, and completeness is not load-bearing for either half of it.

Distributivity is not a proof artifact. The natural question is whether Theorem 58's hypothesis is a genuine dividing line or an artifact of routing the argument through a mathlib lemma named for distributive lattices. It is genuine, and the failure it guards against is total, not merely a failure of separation:

Definition 59 (the diamond^{Lean}). M_3 is the five-element lattice with a bottom z , a top o , and three pairwise-incomparable atoms a, b, c , with $x \sqcup y = o$ and $x \sqcap y = z$ for every two distinct atoms x, y .

M_3 is the smallest lattice that is not distributive: $a \sqcap (b \sqcup c) = a \sqcap o = a$, while $(a \sqcap b) \sqcup (a \sqcap c) = z \sqcup z = z$. It is modular,

so this failure is specific to distributivity rather than some coarser defect of the order.

Theorem 60 (no sharp valuation on the diamond^{Lean}). *No lattice valuation on M_3 is sharp.*

Modularity applied to each of the three pairs of distinct atoms forces $v(x) + v(y) = 1$, since $x \sqcup y = \top$ and $x \sqcap y = \perp$. If v were sharp, $v(a), v(b), v(c) \in \{0, 1\}$ would have to satisfy all three pairwise sum-to-one equations at once, and a two-element codomain cannot support that: fixing any one value propagates through the other two equations to a contradiction with the third. The collapse on M_3 is therefore not a failure of separation, points that happen to coincide, but a failure of existence: there is no all-or-nothing valuation on M_3 to separate anything with, and the first qualifying condition of Section 14 is unsatisfiable outright.

Where this leaves the boundary. Frames are complete distributive lattices. Theorem 58 shows completeness is not needed for the sharp/point correspondence's separation half, and Theorem 60 shows distributivity cannot be given up in exchange. The event algebras on which sharp valuations reconstruct the order, the order-theoretic core of the first qualifying condition of Section 14, are exactly the distributive bounded lattices, no more and no less. The second qualifying condition and the negation-dependent machinery of Sections 3–7, slack, the complement bound, the De Morgan gap, are untouched by this section and remain tied to frames: nothing here shows, or claims, that $\neg$ or $\Rightarrow$ survives dropping completeness. This bears on the orthomodular event lattices of quantum probability discussed in Section 13: giving up distributivity, as those lattices do, does not merely change which valuations exist, it can remove sharp valuations altogether, so the construction of this paper does not, without further modification, have anything to say about that setting even at the order-theoretic level examined here. Whether some weaker notion of certainty survives on non-distributive lattices, in place of the $\{0, 1\}$ -valued sharp valuations examined here, is not addressed by this section.

A second boundary: grading a non-idempotent tensor. Section 13 notes that linear logic's quantale tensor $\otimes$ is not the lattice meet $\sqcap$ this section's valuations are built from, so Definition 51's modularity axiom does not even parse for $\otimes$ without first deciding what to write in $\sqcap$'s place. Granting the substitution provisionally, to see what it costs, shows the obstruction runs deeper than a notational mismatch:

Definition 61 (tensor valuation^{Lean}). Fix a join-semilattice Ω and a binary operation $\otimes$ on Ω . A *tensor valuation* for $\otimes$ is a function $v : \Omega \rightarrow [0, \infty]$ such that, for all $a, b \in \Omega$, $v(a) + v(b) = v(a \sqcup b) + v(a \otimes b)$.

Theorem 62 (the transplant collapses^{Lean}). *For every join-semilattice Ω , binary operation $\otimes$ on Ω , and tensor valuation v for $\otimes$, if $v(a) < \infty$ then $v(a \otimes a) = v(a)$.*

Setting $b := a$ in the defining identity and using only $a \sqcup a = a$, join-idempotence, already gives $v(a) + v(a) = v(a) + v(a \otimes a)$, hence the claim by cancellation. No property of $\otimes$ is used: not associativity, not a unit, not distributivity over $\sqcup$. This is a sharper failure than Theorem 60's: M_3 shows meet-based modularity can fail to admit a sharp valuation on one uncooperative lattice, while Theorem 62 shows the $\otimes$ -based transplant admits no valuation separating a from $a \otimes a$ on *any* Ω and for *any* $\otimes$, idempotent or not. A non-idempotent $\otimes$, linear logic's whole point, is exactly what such a valuation could never distinguish from $\sqcap$'s behavior.

Example 63 (a concrete witness^{Lean}). Take $\Omega = \mathbb{N}$ with $\sqcup = \max$ and $\otimes = +$: addition is associative and commutative but not idempotent ($1 + 1 = 2 \neq 1$), so the hypothesis of Theorem 62 is not vacuous. Substituting $a = 0$, where the join's unit and the tensor's unit coincide, forces $v(n) = v(0)$ for every n : every tensor valuation for $+$ is constant.

This is not a toy chosen merely by analogy with linear logic. Every quantale, the event algebra Section 13 attaches to linear logic, is by definition a complete lattice equipped with an associative $\otimes$ that need not be idempotent, exactly the data $(\Omega, \sqcup, \otimes)$ Theorem 62 quantifies over, with the quantale's own join playing the role of $\sqcup$. Example 63 is a quantale itself ($+$ distributes over $\max$), kept only because it is small enough to check by hand: it fails idempotence for the same reason $\otimes$ does, no algebraic law identifies two copies of a resource with one.

Corollary 64 (grading and discrimination are exclusive^{Lean}). *For every join-semilattice Ω , binary operation $\otimes$ on Ω , and $a \in \Omega$ with $v(a) < \infty$: no v is both a tensor valuation for $\otimes$ and satisfies $v(a \otimes a) \neq v(a)$.*

This restates Theorem 62 as the dichotomy it really is. A candidate probabilistic semantics for a non-idempotent $\otimes$ must choose between being graded by Definition 51's compositional law and discriminating the one thing a non-idempotent $\otimes$ exists to discriminate, a from $a \otimes a$. It cannot keep both. The probabilistic semantics that does exist for linear logic, probabilistic coherence spaces [Danos and Ehrhard 2011], keeps discrimination: two independent copies of a resource are not scored as interchangeable with one. Corollary 64 shows the price of that choice is forced, not chosen: probabilistic coherence spaces cannot also be graded by this paper's modularity law, on pain of losing exactly the discrimination they exist to keep, so they instead evaluate proofs with a classical, Kolmogorov-style structure over $\otimes$'s type-level behavior, never a valuation of propositions in Definition 51's sense. Nothing here is a missed opportunity on either side. The dichotomy forecloses the combination before any further axiom of Definition 51 is even reached. Section 10 replaces these axioms by a construction that applies to linear logic.

10 Probability Calculi from Certain States

Corollary 64 shows that linear logic admits no grading by Definition 51's axioms that distinguishes a from $a \otimes a$. The obstruction is in the modularity axiom, which uses the lattice operations of the event algebra. This section uses instead a construction that needs no lattice operations: the calculus of a logic is the set of mixtures of its certain states, its $\{0, 1\}$ -valued assignments. For classical logic this is de Finetti's characterization of coherent probability assignments [de Finetti 1974], and Paris [Paris 2001] and Williams [Williams 2012] use it for non-classical logics, where it is known as generalized probabilism. On finite frames, and in particular on finite Boolean algebras, it returns exactly the valuations of Definition 2, an instance of Paris's theorem for logics whose certain states respect $\wedge$ and $\vee$ [Paris 2001]. Linear logic is not such a logic. For every formula of multiplicative-additive linear logic over finite data types, its output is contained in the probabilistic coherence space (Definition 71) of the formula, and the certain elements of that space are cliques of Girard's coherence space. At implications between data types it returns the probabilistic coherence space. At the exponential $! \text{Bool}$ it returns a strict subset.

10.1 The Construction

Definition 65 (presentation by certain states, mixtures^{Lean}). A *presentation* is a set E of events together with a set S of $\{0, 1\}$ -valued functions $E \rightarrow [0, \infty]$, the *certain states*. Its *calculus* $\text{Mix}(S)$ is the set of finite convex combinations $\sum_i w_i s_i$ with $s_i \in S$, $w_i \in [0, \infty]$, and $\sum_i w_i = 1$.

For classical logic the certain states are the two-valued models, for intuitionistic logic on a frame they are the sharp valuations (Theorem 12), and for a type of linear logic they are the deterministic programs of that type (Section 10.2). Two properties hold for every presentation.

Theorem 66 (certain elements and mixing^{LeanLean}). *For every presentation (E, S) : (i) a $\{0, 1\}$ -valued x lies in $\text{Mix}(S)$ iff $x \in S$, and (ii) $\text{Mix}(\text{Mix}(S)) = \text{Mix}(S)$.*

Part (i) is the first condition of Section 1, that the $\{0, 1\}$ -valued elements of the calculus reconstruct the logic. In a presentation the logic is given by S , so the condition says that mixing creates no new $\{0, 1\}$ -valued elements. The proof: if $x = \sum_i w_i s_i$ is $\{0, 1\}$ -valued and $w_j \neq 0$, then at each e with $x_e = 0$ every s_i with $w_i \neq 0$ has $s_i(e) = 0$, and at each e with $x_e = 1$ every such s_i has $s_i(e) = 1$, since $\sum_i w_i = 1$ and $s_i(e) \leq 1$. So $s_j = x$, and $x \in S$. Part (ii) states that a convex combination of elements of $\text{Mix}(S)$ is again in $\text{Mix}(S)$. For valuations on a frame this is the closure of valuations under mixing of Section 4^{Lean}. The second condition of Section 1, that the classical case returns the classical theory, is Theorem 69 below.

The three axioms of Definition 2 are instances of one form:

Definition 67 (affine law^{Lean}). For finite E , an *affine law* is given by $a, b : E \rightarrow [0, \infty]$ and $k, k' \in [0, \infty]$. A function $x : E \rightarrow [0, \infty]$ *satisfies* it if $\sum_e a_e x_e + k \leq \sum_e b_e x_e + k'$.

Monotonicity $x_a \leq x_b$, normalization ($x_\top \leq 1$ and $1 \leq x_\bot$, $x_\perp \leq 0$), and modularity (two opposite inequalities) are affine laws.

Theorem 68 (completeness^{Lean}). *For finite E and every set S of $\{0, 1\}$ -valued functions on E , $\text{Mix}(S)$ is the set of functions satisfying every affine law that every $s \in S$ satisfies.*

The inclusion from left to right holds because affine laws are preserved by convex combinations^{Lean}. The converse separates a point outside the convex hull of S by a hyperplane, using mathlib's geometric Hahn–Banach theorem, and reads the hyperplane off as an affine law. For rational points and integer certain states, Section 3 gives a proof without choice by Fourier–Motzkin elimination^{Lean}. Theorem 68 is the generalization of the three axioms: the axioms of the calculus of a presentation are the affine laws valid on its certain states, and nothing else. On frames these are the three axioms of Definition 2:

Theorem 69 (frames^{Lean}). *For a finite poset P , the valuations on the frame $\text{Low}(P)$ of lower sets of P are exactly Mix of its sharp valuations.*

The proof combines Theorem 17 with the fact that a sharp valuation on $\text{Low}(P)$ is a point valuation δ_p ^{Lean}. Every finite frame is of the form $\text{Low}(P)$ (Section 4), and finite Boolean algebras are the case of a discrete P , where the valuations are the probability mass functions on P . So the construction reduces to the valuation calculus on finite intuitionistic event algebras and to Kolmogorov's calculus on finite classical ones.

For infinite E the laws are taken with finite support. A *window* is a finite set $F \subseteq E$ of events, and a *finite law* is an affine law on some window, applied to the restrictions of functions to F .

Theorem 70 (barycenters^{LeanLean}). *Let S be a set of $\{0, 1\}$ -valued functions on an arbitrary set E . For $x : E \rightarrow [0, \infty]$ the following are equivalent: (i) x satisfies every finite law valid on S . (ii) For every window F , the restriction of x to F lies in Mix of the restrictions of S to F . (iii) There is a Borel probability measure μ on $\{0, 1\}^E$, with the product topology, such that $x_e = \mu\{\sigma \mid \sigma_e = 1\}$ for every $e \in E$, and $\mu(\text{bad}_F) = 0$ for every window F , where bad_F is the set of σ whose restriction to F is the restriction of no $s \in S$.*

The equivalence of (i) and (ii), which Paris states for infinite languages [Paris 2001], applies Theorem 68 to each window. For (ii) $\Rightarrow$ (iii), the mixture on a window F gives a probability measure μ_F on $\{0, 1\}^F$, a finite convex combination of point masses at certain states. The windows are directed by inclusion. The probability measures on the compact space

$\{0, 1\}^E$ form a compact space in the weak topology², so the family $(\mu_F)_F$ has a cluster point μ . A *cylinder set* is a set of σ determined by the values of σ on a window. Cylinder sets are clopen, so $\nu \mapsto \nu(C)$ is continuous in the weak topology for every cylinder set C . The sets $\{\sigma \mid \sigma_e = 1\}$ and bad_F are cylinder sets, and μ_F has the required values on them once $e \in F$, so μ has them. For (iii) $\Rightarrow$ (i), a finite law valid on S holds at μ -almost every σ , and integrating it against μ gives the law at x .

Relation to generalized probabilism. Theorem 68 is de Finetti's theorem in the form used for non-classical logics [Paris 2001; Williams 2012]. An affine law valid on every certain state and violated at x is a finite system of bets priced by x with a net loss in every certain state, a Dutch book, so the theorem says that x admits no Dutch book iff it is a mixture of certain states. Paris proves it with a separating hyperplane, as the Lean proof does, and Williams derives from the same fact that these are the assignments not dominated in accuracy [Williams 2012]. Paris also shows that when every certain state respects $\wedge$ and $\vee$ the affine laws reduce to normalization, monotonicity, and modularity [Paris 2001], which gives Theorem 69. Theorem 70(iii) is the counterpart, for $\{0, 1\}$ -valued certain states, of the representation of the states of an MV-algebra as integrals over its valuations [Kroupa 2006; Kühr and Mundici 2007; Panti 2008]. This section adds their mechanization, a proof of Theorem 68 without choice for rational data, and their use for logics that Paris's axiomatization does not cover: for linear logic, where modularity is not available (Corollary 64), in Sections 10.2 and 11, and for quantum event logic in Section 12.

$\text{Mix}(S)$ is the least convex set containing S . Criteria (1) and (2) of Section 1 do not single it out: the probabilistic coherence space !Bool satisfies (1) and strictly contains the calculus (Theorem 73). Whether a further criterion selects $\text{Mix}(S)$, and how the criteria should treat logics without certain states (Section 12), is open.

10.2 Linear Logic

Definition 71 (probabilistic coherence space [Danos and Ehrhard 2011]^{Lean}). For a set W and a set Q of functions $W \rightarrow [0, \infty]$, let $Q^\perp = \{y : W \rightarrow [0, \infty] \mid \sum_w x_w y_w \leq 1 \text{ for all } x \in Q\}$. A *probabilistic coherence space* (PCS) with *web* W is a set P of functions $W \rightarrow [0, \infty]$ such that $P = P^{\perp\perp}$, every $w \in W$ has some $x \in P$ with $x_w \neq 0$ (totality), and $\sup_{x \in P} x_w < \infty$ for every $w \in W$ (boundedness).

On a finite web, $P = P^{\perp\perp}$ holds iff P contains 0 and is closed, convex, and downward closed, by Girard's bipolar theorem [Girard 2004]. The map $Q \mapsto Q^\perp$ is the antiblocking duality of polyhedral combinatorics [Fulkerson 1971], which Section 11.4 uses.

²In mathlib this is an instance proved from Prokhorov's theorem.

We use finite webs. For a finite set X , the *data type* X is the PCS of $x : X \rightarrow [0, \infty]$ with $\sum_a x_a \leq 1$. The PCS $X \multimap Y$ on $X \times Y$ is the set of t with $\sum_b \sum_a x_a t(a, b) \leq 1$ for every x in X , equivalently $\sum_b t(a, b) \leq 1$ for every a ^{Lean}. Both satisfy the three conditions^{Lean}. A *coherence space* [Girard 1987] is a set with a reflexive symmetric relation, its coherence, and a *clique* is a set of pairwise coherent elements. In the coherence space $X \multimap Y$ of two data types, two points (a, b) and (a', b') cohere iff they are equal or $a \neq a'$.

Probabilistic coherence spaces interpret every formula of linear logic, including the exponentials, and model probabilistic PCF [Danos and Ehrhard 2011]. The construction of Definition 65 applies to every such type, with the $\{0, 1\}$ -valued elements of its probabilistic coherence space as certain states. At first order the two agree (Theorem 72), and the exponential !Bool separates them (Theorem 73). Section 10.3 proves the facts that hold at every multiplicative-additive formula, and Section 11.2 shows that the two differ at every formula with a negative occurrence of a second-order core. Whether they agree at the remaining types is open.

Theorem 72 (first order^{LeanLean}). $X \multimap Y = \text{Mix}\{\det f \mid f : X \rightarrow Y_\perp\}$, where $\det f(a, b) = 1$ if $f(a) = b$ and 0 otherwise, and $Y_\perp$ adds a divergent value. A $\{0, 1\}$ -valued t lies in $X \multimap Y$ iff its support is a clique of the coherence space $X \multimap Y$.

At first order the certain states are the deterministic partial programs, they are the cliques of the coherence-space model, and the calculus is the PCS.

The PCS !Bool has web the finite multisets over $\{t, f\}$, and is the biorthogonal closure of the promotions $x^i(i, j) = x_t^i x_f^j$ of the elements x of !Bool , where (i, j) is the multiset with i copies of t and j copies of f .

Theorem 73 (exponential^{LeanLean}). Every $\{0, 1\}$ -valued element of !Bool vanishes at every (i, j) with $i, j \geq 1$. The promotion of $x = (\frac{1}{2}, \frac{1}{2})$ takes the value $\frac{1}{4}$ at $(1, 1)$, so it lies in !Bool and not in Mix of the $\{0, 1\}$ -valued elements.

The first statement uses the dual element $4\delta_{(i,j)}$, which is admissible because $x_t + x_f \leq 1$ implies $4x_t x_f \leq 1$. The $\{0, 1\}$ -valued elements are therefore supported on the multisets whose underlying set is a clique of !Bool . These underlying sets are the points of the web of !Bool in Girard's coherence spaces. Danos and Ehrhard [Danos and Ehrhard 2011] observe that elements of the PCS !Bool are nonzero on multisets whose underlying set is not a clique. Theorem 73 shows that such elements are not mixtures of certain elements.

10.3 All Multiplicative-Additive Formulas

Definition 74 (formulas and their semantics^{Lean}). The formulas are generated from the data types $\mathbf{n}$, for $n \in \mathbb{N}$, by linear negation $A^\perp$, $A \otimes B$, and $A \& B$. Write $A \wp B = (A^\perp \otimes B^\perp)^\perp$, $A \oplus B = (A^\perp \& B^\perp)^\perp$, and $A \multimap B = (A \otimes B^\perp)^\perp$. The web $|A|$ is $\{1, \dots, n\}$ for $\mathbf{n}$, $|A|$ for $A^\perp$, $|A| \times |B|$ for $A \otimes B$, and $|A| \sqcup |B|$ for $A \& B$. The PCS $P(A)$ is the data type for $\mathbf{n}$, $P(A)^\perp$

for $A^\perp$, $\{x \otimes y \mid x \in P(A), y \in P(B)\}^{\perp\perp}$ for $A \otimes B$, where $(x \otimes y)(a, b) = x_a y_b$, and the set of z whose restrictions to $|A|$ and $|B|$ lie in $P(A)$ and $P(B)$ for $A \& B$ [Danos and Ehrhard 2011]. The coherence $\asymp_A$ is equality on $\mathbf{n}$, and $s \asymp_{A^\perp} t$ iff $s = t$ or not $s \asymp_A t$. On $A \otimes B$ it is componentwise. On $A \& B$ it is $\asymp_A$ and $\asymp_B$ on the two components, and any two points from different components cohere [Girard 1987].

Theorem 75 (every formula^{LeanLeanLeanLean}). *For every formula A : (i) $P(A)$ is a PCS. (ii) The indicator of every point and of every pair of distinct coherent points lies in $P(A)$, and $x_s + x_t \leq 1$ for every $x \in P(A)$ and every pair of distinct incoherent points s, t . (iii) Every $\{0, 1\}$ -valued element of $P(A)$ is the indicator of a clique of the coherence space A . (iv) Mix of the $\{0, 1\}$ -valued elements of $P(A)$ is contained in $P(A)$, and its $\{0, 1\}$ -valued elements are exactly those of $P(A)$.*

Part (ii) is proved by simultaneous induction on A . Negation exchanges the statement about coherent pairs with the statement about incoherent pairs. For $A \otimes B$, the indicator of a coherent pair is bounded by the tensor of indicators of its projections, and a pair incoherent in $A \otimes B$ is incoherent in one component, where the bound of A or B applies. Part (iii) follows from (ii): two incoherent points in the support of a $\{0, 1\}$ -valued element would give it total 2 on a pair bounded by 1. Part (iv) is Theorem 66 together with the closure of an orthogonal under mixing^{Lean}. Girard introduces probabilistic coherence spaces with the intuition that two incoherent atoms have weights summing to at most 1, and then sets coherence aside [Girard 2004]. Part (ii) shows that the intuition holds at every multiplicative-additive formula over data types. The converse of (iii) fails (Section 11.3).

The semantics of Definition 74 computes the spaces of Section 10.2: $P(\mathbf{n} \multimap \mathbf{m})$ is the PCS $X \multimap Y$ of that section for $X = \{1, \dots, n\}$ and $Y = \{1, \dots, m\}$ ^{Lean}. So Theorem 72 is a statement about these formulas^{Lean}.

The exponentials, whose webs are infinite, are not covered by Definition 74. Theorem 73 treats $! \text{Bool}$ separately.

11 Second-Order Linear Types

This section studies the construction of Section 10 at second-order types, where it returns a strict subset of the probabilistic coherence space. Section 11.1 treats the type T of a program with two functional arguments, Section 11.2 extends the strict inclusion to every formula with a negative occurrence of the core of T , and Sections 11.3 and 11.4 relate the gap to coherence spaces and to graph polytopes. By computation, not formalized, the construction and the probabilistic coherence space agree on $(\text{Bool} \multimap \text{Bool}) \multimap \text{Bool}$ and $((\text{Bool} \multimap \text{Bool}) \multimap \text{Bool}) \multimap \text{Bool}$, the two iterated implications into Bool we checked.

11.1 Randomized Sequential Programs

At second order, let $T = ((X \multimap Y) \otimes (X' \multimap Y'))^\perp$, the type of a program that receives $f : X \multimap Y$ and $g : X' \multimap Y'$, uses

each once, and halts or diverges. We take T to be the set of $t : (X \times Y) \times (X' \times Y') \rightarrow [0, \infty]$ with $\sum_{p,q} y_p t(p, q) z_q \leq 1$ for all y in $X \multimap Y$ and z in $X' \multimap Y'$, the orthogonal of the products $y \otimes z$. For $X = Y = X' = Y' = \text{Bool}$, Lean checks that T satisfies the three PCS conditions^{Lean}. A *randomized sequential program* queries one argument first. Querying f first, it chooses the input a with probability $\mu(a)$, after the answer b chooses the input c to g with probability $\kappa(a, b, c)$, and after the answer d halts with probability $\rho(a, b, c, d)$. Its value at $((a, b), (c, d))$ is $\mu(a)\kappa(a, b, c)\rho(a, b, c, d)$ plus the analogous term for programs that query g first, with $\sum \mu \leq 1$ over both orders, $\sum_c \kappa(a, b, c) \leq 1$, and $\rho \leq 1$. It is *deterministic* if its values are in $\{0, 1\}$.

Theorem 76 (second order^{LeanLean}). *For nonempty finite X, X' and finite Y, Y' : (i) A $\{0, 1\}$ -valued z lies in T iff its support lies in a tree $\{((a, b), (c(b), d)) \mid b \in Y, d \in Y'\}$ for some $a \in X$ and $c : Y \rightarrow X'$, or in a tree $\{((a(d), b), (c, d))\}$ that queries g first, and these are exactly the deterministic sequential programs^{Lean}. (ii) Mix of the $\{0, 1\}$ -valued elements of T is exactly the set of randomized sequential programs.*

Part (ii) contains both directions of Kuhn's theorem for this type [Kuhn 1953]: every randomized sequential program is a mixture of deterministic ones^{Lean}, and every mixture of randomized sequential programs is a randomized sequential program^{Lean}. The second direction conditions the choices at each node on the history, and nodes reached with probability 0 are given weight 0. For $X = Y = X' = Y' = \text{Bool}$ the same statements are also proved directly^{Lean}.

A language for T . Randomized sequential programs are the denotations of a small language^{Lean}. It has four syntactic stages: no function called, f called, g called, and both called. Each stage has divergence and a finite probabilistic choice $\text{choose}(w_1 P_1, \dots, w_k P_k)$ with $\sum_i w_i \leq 1$, the missing mass diverging. The first stage has $\text{call}_f a k$ and $\text{call}_g c k$ with a continuation k for each answer, the middle stages call the remaining function, and the last stage halts. The stages make each function called exactly once. A choice denotes the weighted sum of the denotations of its branches, and $\text{call}_f a k$ denotes $((a', b), (c, d)) \mapsto [a' = a] \llbracket k b \rrbracket(c, d)$.

Theorem 77 (soundness and definability^{LeanLean}). *For nonempty finite X, X' and finite Y, Y' , every program denotes a randomized sequential program, hence an element of Mix of the $\{0, 1\}$ -valued elements of T . Every $\{0, 1\}$ -valued element of T is the denotation of a program without choice. So Mix of the $\{0, 1\}$ -valued elements of T is exactly the set of denotations of programs^{Lean}.*

The language is not a full programming language: it has no recursion and no higher-order arguments beyond f and g . Relating it to probabilistic PCF at this type is not done here.

The calculus is strictly smaller than the PCS. Call two distinct web points of T *exclusive* if a single pair of deterministic

partial programs (f, g) produces both, that is, if their first components cohere in $X \multimap Y$ and their second components cohere in $X' \multimap Y'$. The *exclusivity graph* of T has the web points as vertices and the exclusive pairs as edges.

Theorem 78 (strict inclusion^{LeanLean}). *Let X, Y, X', Y' have at least two elements each. There are five web points of T such that the function with value $\frac{1}{2}$ on them and 0 elsewhere lies in T and is not in Mix of the $\{0, 1\}$ -valued elements of T . For $X = Y = X' = Y' = \text{Bool}$, the total weight on the five points is at most 2 for every randomized sequential program, 2 is attained, and it is at most $\frac{5}{2}$ on T , attained by the function above.*

Over Bool , writing (a, b) for the observation $f(a) = b$, the five points are $((t, t), (t, t))$, $((t, f), (f, f))$, $((f, t), (t, f))$, $((f, t), (f, t))$, and $((f, f), (f, f))$. They induce a 5-cycle in the exclusivity graph. Lean checks the two facts the proof uses, that the induced subgraph has no triangle and no three pairwise non-adjacent vertices^{Lean}, and embeds the five points into larger types by injections from Bool .

The semantics of Definition 74 computes $T: P(((\mathbf{n} \multimap \mathbf{m}) \otimes (\mathbf{n}' \multimap \mathbf{m}'))^\perp)$ is T for $X = \{1, \dots, n\}$, $Y = \{1, \dots, m\}$, $X' = \{1, \dots, n'\}$, $Y' = \{1, \dots, m'\}$ ^{Lean}. So Theorem 76(ii) and the strict inclusion of Theorem 78 are statements about these formulas^{Lean}.

Related work. Full abstraction of probabilistic coherence spaces for probabilistic PCF [Ehrhard et al. 2018] is proved from the analyticity of the morphisms, not from definability of every point, and in the concurrent game model of Castellan, Clairambault, Paquet and Winskel [Castellan et al. 2018] definability fails until strategies are restricted to sequential probabilistic innocent ones. Kuhn's theorem [Kuhn 1953] is the classical form of Theorem 76(ii) for games with perfect recall. The tensor of probabilistic coherence spaces has been related to no-signalling composition [Simmons and Kissinger 2022]. What is added here is an explicit point outside the calculus at a type without exponentials, its transfer to every formula with a negative occurrence of the core (Theorem 81), the description of the calculus and of the space as STAB and QSTAB of one graph (Theorem 83) with the exact condition for a gap, and the mechanization.

11.2 Other Formulas

The strict inclusion transfers along a simple kind of embedding of webs.

Definition 79 (coordinate retract^{Lean}). For formulas D and A of Definition 74, a *coordinate retract* of D into A is an injective map $j: |D| \rightarrow |A|$ such that the pushforward j_* , with $(j_*x)(j(d)) = x_d$ and value 0 outside the image of j , maps $P(D)$ into $P(A)$, and the pullback $z \mapsto z \circ j$ maps $P(A)$ into $P(D)$.

Theorem 80 (transfer^{LeanLean}). (i) *If D is a coordinate retract of A and the calculus of D is strictly smaller than $P(D)$, then*

*the calculus of A is strictly smaller than $P(A)$. (ii) *Coordinate retracts compose. If D is a coordinate retract of A , then $D^\perp$ is a coordinate retract of $A^\perp$. The formula D is a coordinate retract of $D \otimes B$, $B \otimes D$, and, if $|B|$ is nonempty, of $D \otimes B$ and $B \otimes D$.**

For (i), take $x \in P(D)$ outside the calculus. If j_*x were a mixture of $\{0, 1\}$ -valued elements of $P(A)$, pulling the mixture back along j would write $x = (j_*x) \circ j$ as a mixture of $\{0, 1\}$ -valued elements of $P(D)$. For negation, the pairing satisfies $\sum_a (j_*u)_a z_a = \sum_d u_d z_{j(d)}$, which exchanges the two conditions. For $D \otimes B$, the map is $j(d) = (d, b)$ for a point b of B , so $j_*x = x \otimes \delta_b$, and the pullback condition uses that every element of $P(B)$ is at most 1 at b (Theorem 75(ii)).

An occurrence of a formula D in A is *positive* if it lies under an even number of negations and *negative* otherwise, the derived connectives $\wp$, $\oplus$, and $\multimap$ being unfolded. So the left argument of $\multimap$ is a negative position.

Theorem 81 (the gap at other formulas^{Lean}). *Let $n, m, n', m' \geq 2$, and let A be a formula all of whose data types are nonempty. If $(\mathbf{n} \multimap \mathbf{m}) \otimes (\mathbf{n}' \multimap \mathbf{m}')$ occurs negatively in A , the calculus of A is strictly smaller than $P(A)$.*

A positive occurrence of D gives a coordinate retract of D into A , and a negative one a coordinate retract of $D^\perp$, by Theorem 80(ii) and induction on A , using that D is a coordinate retract of $D^{\perp\perp}$. Theorem 80(i) and Theorem 78 then give the claim. Examples are T itself, the type $((\mathbf{n} \multimap \mathbf{m}) \otimes (\mathbf{n}' \multimap \mathbf{m}')) \multimap B$ of a program that receives two functions and returns a value of type B , for every B with nonempty data types (Lean checks the instance $B = 2$), and every formula containing one of these positively. The condition is syntactic: the curried type $(\mathbf{n} \multimap \mathbf{m}) \multimap ((\mathbf{n}' \multimap \mathbf{m}') \multimap B)$ is isomorphic to the uncurried one but is not covered as stated. A positive occurrence of the core does not suffice: Section 11.4 shows that at the core itself the calculus equals the probabilistic coherence space.

11.3 A Clique Outside the PCS

The converse of Theorem 75(iii) fails. Let $T = ((2 \multimap 2) \otimes (2 \multimap 2))^\perp$. On the five points of Theorem 78, two distinct points cohere in T iff they are not exclusive. Order them as $c_0, \dots, c_4$ along the 5-cycle of the exclusivity graph. The coherence graph on them is the complement of this cycle, again a 5-cycle. Two distinct points of $T \otimes T$ are incoherent iff they are incoherent in one component. A clique of $(T \otimes T)^\perp$ contained in $\{c_0, \dots, c_4\}^2$ is therefore a stable set of the strong product of the two coherence graphs, a copy of $C_5 \boxtimes C_5$. The set $\{(c_i, c_{2i}) \mid i \in \mathbb{Z}/5\}$ is such a stable set, of size 5. It is Shannon's stable set, which shows that the Shannon capacity of C_5 is at least $\sqrt{5}$ [Lovász 1979].

Theorem 82 (a clique outside the PCS^{LeanLean}). *Shannon's set is a clique of the coherence space $(T \otimes T)^\perp$, and its indicator is not an element of $P((T \otimes T)^\perp)$.*

The function z of Theorem 78, with value $\frac{1}{2}$ on the five points, lies in $P(T)$, so $z \otimes z$ lies in $P(T \otimes T)$. Its pairing with the indicator of Shannon's set is $5 \cdot \frac{1}{4} = \frac{5}{4} > 1$. The theorem is also proved for the web of T given directly by $\text{Bool}^{\text{Lean}}$. From $(T \otimes T)^\perp$ on, the two natural choices of certain states therefore differ. With the $\{0, 1\}$ -valued elements of the PCS, the calculus is contained in the PCS at every formula (Theorem 75(iv)). With the cliques of the coherence space, it is not.

11.4 When the Calculus Equals the PCS

For a finite graph G , a *clique* is a set of pairwise adjacent vertices and a *stable set* a set of pairwise non-adjacent vertices. Let $\text{STAB}(G)$ be Mix of the indicator functions of stable sets, and $\text{QSTAB}(G)$ the set of $x : V \rightarrow [0, \infty]$ with $\sum_{v \in C} x_v \leq 1$ for every clique C [Grötschel et al. 1988].

Theorem 83 (graph polytopes^{LeanLean}). *$T = \text{QSTAB}(G)$ and Mix of the $\{0, 1\}$ -valued elements of T is $\text{STAB}(G)$, for G the exclusivity graph of T .*

Theorem 84 (odd holes and antiholes^{LeanLean}). *If $\text{STAB}(G) = \text{QSTAB}(G)$, then G contains no induced cycle of odd length $2k + 1 \geq 5$ and no induced complement of one. In particular, if Mix of the $\{0, 1\}$ -valued elements of T equals T , the exclusivity graph of T has neither.*

A graph with neither is *Berge*. A graph is *perfect* if every induced subgraph has chromatic number equal to its largest clique size. The converse of Theorem 84 holds: $\text{STAB}(G) = \text{QSTAB}(G)$ iff G is perfect [Chvátal 1975; Lovász 1972], and G is perfect iff it is Berge, by the strong perfect graph theorem [Chudnovsky et al. 2006]. The converse is not formalized.

The core and the exact condition. The duality of Definition 71 exchanges STAB and QSTAB of complementary graphs. For every graph G , $\text{STAB}(G)^\perp = \text{QSTAB}(\overline{G})$ directly from the definitions, since the stable sets of G are the cliques of $\overline{G}$, and therefore $\text{QSTAB}(\overline{G})^\perp = \text{STAB}(G)$ by Fulkerson's antiblocking theorem [Fulkerson 1971]. Applied to the exclusivity graph G of T , Theorem 83 gives $T^\perp = \text{QSTAB}(G)^\perp = \text{STAB}(\overline{G})$, the set of mixtures of the indicators of the cliques of G , which are the $\{0, 1\}$ -valued elements of $T^\perp$. So at the core $T^\perp = (X \multimap Y) \otimes (X' \multimap Y')$ the calculus equals the probabilistic coherence space, and Theorem 81 needs the negative occurrence.

The condition of Theorem 78 that X, Y, X', Y' have at least two elements is also necessary. If X has one element, two points of $X \multimap Y$ cohere iff they are equal, and G is a disjoint union of copies of the graph on $X' \times Y'$ that joins points with different inputs, a complete multipartite graph. If Y has one element, all points of $X \multimap Y$ cohere, and G is obtained from that complete multipartite graph by replacing each vertex with a clique of size $|X|$. Both graphs are perfect, the second by Lovász's replacement lemma [Lovász 1972], and

the cases of X' and Y' are symmetric. So the calculus of T equals T iff one of X, Y, X', Y' has at most one element. This direction and the duality argument above are not formalized. We checked the condition numerically for ten choices of the four sizes between 1 and 3.

The *independence number* $\alpha(G)$ is the largest size of a stable set, and the *fractional packing number* $\alpha^*(G)$ is the largest value of $\sum_v x_v$ over $\text{QSTAB}(G)$. The bounds 2 and $\frac{5}{2}$ of Theorem 78 are $\alpha(C_5)$ and $\alpha^*(C_5)$. Cabello, Severini and Winter [Cabello et al. 2014] show that in a contextuality scenario with exclusivity graph G the noncontextual bound is $\alpha(G)$, the quantum bound is the Lovász number $\vartheta(G)$ [Lovász 1979], and the bound for theories satisfying only exclusivity is $\alpha^*(G)$. For C_5 this is the KCBS inequality [Klyachko et al. 2008], with $\vartheta(C_5) = \sqrt{5}$. A calculus for T corresponding to the theta body of the exclusivity graph [Grötschel et al. 1988], which lies between STAB and QSTAB , is not constructed here. Section 12 makes the same comparison for a Kochen–Specker set.

12 Quantum Event Logic

In quantum logic the events are the closed subspaces of a Hilbert space [Birkhoff and von Neumann 1936]. We restrict to rays, the one-dimensional subspaces, of $\mathbb{R}^d$. A *context* is an orthogonal basis, a maximal set of pairwise orthogonal rays, whose events can be observed jointly. A *state* assigns to each ray a number in $[0, \infty]$ such that the numbers on every context sum to 1. For a unit vector ψ , the *Born rule* $x_v = (\psi \cdot v)^2 / (v \cdot v)$ gives a state, and mixtures of such states, the density matrices, are the *quantum states*. For $d \geq 3$, Gleason's theorem [Gleason 1957] states that every state on all rays of $\mathbb{R}^d$ is a quantum state.

This section tests the construction of Section 10 on this event logic. The mathematics is known: the Kochen–Specker theorem [Kochen and Specker 1967], its 18-ray instance [Cabello et al. 1996], and the bounds of the graph-theoretic approach to contextuality [Cabello et al. 2014]. What is added is its reading in terms of the construction, with the same graph polytopes as Section 11.4, and its mechanization.

The presentation of Definition 65 takes the rays as events and, as certain states, the $\{0, 1\}$ -valued states, which give exactly one ray of every context the value 1. By the Kochen–Specker theorem [Kochen and Specker 1967], for $d \geq 3$ there are none. We formalize a finite instance.

Definition 85 (Cabello's set^{Lean}). Cabello, Estebaranz, and García-Alcaine [Cabello et al. 1996] give 18 vectors of $\mathbb{R}^4$ with coordinates in $\{0, \pm 1\}$, and 9 sets of 4 of them:

$\{0001, 0010, 1100, 1\bar{1}00\}$, $\{0001, 0100, 1010, 10\bar{1}0\}$, $\{1\bar{1}1\bar{1}, 1\bar{1}\bar{1}1, 1100, 0011\}$,
 $\{1\bar{1}1\bar{1}, 1111, 10\bar{1}0, 010\bar{1}\}$, $\{0010, 0100, 1001, 100\bar{1}\}$, $\{1\bar{1}\bar{1}1, 1111, 100\bar{1}, 01\bar{1}0\}$,
 $\{1\bar{1}\bar{1}1, 111\bar{1}, 1\bar{1}00, 0011\}$, $\{1\bar{1}\bar{1}1, \bar{1}111, 1010, 010\bar{1}\}$, $\{111\bar{1}, \bar{1}111, 1001, 01\bar{1}0\}$

where $\bar{1}$ stands for -1 .

Theorem 86 (Kochen–Specker for Cabello’s set^{LeanLean}). *Each of the 9 sets is an orthogonal basis of $\mathbb{R}^4$, and each of the 18 vectors lies in exactly two of them. No function from the 18 vectors to $\{0, 1\}$ takes the value 1 at exactly one vector of every basis.*

The first two statements are checked by evaluation. The third is a parity count. Summing such a function over the 9 bases gives 9, and counts every vector twice, so the sum is even.

Theorem 87 (no certain states^{LeanLean}). *For the presentation whose events are the 18 vectors and whose certain states are the $\{0, 1\}$ -valued functions summing to 1 on every basis, the set of certain states and Mix of it are empty. For every $\psi \in \mathbb{R}^4$ with $\psi \cdot \psi = 1$, the Born-rule assignment sums to 1 on every basis, so it is a state that is not in Mix of the certain states.*

The Born-rule statement is Parseval’s identity $\sum_{v \in B} (\psi \cdot v)^2 / (v \cdot v) = \psi \cdot \psi$ for each orthogonal basis B , checked basis by basis^{Lean}.

Allowing divergence. In a probabilistic coherence space an element may have total mass below 1, the missing mass being the probability of divergence [Danos and Ehrhard 2011]. Allowing the same here, a state sums to at most 1 on every context. The certain states are then the $\{0, 1\}$ -valued functions with at most one ray of value 1 in each basis, the indicators of the stable sets of the graph that joins two rays when they lie in a common basis. These exist, and the construction is no longer empty.

Theorem 88 (a stable-set bound^{Lean}). *Every mixture of these certain states has total weight at most 4 on the 18 rays, and 4 is attained. Every Born-rule assignment of a unit vector has total weight $\frac{9}{2}$. In particular no Born-rule assignment is a mixture of these certain states.*

Summing over the nine bases counts every ray twice. A certain state has weight at most 1 on each basis, so its total weight is at most $\frac{9}{2}$, hence at most 4 as it is an integer. A Born-rule assignment has weight 1 on each basis. Summed over the bases, every mixture of certain states has weight at most 8, so it diverges with probability at least $\frac{1}{9}$ on some basis, while a quantum state diverges on none. The bound 4 is the independence number of the graph, and the quantum value $\frac{9}{2}$ does not depend on the state. This is the comparison of Section 11.4 for a different graph, and the gap between 4 and $\frac{9}{2}$ is the Kochen–Specker contradiction in quantitative form [Cabello 2008; Cabello et al. 2014].

Quantum probability therefore fails criterion (1) of Section 1: no state is $\{0, 1\}$ -valued, so the $\{0, 1\}$ -valued states cannot reconstruct the event logic. Theorem 87 proves this for Cabello’s set. For all rays of $\mathbb{R}^d$ with $d \geq 3$ it follows from the Kochen–Specker theorem, which is not formalized. With normalized certain states, the construction of Section 10 produces nothing at this event logic: with no certain states, every affine law is valid on the certain states, including $1 \leq 0$, and

Theorem 68 returns the empty set. The mixtures of certain states are the noncontextual hidden-variable models of the event logic, and Theorem 86 is the statement that this event logic has none [Kochen and Specker 1967]. For lattice valuations, Theorem 60 shows the corresponding absence on the non-distributive lattice M_3 , which has no sharp valuation.

13 Related Work

Informal precedents. Pairing probability with intuitionistic logic is not a new idea, though the literature on it is scattered. Weatherson develops intuitionistic probability axiomatically and defends it with Dutch-book arguments [Weatherson 2003]. His axioms match our valuation axioms exactly, with negation derived rather than assumed, so the object itself and the bound $v(a) + v(\neg a) \leq 1$ are his. What is new here is the decomposition of that bound’s slack into dnGap and deMorganGap and its determinacy from v alone (Theorem 6). Paris shows that the assignments avoiding a Dutch book are the mixtures of a logic’s two-valued valuations, and that when the valuations respect $\wedge$ and $\vee$, as in intuitionistic and modal logics, they are exactly the normalized, monotone, modular assignments [Paris 2001]. On finite frames this is Theorem 17. Williams obtains the same set from accuracy domination, under the name generalized probabilism [Williams 2012]. Paris also shows, from Shafer’s results, that the mixtures of the “known true” valuations, which make a sentence true when it follows from what is known, are exactly the Dempster–Shafer belief functions [Paris 1994, 2001; Shafer 1976]. These valuations do not respect $\vee$. We mechanize an analogous fact for our intuitionistic setting: the 2-monotone case follows directly from modularity^{Lean}, and the full tower is Theorem 23. The identification $\text{Bel} = P(\Box \cdot)$ connecting belief functions [Dempster 1967; Shafer 1976] to modal logic is established [Ruspini 1986]. Section 7 shows this identification is not a separate axiom for us but a special case of our frame-level slack, recovered automatically once the frame is a topology and $\neg$ is read as the interior of the complement^{Lean}.

On the localic side, valuations on frames are standard objects of constructive measure theory, with Vickers’s work and the Coquand–Spitters development as canonical references [Coquand and Spitters 2009; Vickers 2008]. These normally build in Scott continuity and target additive, Riesz-style goals. Section 4 shows Scott continuity to be a substantive assumption (it decides point-representability on the frames we examine), so our base structure omits it. Cox, Jaynes, Halpern’s counterexample, and the Van Horn and Arnborg–Sjödín repairs [Arnborg and Sjödín 2000; Cox 1946; Halpern 1999; Horn 2003; Jaynes 2003] are classical throughout, and Cox’s and Van Horn’s derivations both lean on double-negation elimination. Section 6 proves the sum-rule half without that move, at the price of positing modularity

as an axiom rather than deriving it (Example 29). Van Horn observes that dropping axiom R3 yields an interval-valued theory, exactly the belief/plausibility pair our slack separates. To our knowledge, no prior work, informal or formal, states either characterization theorem: complement rule $\iff$ excluded middle (Theorem 10), or Dempster = Bayes $\iff$ zero slack at the evidence (Theorem 27).

Prior formalizations. Two bodies of mechanized work are adjacent, and neither overlaps. ALEA [Audebaud and Paulin-Mohring 2009] formalizes a discrete distribution monad in Coq for randomized programs: additive probability, no event logic in our sense. Closest is the HoTT/Coq line of Faissole and Spitters on *valuations* over lower reals [Faissole and Spitters 2017], done constructively with Scott continuity and aimed at integration (Riesz) and a Giry-style valuation *monad*. Their valuations differ from ours in carrying Scott continuity, and their monad is the Scott-continuous instance of the question examined in Section 8. Our development drops the continuity their base structure builds in, and the two mechanizations do not otherwise overlap. On the domain-theoretic side, commutativity of the full valuations monad on the category of directed-complete partial orders is open. A hierarchy of commutative submonads, generated from simple valuations (finite mixtures of point valuations), has been constructed instead [Jia et al. 2021]. Our countably-presented fragment, the valuations given as countable mixtures (Proposition 50), is the frame-side analogue.

Non-Boolean probability elsewhere. Quantum probability on orthomodular lattices, with Gleason’s theorem [Birkhoff and von Neumann 1936; Gleason 1957], is the long-standing precedent for re-basing probability on a non-Boolean event logic. Orthomodular lattices give up distributivity itself, while our frames stay distributive by definition (Definition 1), locating all the non-classical behavior in the failure of excluded middle alone. Section 12 applies the construction from certain states to a quantum event logic.

Linear logic marks a different kind of departure. Its event algebras are quantales, complete lattices equipped with an associative tensor $\otimes$ that need not be idempotent [Girard 1987]. The lattice reduct of a quantale is already an instance of Section 9’s generalization, but $\otimes$ itself is not $\sqcap$, and a valuation grading it cannot be phrased in Definition 51’s vocabulary without first deciding what modularity even means for a non-idempotent operation. Section 10 replaces modularity by the affine laws valid on the certain states. The probabilistic semantics that does exist for linear logic, Girard’s probabilistic coherence spaces as developed by Danos and Ehrhard [Danos and Ehrhard 2011], grades the multiplicative structure directly rather than valuing a lattice. Section 10 shows that it contains the calculus of certain states at every multiplicative-additive formula and equals it at implications

between data types, and Section 11 that the inclusion is strict at second order.

Free probability, fuzzy measures, and imprecise probability [Walley 1991] vary other parameters (independence, additivity, single-valuedness). Where Walley’s coherent lower previsions posit the belief/plausibility gap directly as a coherence primitive, our slack (Section 2) is derived, forced by modularity and negation rather than assumed. Fritz’s synthetic Markov-category framework [Fritz 2020] varies the ambient structure categorically rather than order-theoretically, axiomatizing conditional independence and sufficient statistics abstractly across discrete, measure-theoretic, and Gaussian probability alike. It remains unmechanized and is orthogonal to our variation in the underlying logic. Wolpert and Kinney model mathematics itself as a stochastic process, replacing binary theoremhood with a probability distribution [Wolpert and Kinney 2024]. Theoremhood is a semidecidable event, so the computability guard of Section 7 bears directly on that program: sharp credences about such events are uncomputable. Our variation is specifically the *logic*, holding the Cox-style methodology fixed. Within DS theory, the two conditionings and their inequivalence are classical [Fagin and Halpern 1991; Shafer 1976]. What is new in Theorem 27 is the exact logical characterization of their agreement, and its mechanization.

14 Conclusion

In this work, we take seriously the Cox–Jaynes reading of probability as *extended logic*, and make the extension a construction with the logic of events as a parameter. The construction fixes an event algebra (the chosen logic in algebraic form) and takes as its assignments of degrees of certainty the valuations: the maps from events to numbers satisfying monotonicity, normalization, and modularity. We specify the two conditions that qualify the result as a probability calculus: when events are certain (0 or 1 certainty), the valuations must reconstruct the underlying event logic, and the value of the construction at classical logic must be the classical theory.

The motivation for developing this extension comes from the study of program behaviour in terms of outcome probabilities. Many properties of programs are only semidecidable. However, the Boolean event algebra underlying Kolmogorov’s axioms imposes the law of excluded middle (and therefore decidability) on the structure of logical connectives applied to events. This makes it not a good fit for reasoning about events that are not decidable, such as whether an arbitrary program halts: insisting on a sharp classical readout for such an event does not decide it, but instead produces an object we show to be uncomputable, and a non-sharp classical credence assigns its remainder to non-termination, an event no finite observation confirms. The extension process we propose, when fixing its logic parameter to be intuitionistic,

does not conflict with the underlying event logic of computation by construction, and is capable of reasoning about the possibility of non-termination.

Substituting the Heyting event algebra (and therefore intuitionistic logic) for the Boolean algebra representing the classical logic case changes only the consequences that excluded middle supplied. Both qualifying conditions hold at this parameter value (Sections 2 and 3), and classicality enters only through the complement rule (Theorem 10) and its dynamic counterpart, the agreement of Dempster and Bayesian conditioning (Theorem 27). The complement rule degrades to the inequality $v(a) + v(\neg a) \leq 1$. The deficit in this inequality, for which we re-purpose the term slack, is a new quantity: the value of the undecided region, given by the boundary mass in the measure model. Slack is submodular yet does not determine the valuation. The weaker logic also adds the two-gap decomposition of slack, the atomic/diffuse divide, and a belief-function reading through $\neg\neg$.

Everything else holds verbatim on frames: conditioning, the product rule, Bayes symmetry, disjoint additivity, total probability over genuine partitions, and inclusion–exclusion itself. On finite frames, every valuation is a classical mass function on the frame’s points. The product-rule half of Cox’s theorem is independent of the logic while the sum-rule half must be posited, so the direct transcription of the classical Cox statement is unsatisfiable. Every result is machine-checked in Lean 4 over mathlib.

A recurring shape. Three of this paper’s negative results share a form worth naming: two individually reasonable demands on a valuation, jointly unsatisfiable, with the proof identifying which demand is forced to give rather than merely asserting that some construction fails. On M_3 (Theorem 60), sharpness and modularity together are unsatisfiable: a 2-valued readout cannot satisfy all three pairwise sum-to-one equations at once, and distributivity is the hypothesis that has to give. Grading a non-idempotent tensor (Corollary 64) is the same shape one level up, at the axiom itself: being graded by modularity’s compositional law and discriminating a from $a \otimes a$ cannot both hold, for any Ω and any $\otimes$, and probabilistic coherence spaces [Danos and Ehrhard 2011] are the witness that keeps discrimination and gives up gradability. Demanding a sharp, uniform, *computable* readout of a semidecidable event (Theorem 45) is the same trade-off again, one level further out: bivalence and computability cannot both hold, and the classical route keeps bivalence by fiat and gives up an algorithm. None of the three results says only that a construction fails; each names the exact pair of demands responsible and shows the failure is forced, not an artifact of how the construction happened to be set up.

Toward a monad. Section 8 lays the groundwork for a categorical exploration of the construction: Val is a mechanized functor on locales, and its countably-presented fragment composes as kernels, though whether that composition

is canonical on valuations rather than their presentations remains open. A commutative monad structure, where it exists, would be the intuitionistic counterpart of the Giry monad and a step toward compositional semantics for probabilistic programs with semidecidable events. One candidate formulation exists: the space of Scott-continuous valuations is the classifying locale of a geometric theory [Coquand and Spitters 2009; Vickers 2008]. Deleting its continuity axiom still leaves a geometric theory, whose points, read classically, are exactly our valuations. Its multiplication, however, must integrate against a merely modular valuation, and the diffuse part of Example 18 is exactly what such integration fails to capture. We offer the formulation as a target, not a result. Beyond intuitionistic logic, no ambient category sending a logic to its probability calculus functorially is evident from our construction, and this question remains open.

Other open problems and future work. A natural extension of our mechanization effort involves completing the Aczél development, and mechanizing the Dutch-book theorem [Weatherson 2003]. Both require substantial additional effort and were deemed out of scope for this work. To extend our mechanization effort with preliminary monad results, we would quotient the countable presentations (Proposition 50) into a Kleisli category of countably-presented kernels, and mechanize the classifying-locale formulation (which needs classifying locales of geometric theories, infrastructure mathlib currently lacks).

Some related mathematical problems remain open as well. The first is developing a representation of the diffuse part on non-spatial frames. The second is deriving modularity from a criterion more primitive than the mixture characterization (Theorem 17), knowing it cannot come from disjunction data alone (Example 29).

The construction method applies beyond intuitionistic logic: state the classical desiderata, delete or substitute the axioms the target logic no longer licenses, and check what survives, what splits, and what becomes unsatisfiable. Linear logic was the first case tried, and Section 9 closes it in the negative rather than leaving it open: substituting the quantale tensor $\otimes$ for $\sqcap$ in modularity is not merely difficult but unsatisfiable (Corollary 64), so no valuation in this paper’s sense can grade $\otimes$ ’s consumption of evidence without losing the very discrimination $\otimes$ exists to carry [Danos and Ehrhard 2011]. What remains open is the next question this raises, not this instance of it: whether some compositional law other than modularity transplanted verbatim could play our construction’s role for logics whose primitive connectives are not idempotent.

References

- János Aczél. 1966. *Lectures on Functional Equations and Their Applications*. Academic Press, New York, NY, USA.
- Reynald Affeldt, Cyril Cohen, and Damien Rouhling. 2018. Formalization Techniques for Asymptotic Reasoning in Classical Analysis. *Journal of*

- Formalized Reasoning 11, 1 (2018), 43–76.
- Stefan Arnborg and Gunnar Sjödin. 2000. Bayes Rules in Finite Models. In *European Conference on Artificial Intelligence (ECAI)*. IOS Press, Amsterdam, Netherlands, 571–575.
- Philippe Audebaud and Christine Paulin-Mohring. 2009. Proofs of Randomized Algorithms in Coq. *Science of Computer Programming* 74, 8 (2009), 568–589.
- Garrett Birkhoff. 1937. Rings of Sets. *Duke Mathematical Journal* 3, 3 (1937), 443–454.
- Garrett Birkhoff and John von Neumann. 1936. The Logic of Quantum Mechanics. *Annals of Mathematics* 37, 4 (1936), 823–843.
- Adán Cabello. 2008. Experimentally Testable State-Independent Quantum Contextuality. *Physical Review Letters* 101, 21 (2008), 210401.
- Adán Cabello, José M. Estebaranz, and Guillermo García-Alcaine. 1996. Bell-Kochen-Specker Theorem: A Proof with 18 Vectors. *Physics Letters A* 212, 4 (1996), 183–187.
- Adán Cabello, Simone Severini, and Andreas Winter. 2014. Graph-Theoretic Approach to Quantum Correlations. *Physical Review Letters* 112, 4 (2014), 040401.
- Simon Castellan, Pierre Clairambault, Hugo Paquet, and Glynn Winskel. 2018. The Concurrent Game Semantics of Probabilistic PCF. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*. 215–224.
- Gregory J. Chaitin. 1975. A Theory of Program Size Formally Identical to Information Theory. *J. ACM* 22, 3 (1975), 329–340.
- Maria Chudnovsky, Neil Robertson, Paul Seymour, and Robin Thomas. 2006. The Strong Perfect Graph Theorem. *Annals of Mathematics* 164, 1 (2006), 51–229.
- Václav Chvátal. 1975. On Certain Polytopes Associated with Graphs. *Journal of Combinatorial Theory, Series B* 18, 2 (1975), 138–154.
- Thierry Coquand and Bas Spitters. 2009. Integrals and Valuations. *Journal of Logic and Analysis* 1, 3 (2009), 1–22.
- Richard T. Cox. 1946. Probability, Frequency and Reasonable Expectation. *American Journal of Physics* 14, 1 (1946), 1–13.
- Vincent Danos and Thomas Ehrhard. 2011. Probabilistic Coherence Spaces as a Model of Higher-Order Probabilistic Computation. *Information and Computation* 209, 6 (2011), 966–991.
- Bruno de Finetti. 1974. *Theory of Probability*. Vol. 1. Wiley.
- Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Conference on Automated Deduction (CADE)*. Springer, Cham, Switzerland, 625–635.
- Arthur P. Dempster. 1967. Upper and Lower Probabilities Induced by a Multivalued Mapping. *Annals of Mathematical Statistics* 38, 2 (1967), 325–339.
- Dieter Denneberg. 1994. *Non-Additive Measure and Integral*. Kluwer, Dordrecht, Netherlands.
- Thomas Ehrhard, Michele Pagani, and Christine Tasson. 2018. Full Abstraction for Probabilistic PCF. *J. ACM* 65, 4 (2018), 23:1–23:44.
- Martin H. Escardó. 2004. Synthetic Topology of Data Types and Classical Spaces. *Electronic Notes in Theoretical Computer Science* 87 (2004), 21–156.
- Ronald Fagin and Joseph Y. Halpern. 1991. A New Approach to Updating Beliefs. *Uncertainty in Artificial Intelligence* 6 (1991), 347–374.
- Florian Faissole and Bas Spitters. 2017. Synthetic Topology in Homotopy Type Theory for Probabilistic Programming. In *Programming Languages meets Program Verification (PPS)*. Full version with M. E. Bidlingmaier: Faissole-Bidlingmaier-Spitters, “Synthetic topology in Homotopy Type Theory for probabilistic programming,” *Math. Struct. Comput. Sci.* 31(4), 2021..
- Yannick Forster, Dominik Kirst, and Gert Smolka. 2019. On Synthetic Undecidability in Coq, with an Application to the Entscheidungsproblem. In *Proceedings of the 8th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*. 38–51.
- Tobias Fritz. 2020. A Synthetic Approach to Markov Kernels, Conditional Independence and Theorems on Sufficient Statistics. *Advances in Mathematics* 370 (2020), 107239.
- Delbert R. Fulkerson. 1971. Blocking and Anti-Blocking Pairs of Polyhedra. *Mathematical Programming* 1 (1971), 168–194.
- Paolo Ghirardato. 1997. On Independence for Non-Additive Measures, with a Fubini Theorem. *Journal of Economic Theory* 73, 2 (1997), 261–291.
- Jean-Yves Girard. 1987. Linear Logic. *Theoretical Computer Science* 50, 1 (1987), 1–101.
- Jean-Yves Girard. 2004. Between Logic and Quantic: A Tract. In *Linear Logic in Computer Science*, Thomas Ehrhard, Jean-Yves Girard, Paul Ruet, and Philip Scott (Eds.). London Mathematical Society Lecture Note Series, Vol. 316. Cambridge University Press, 346–381.
- Michèle Giry. 1982. A Categorical Approach to Probability Theory. In *Categorical Aspects of Topology and Analysis*. Lecture Notes in Mathematics, Vol. 915. Springer, Berlin, Germany, 68–85.
- Andrew M. Gleason. 1957. Measures on the Closed Subspaces of a Hilbert Space. *Journal of Mathematics and Mechanics* 6, 6 (1957), 885–893.
- Kurt Gödel. 1933. Eine Interpretation des intuitionistischen Aussagenkalküls. *Ergebnisse eines mathematischen Kolloquiums* 4 (1933), 39–40.
- Martin Grötschel, László Lovász, and Alexander Schrijver. 1988. *Geometric Algorithms and Combinatorial Optimization*. Springer.
- Joseph Y. Halpern. 1999. A Counterexample to Theorems of Cox and Fine. *Journal of Artificial Intelligence Research* 10 (1999), 67–85.
- Carl G. Hempel. 1945. Studies in the Logic of Confirmation (I.). *Mind* 54, 213 (1945), 1–26.
- Otto Hölder. 1901. Die Axiome der Quantität und die Lehre vom Mass. *Berichte über die Verhandlungen der Sächsischen Gesellschaft der Wissenschaften zu Leipzig* 53 (1901), 1–64.
- Kevin S. Van Horn. 2003. Constructing a Logic of Plausible Inference: A Guide to Cox’s Theorem. *International Journal of Approximate Reasoning* 34, 1 (2003), 3–24.
- Edwin T. Jaynes. 2003. *Probability Theory: The Logic of Science*. Cambridge University Press, Cambridge, UK.
- Xiaodong Jia, Bert Lindenhovius, Michael Mislove, and Vladimir Zamdzhiev. 2021. Commutative Monads for Probabilistic Programming Languages. In *Logic in Computer Science (LICS)*. IEEE.
- Peter T. Johnstone. 1980. Conditions related to De Morgan’s law. In *Applications of Sheaves*. Lecture Notes in Mathematics, Vol. 753. Springer, Berlin, 479–491.
- Peter T. Johnstone. 1982. *Stone Spaces*. Cambridge University Press, Cambridge, UK.
- Alexander A. Klyachko, M. Ali Can, Sinem Binicioğlu, and Alexander S. Shumovsky. 2008. Simple Test for Hidden Variables in Spin-1 Systems. *Physical Review Letters* 101, 2 (2008), 020403.
- Simon Kochen and Ernst P. Specker. 1967. The Problem of Hidden Variables in Quantum Mechanics. *Journal of Mathematics and Mechanics* 17, 1 (1967), 59–87.
- Anders Kock. 2012. Commutative Monads as a Theory of Distributions. *Theory and Applications of Categories* 26 (2012), 97–131.
- Andrey N. Kolmogorov. 1933. *Grundbegriffe der Wahrscheinlichkeitsrechnung*. Springer, Berlin, Germany.
- Tomáš Kroupa. 2006. Every State on Semisimple MV-Algebra Is Integral. *Fuzzy Sets and Systems* 157, 20 (2006), 2771–2782.
- Harold W. Kuhn. 1953. Extensive Games and the Problem of Information. In *Contributions to the Theory of Games II*. Annals of Mathematics Studies, Vol. 28. Princeton University Press, 193–216.
- Jan Kühr and Daniele Mundici. 2007. De Finetti Theorem and Borel States in $[0, 1]$ -Valued Algebraic Logic. *International Journal of Approximate Reasoning* 46, 3 (2007), 605–616.
- László Lovász. 1972. Normal Hypergraphs and the Perfect Graph Conjecture. *Discrete Mathematics* 2, 3 (1972), 253–267.
- László Lovász. 1979. On the Shannon Capacity of a Graph. *IEEE Transactions on Information Theory* 25, 1 (1979), 1–7.

Assia Mahboubi and Enrico Tassi. 2021. *Mathematical Components*. Zenodo. <https://doi.org/10.5281/zenodo.4457887>

The mathlib Community. 2020. The Lean Mathematical Library. In *Certified Programs and Proofs (CPP)*. ACM, New York, NY, USA, 367–381.

J. C. C. McKinsey and Alfred Tarski. 1944. The Algebra of Topology. *Annals of Mathematics* 45, 1 (1944), 141–191.

Giovanni Panti. 2008. Invariant Measures in Free MV-Algebras. *Communications in Algebra* 36, 8 (2008), 2849–2861.

Jeff B. Paris. 1994. *The Uncertain Reasoner's Companion: A Mathematical Perspective*. Cambridge University Press, Cambridge, UK.

Jeff B. Paris. 2001. A Note on the Dutch Book Method. In *Proceedings of the Second International Symposium on Imprecise Probabilities and Their Applications (ISIPTA)*. 301–306.

K. P. S. Bhaskara Rao and M. Bhaskara Rao. 1983. *Theory of Charges: A Study of Finitely Additive Measures*. Pure and Applied Mathematics, Vol. 109. Academic Press, New York, NY, USA.

Giuseppe Rosolini. 1986. *Continuity and Effectiveness in Topoi*. Ph. D. Dissertation. University of Oxford.

Enrique H. Ruspini. 1986. *The Logical Foundations of Evidential Reasoning*. Technical Report 408. SRI International, Menlo Park, CA, USA.

Glenn Shafer. 1976. *A Mathematical Theory of Evidence*. Princeton University Press, Princeton, NJ, USA.

Will Simmons and Aleks Kissinger. 2022. Higher-Order Causal Theories Are Models of BV-Logic. In *47th International Symposium on Mathematical Foundations of Computer Science (MFCS) (LIPIcs, Vol. 241)*. 80:1–80:14.

Steven Vickers. 2008. A Localic Theory of Lower and Upper Integrals. *Mathematical Logic Quarterly* 54, 1 (2008), 109–123.

Peter Walley. 1991. *Statistical Reasoning with Imprecise Probabilities*. Chapman and Hall, London, UK.

Brian Weatherston. 2003. From Classical to Intuitionistic Probability. *Notre Dame Journal of Formal Logic* 44, 2 (2003), 111–123.

J. Robert G. Williams. 2012. Generalized Probabilism: Dutch Books and Accuracy Domination. *Journal of Philosophical Logic* 41, 5 (2012), 811–840.

David H. Wolpert and David B. Kinney. 2024. A Stochastic Model of Mathematics and Science. *Foundations of Physics* 54 (2024), 21.

Adam Yedidia and Scott Aaronson. 2016. A Relatively Small Turing Machine Whose Behavior Is Independent of Set Theory. *Complex Systems* 25, 4 (2016), 297–327.

A Values in the Extended Reals

Valuations take values in the extended non-negative reals $[0, \infty]$, following the measure-theoretic convention of mathlib. As a result, sums indexed by an arbitrary family are unconditionally defined, with no summability hypothesis, and they commute over dependent sums: for a function f on a sigma type $\Sigma a, \beta(a)$, one has $\sum_p f(p) = \sum_a \sum_b f(a, b)$ with no side condition. This is what lets the countable-mixture results of Section 8 hold as stated. Additionally, subtraction is truncated at 0. This never affects the main text, since every subtraction there is applied to a pair already ordered by monotonicity, as with the mass of Section 4, so the truncation is never triggered.

B Lean Statements

B.1 Introduction

B.2 Valuations on Frames

```

structure Valuation (Ω : Type*) [Order.
  Frame Ω] where
  toFun : Ω → ℝ≥0∞
  map_bot' : toFun ⊥ = 0
  map_top' : toFun ⊤ = 1
  mono' : Monotone toFun
  modular' : ∀ a b, toFun a + toFun b =
    toFun (a ⊔ b) + toFun (a ⊓ b)

theorem Valuation.add_compl_le_one (v :
  Valuation Ω) (a : Ω) :
  v a + v ac ≤ 1

theorem Valuation.add_compl_eq_sup (v :
  Valuation Ω) (a : Ω) :
  v a + v ac = v (a ⊔ ac)

theorem Valuation.le_one (v : Valuation Ω)
  (a : Ω) : v a ≤ 1

noncomputable def slack (v : Valuation Ω)
  (a : Ω) : ℝ≥0∞ := 1 - (v a + v ac)

noncomputable def dnGap (v : Valuation Ω)
  (a : Ω) : ℝ≥0∞ := v acc - v a

noncomputable def deMorganGap (v :
  Valuation Ω) (a : Ω) : ℝ≥0∞ := 1 - (v acc + v ac)

theorem Valuation.
  slack_eq_dnGap_add_deMorganGap (v :
  Valuation Ω) (a : Ω) :
  v.slack a = v.dnGap a + v.deMorganGap
    a

theorem Valuation.slack_eq_sub_add_sub (v
  : Valuation Ω) (a : Ω) :
  v.slack a = (v acc - v a) + (1 - (v acc + v ac))

theorem Valuation.le_compl_compl (v :
  Valuation Ω) (a : Ω) : v a ≤ v acc

theorem Valuation.slack_submodular (v :
  Valuation Ω) (a b : Ω) :
  v.slack (a ⊔ b) + v.slack (a ⊓ b) ≤ v.
    slack a + v.slack b

def Order.Frame.IsDeMorgan (Ω : Type*) [
  Order.Frame Ω] : Prop :=
  ∀ a : Ω, acc ⊔ ac = ⊤

```

```

2751 theorem Valuation.
2752   slack_eq_dnGap_of_isDeMorgan
2753   (hΩ : Order.Frame.IsDeMorgan Ω) (v :
2754     Valuation Ω) (a : Ω) :
2755     v.slack a = v.dnGap a
2756
2757 noncomputable def Valuation.condVal (v :
2758   Valuation Ω) (b : Ω) (hb : v b ≠ 0) :
2759   Valuation Ω
2760
2761 theorem Valuation.condVal_mul (v :
2762   Valuation Ω) (b : Ω) (hb : v b ≠ 0) (a
2763   : Ω) :
2764   v.condVal b hb a * v b = v (a ⊓ b)
2765
2766 theorem Valuation.condVal_symm (v :
2767   Valuation Ω) {a b : Ω}
2768   (ha : v a ≠ 0) (hb : v b ≠ 0) :
2769   v.condVal b hb a * v b = v.condVal a
2770     ha b * v a
2771
2772 theorem Valuation.cond_add_compl_le (v :
2773   Valuation Ω) (a b : Ω) :
2774   v (a ⊓ b) + v (ac ⊓ b) ≤ v b
2775
2776 theorem Valuation.
2777   add_compl_eq_one_of_complemented (v :
2778   Valuation Ω) {a : Ω}
2779   (ha : a ⊔ ac = ⊤) : v a + v ac = 1
2780
2781 variable [CompleteBooleanAlgebra Ω]
2782
2783 theorem classical_additivity (v :
2784   Valuation Ω) (a : Ω) :
2785   v a + v ac = 1
2786
2787 variable [CompleteBooleanAlgebra Ω]
2788
2789 theorem classical_slack_zero (v :
2790   Valuation Ω) (a : Ω) :
2791   v.slack a = 0
2792
2793 noncomputable def Valuation.mix (w : ι → ℝ
2794   ≥ 0∞) (hw : ∑ i, w i = 1)
2795   (v : ι → Valuation Ω) : Valuation Ω
2796
2797
2798

```

B.3 Where Classicality Lives

```

2799 def Valuation.HasClassicalNegation (v :
2800   Valuation Ω) : Prop :=
2801   ∀ a, v a + v ac = 1
2802
2803 theorem hasClassicalNegation_of_em (hem :
2804   ∀ a : Ω, a ⊔ ac = ⊤)
2805

```

```

2806   (v : Valuation Ω) : v.
2807     HasClassicalNegation
2808
2809 theorem em_of_forall_hasClassicalNegation
2810   (h : ∀ v : Valuation Ω, v.
2811     HasClassicalNegation) : ∀ a : Ω, a
2812     ⊔ ac = ⊤
2813
2814 def Valuation.IsSharp (v : Valuation Ω) :
2815   Prop := ∀ a, v a = 0 ∨ v a = 1
2816
2817 noncomputable def Ideal.toValuation (J :
2818   Order.Ideal Ω) (hJ : J.IsPrime) (htop :
2819     ⊤ ∉ J) :
2820   Valuation Ω where
2821     toFun x := if x ∈ J then 0 else 1
2822
2823 theorem sharp_iff_point (v : Valuation Ω)
2824   :
2825     v.IsSharp ↔ ∃ (J : Order.Ideal Ω) (hJ
2826       : J.IsPrime) (htop : ⊤ ∉ J),
2827       v = Ideal.toValuation J hJ htop
2828
2829 def ScottContinuous (v : Valuation Ω) :
2830   Prop :=
2831     ∀ S : Set Ω, S.Nonempty → DirectedOn (·
2832       ≤ ·) S → v (sSup S) = ⊔ a ∈ S, v a
2833
2834 def Ideal.IsCompletelyPrime (J : Order.
2835   Ideal Ω) : Prop :=
2836     J.IsPrime ∧ ∀ S : Set Ω, (∀ x ∈ S, x ∈ J
2837       ) → sSup S ∈ J
2838
2839 theorem
2840   sharp_scott_iff_completelyPrimePoint (v
2841     : Valuation Ω) :
2842     v.IsSharp ∧ v.ScottContinuous ↔
2843     ∃ (J : Order.Ideal Ω) (hJ : Ideal.
2844       IsCompletelyPrime J) (htop : ⊤ ∉
2845       J),
2846     v = Ideal.toValuation J hJ.1 htop
2847

```

B.4 Valuation Representation

```

2850 noncomputable def Valuation.mass (v :
2851   Valuation (LowerSet P)) (p : P) : ℝ ≥ 0∞
2852
2853 noncomputable def LowerSet.toFinset (U :
2854   LowerSet P) : Finset P :=
2855   (Set.toFinite (U : Set P)).toFinset
2856
2857 theorem Valuation.eq_sum_mass (v :
2858   Valuation (LowerSet P)) (U : LowerSet P
2859   ) :
2860

```

2861	$v \ U = \sum p \in U.\text{toFinset}, v.\text{mass } p$	$(\text{hsc} : v \ T = \bigsqcup n, v \ (\text{LowerSet.Iic } n))$	2916
2862		$: v \ T = \sum' n, v.\text{mass } n$	2917
2863	theorem Valuation.sum_mass (v : Valuation		2918
2864	(LowerSet P)) : $\sum p, v.\text{mass } p = 1$	variable [$\forall p : P, \text{Finite } (\text{Set.Iic } p)$]	2919
2865			2920
2866	noncomputable def Valuation.toPMF (v :	theorem Valuation.isPurelyAtomic_of_scott	2921
2867	Valuation (LowerSet P)) : PMF P	(v : Valuation (LowerSet P))	2922
2868		(hsc : $v \ T = \bigsqcup (S : \text{Finset } P), v \ (S.$	2923
2869	def Valuation.mapEquiv (e : $\Omega \approx_o \Gamma$) (v :	sup LowerSet.Iic)) : IsPurelyAtomic	2924
2870	Valuation Ω) : Valuation Γ where	v	2925
2871	toFun b := v (e.symm b)		2926
2872		theorem tsum_mass_le (v : Valuation (	2927
2873	theorem Valuation.exists_pmf_presentation	LowerSet P)) : $\sum' p, v.\text{mass } p \leq v \ T$	2928
2874	{ $\Theta : \text{Type } u$ } [Order.Frame Θ] [Finite Θ]		2929
2875	(v : Valuation Θ) :		2930
2876	$\exists (P : \text{Type } u) (_ : \text{PartialOrder } P) (_$	B.5 The Dempster–Shafer Reading	2931
2877	: Fintype P) ($_ : \text{DecidableEq } P$)		2932
2878	(e : $\Theta \approx_o \text{LowerSet } P$),	noncomputable def iesOdd (v : Valuation Ω)	2933
2879	($\forall a, v \ a = \sum p \in (e \ a).\text{toFinset}, (v$	(s : Finset ι) (a : $\iota \rightarrow \Omega$) : $\mathbb{R}_{\geq 0}^\infty :=$	2934
2880	.mapEquiv e).mass p) $\wedge$	$\sum T \in s.\text{powerset}, \text{if } \text{Odd } T.\text{card} \text{ then } v \ ($	2935
2881	$\sum p, (v.\text{mapEquiv } e).\text{mass } p = 1$	T.inf a) else 0	2936
2882			2937
2883	noncomputable def deltaPoint (p : P) :	noncomputable def iesEven (v : Valuation Ω	2938
2884	Valuation (LowerSet P)	) (s : Finset ι) (a : $\iota \rightarrow \Omega$) : $\mathbb{R}_{\geq 0}^\infty :=$	2939
2885		$\sum T \in s.\text{powerset}, \text{if } T.\text{Nonempty} \wedge \text{Even } T$	2940
2886	theorem Valuation.eq_mix_deltaPoint (v :	.card then v (T.inf a) else 0	2941
2887	Valuation (LowerSet P)) :		2942
2888	v = Valuation.mix v.mass v.sum_mass (	theorem Valuation.inclusion_exclusion (v :	2943
2889	fun p => deltaPoint p)	Valuation Ω) (s : Finset ι)	2944
2890		(a : $\iota \rightarrow \Omega$) : v (s.sup a) + v.iesEven	2945
2891	noncomputable def topIndicator : Valuation	s a = v.iesOdd s a	2946
2892	(LowerSet $\mathbb{N}$) where		2947
2893	toFun U := if U = T then 1 else 0	theorem Valuation.infty_monotone (v :	2948
2894		Valuation Ω) (s : Finset ι) (a : $\iota \rightarrow \Omega$)	2949
2895	theorem	:	2950
2896	exists_valuation_not_point_representable	v.iesOdd s a $\leq v \ ((s.\text{sup } a)^{\text{cc}}) + v.$	2951
2897	:	iesEven s a	2952
2898	$\exists v : \text{Valuation } (\text{LowerSet } \mathbb{N}), \sum' p, v.$		2953
2899	mass p $\neq v \ T$	theorem Valuation.two_monotone (v :	2954
2900		Valuation Ω) (a b : Ω) :	2955
2901	theorem topIndicator_mass (n : $\mathbb{N}$) :	v a + v b $\leq v \ (a \sqcup b)^{\text{cc}} + v \ (a \sqcap b)$	2956
2902	topIndicator.mass n = 0		2957
2903		noncomputable def Valuation.plausibility (	2958
2904	theorem topIndicator_not_scott :	v : Valuation Ω) (a : Ω) : $\mathbb{R}_{\geq 0}^\infty := 1 -$	2959
2905	($\bigsqcup n, \text{topIndicator } (\text{LowerSet.Iic } n)) \neq$	v a ^c	2960
2906	topIndicator T		2961
2907		theorem Valuation.plausibility_sub_self (v	2962
2908	def IsPurelyAtomic (v : Valuation (	: Valuation Ω) (a : Ω) :	2963
2909	LowerSet P)) : Prop := $\sum' p, v.\text{mass } p =$	v.plausibility a - v a = v.slack a	2964
2910	v T		2965
2911		noncomputable def emGap (v : Valuation Ω)	2966
2912	theorem eq_tsum_mass_of_scott (v :	(a b : Ω) : $\mathbb{R}_{\geq 0}^\infty :=$	2967
2913	Valuation (LowerSet $\mathbb{N}$))	v a - v (a $\sqcap$ (b $\sqcup$ b ^c))	2968
2914			2969
2915			2970

```

2971 theorem Valuation.sup_compl_decomp (v :
2972   Valuation Ω) (a b : Ω) :
2973   v (a ⊔ bc) = v bc + (v (a ⊓ b) + v.
2974     emGap a b)
2975
2976 noncomputable def Valuation.dempsterCond (
2977   v : Valuation Ω) (b : Ω)
2978   (hb : v b ≠ 0) : Valuation Ω
2979
2980 theorem Valuation.dempsterCond_apply (v :
2981   Valuation Ω) (b : Ω) (hb : v b ≠ 0)
2982   (a : Ω) : v.dempsterCond b hb a = (v (
2983     a ⊔ bc) - v bc) / (1 - v bc)
2984
2985 theorem Valuation.
2986   dempsterCond_eq_condVal_iff_slack (v :
2987     Valuation Ω) {b : Ω}
2988     (hb : v b ≠ 0) :
2989     (∀ a, v.dempsterCond b hb a = v.
2990       condVal b hb a) ↔ v.slack b = 0

```

B.6 Cox's Theorem

```

2994 structure CoxModel (Ω : Type*) [Order.
2995   Frame Ω] where
2996   pl : Ω → Ω → ℝ
2997   mono_left : ∀ c, Monotone fun a => pl a
2998     c
2999   pl_bot : ∀ c, pl ⊥ c = 0
3000   pl_top : ∀ c, pl ⊤ c = 1
3001   F : ℝ → ℝ → ℝ
3002   conj : ∀ a b c, pl (a ⊓ b) c = F (pl a (
3003     b ⊓ c)) (pl b c)
3004   F_assoc : ∀ x y z, F (F x y) z = F x (F
3005     y z)
3006   F_strictMono_left : ∀ y, 0 < y →
3007     StrictMono fun x => F x y
3008
3009 theorem isDeMorgan_lowerSetV : Order.Frame
3010   .IsDeMorgan (LowerSet V)
3011
3012 /-- `o` below the incomparable `a` and `b
3013   `.-/
3014 inductive V : Type
3015   | o | a | b
3016
3017 noncomputable def qV (S : LowerSet V) : ℝ≥0
3018   :=
3019   if V.o ∈ S then (if V.a ∈ S ∧ V.b ∈ S
3020     then 1 else 2-1) else 0
3021
3022 theorem modularity_irreducible :
3023   ∃ (Ω : Type) (· : Order.Frame Ω) (q :
3024     Ω → ℝ≥0),
3025

```

```

3026   Monotone q ∧ q ⊥ = 0 ∧ q ⊤ = 1 ∧
3027   (∀ x y : Ω, x ⊓ y = ⊥ → q (x ⊔ y) =
3028     q x + q y) ∧
3029   ¬ (∀ x y : Ω, q x + q y = q (x ⊔ y)
3030     + q (x ⊓ y))
3031
3032 theorem no_disjunction_functional :
3033   ¬ ∃ S : ℝ≥0 → ℝ≥0 → ℝ≥0,
3034     ∀ x y : LowerSet V, qV (x ⊔ y) = S
3035       (qV x) (qV y)
3036
3037 theorem constructive_cox (M : CoxModel Ω)
3038   (hmod : ∀ a b : Ω, M.pl a ⊤ + M.pl b ⊤
3039     = M.pl (a ⊔ b) ⊤ + M.pl (a ⊓ b) ⊤) :
3040   :
3041   ∃ (v : Valuation Ω) (g : ℝ → ℝ≥0),
3042     StrictMonoOn g (Set.Icc 0 1) ∧ g 0 =
3043       0 ∧ g 1 = 1 ∧
3044     ∀ a, v a = g (M.pl a ⊤)
3045
3046 noncomputable def coxModelENNReal :
3047   CoxModel ℝ≥0
3048
3049 theorem Scale.exists_mul_generator :
3050   ∃ G : ℝ → ℝ,
3051     (∀ x y, S.u ≤ x → S.u ≤ y → G (S.F x
3052       y) = G x * G y) ∧
3053     (∀ x y, S.u ≤ x → x < y → G x < G y)
3054
3055 theorem Scale.aczelStatement_cone :
3056   ∃ G : ℝ → ℝ,
3057     (∀ x y, S.u ≤ x → S.u ≤ y → G (S.F x
3058       y) = G x * G y) ∧
3059     StrictMonoOn G (Set.Ici S.u)
3060
3061 theorem exists_diag_eq {F : ℝ → ℝ → ℝ} {a
3062   b c : ℝ} (hab : a ≤ b)
3063   (hcont : ContinuousOn (fun x => F x x)
3064     (Set.Icc a b))
3065   (hc : c ∈ Set.Icc (F a a) (F b b)) : ∃
3066     x ∈ Set.Icc a b, F x x = c
3067
3068 structure Scale where
3069   F : ℝ → ℝ → ℝ
3070   hdiv : ∀ c, ∃ x, F x x = c
3071   hassoc : ∀ x y z, F (F x y) z = F x (F y
3072     z)
3073   hpos : ∀ c x, x < F x c
3074   hmono : ∀ x1 x2 y1 y2 : ℝ, x1 ≤ x2 → y1
3075     ≤ y2 → F x1 y1 ≤ F x2 y2
3076   hcont : Continuous fun p : ℝ × ℝ => F p
3077     .1 p.2
3078   hbot : ∀ x, Filter.Tendsto (fun z => F x
3079     z) Filter.atBot (nhds x)
3080

```

B.7 Two Grounding Models

u : $\mathbb{R}$

noncomputable def MeasureTheory.Measure.
toValuationOpens
(μ : Measure X) [IsProbabilityMeasure
 μ] : Valuation (Opens X)

theorem toValuationOpens_slack_eq_frontier
(μ : Measure X)
[IsProbabilityMeasure μ] (U : Opens X)
:
 μ .toValuationOpens.slack U = μ (
frontier (U : Set X))

theorem classical_posterior_compl_eq (μ :
Measure X)
[IsProbabilityMeasure μ] (a b : Opens
X)
(hb : μ .toValuationOpens b $\neq$ 0) :
 μ ((a : Set X)^c $\cap$ b) / μ b
= μ .toValuationOpens.condVal b hb a^c
+ μ (frontier (a : Set X) $\cap$ b) / μ
b

noncomputable def MeasureTheory.Measure.
interiorMeasure (μ : Measure X) (S :
Set X) :
 $\mathbb{R}_{\geq 0\infty}$:= μ (interior S)

theorem interiorMeasure_add_compl_le (μ :
Measure X) [IsProbabilityMeasure μ]
(S : Set X) : μ .interiorMeasure S + μ .
interiorMeasure S^c $\leq$ 1

theorem
isDeMorgan_opens_iff_extremallyDisconnected
:
Order.Frame.IsDeMorgan (Opens X) $\leftrightarrow$
ExtremallyDisconnected X

theorem isDeMorgan_opens_iff_stonean [
CompactSpace X] [T2Space X] :
Order.Frame.IsDeMorgan (Opens X) $\leftrightarrow$
ExtremallyDisconnected X

inductive ThreePt : Type
| ptA : ThreePt
| ptB : ThreePt
| ptZ : ThreePt

theorem frontier_subset_ptZ {U : Set
ThreePt} (hU : IsOpen U) :

frontier U $\subseteq$ {ptZ}

theorem exists_ne_valuation_same_slack :
 $\exists v v' : \text{Valuation (Opens ThreePt)}, v.$
slack = v'.slack $\wedge v \neq v'$

theorem haltsOpen_compl : haltsOpen^c = ($\perp$
: Opens Prop)

def haltsOpen : Opens Prop := {q | q},
isOpen_setOf_self

noncomputable def coinMeasure (p : $\mathbb{R}_{\geq 0\infty}$) :
Measure Prop :=
p • Measure.dirac True + (1 - p) •
Measure.dirac False

noncomputable def sierpinskiValuation {p :
 $\mathbb{R}_{\geq 0\infty}$ } (hp1 : p $\leq$ 1) : Valuation (Opens
Prop) :=
(coinMeasure p).toValuationOpens

theorem slack_eq_boundary {p : $\mathbb{R}_{\geq 0\infty}$ } (hp1
: p $\leq$ 1) :
(sierpinskiValuation hp1).slack
haltsOpen
= coinMeasure p (frontier {q : Prop
| q})

noncomputable def haltingWeight (n : $\mathbb{N}$) (c
: Code) : $\mathbb{R}_{\geq 0\infty}$:=
if (c.eval n).Dom then 1 else 0

noncomputable def sharpReadout (n : $\mathbb{N}$) (c
: Code) : Valuation (Opens Prop) :=
sierpinskiValuation (
haltingWeight_le_one n c)

theorem sharpReadout_not_computable (n : $\mathbb{N}$
) :
 $\neg \text{ComputablePred fun c : Code =>}$
sharpReadout n c haltsOpen = 1

-- One fair coin flip, `Ber(true, false,
1/2)`.

noncomputable def fairCoin : Measure Bool
:= Ber(true, false, halfI)

noncomputable def cantorMeasure : Measure
($\mathbb{N} \rightarrow \text{Bool}$) :=
Measure.infinitePi (fun _ : $\mathbb{N} \Rightarrow$
fairCoin)

```

3191 noncomputable def wordRestriction (w :
3192   List Bool) :  $\mathbb{N} \rightarrow \text{Set Bool} :=
3193   \text{fun } i \Rightarrow \text{if } h : i < w.\text{length} \text{ then } \{w[i]\}
3194   \text{ else } \text{Set.univ}
3195
3196 def wordCylinder (w : List Bool) : Set ( $\mathbb{N}$ 
3197    $\rightarrow$  Bool) :=
3198   Set.pi (Finset.range w.length : Set  $\mathbb{N}$ ) (
3199     wordRestriction w)
3200
3201 def PrefixFree (F : Finset (List Bool)) :
3202   Prop :=
3203    $\forall v \in F, \forall w \in F, v \neq w \rightarrow \neg v <+: w$ 
3204
3205 theorem isOpen_wordCylinder (w : List Bool
3206   ) : IsOpen (wordCylinder w)
3207
3208 theorem cantorMeasure_wordCylinder (w :
3209   List Bool) :
3210   cantorMeasure (wordCylinder w) = (1 /
3211     2) ^ w.length
3212
3213 theorem
3214   disjoint_wordCylinder_of_not_prefix {v
3215     w : List Bool}
3216   (hv :  $\neg v <+: w$ ) (hw :  $\neg w <+: v$ ) :
3217   Disjoint (wordCylinder v) (
3218     wordCylinder w)
3219
3220 theorem cantorMeasure_biUnion_wordCylinder
3221   {F : Finset (List Bool)}
3222   (hF : PrefixFree F) :
3223   cantorMeasure ( $\bigcup w \in F, \text{wordCylinder } w$ 
3224     ) =  $\sum w \in F, (1 / 2 : \mathbb{R}_{\geq 0\infty}) ^ w.$ 
3225     length
3226
3227 B.8 The Monad Question
3228
3229 def Valuation.comap (f : FrameHom  $\Gamma \Omega$ ) (v
3230   : Valuation  $\Omega$ ) : Valuation  $\Gamma$ 
3231
3232 theorem Valuation.comap_id (v : Valuation
3233    $\Omega$ ) : v.comap (FrameHom.id  $\Omega$ ) = v
3234
3235 theorem Valuation.comap_comp (f : FrameHom
3236    $\Gamma \Omega$ ) (g : FrameHom  $\Delta \Gamma$ )
3237   (v : Valuation  $\Omega$ ) : v.comap (f.comp g)
3238   = (v.comap f).comap g
3239
3240 theorem exists_rect_decomposition (W :
3241   LowerSet ( $\mathbb{N} \times \mathbb{N}$ )) :
3242    $\exists (N : \mathbb{N}) (A B : \mathbb{N} \rightarrow \text{LowerSet } \mathbb{N}),$ 
3243     W = (Finset.range N).sup fun i => A
3244     i  $\times^s$  B i
3245$ 
```

```

theorem Valuation.eq_on_finsetSup_of_eq_on
  {G : Set  $\Omega$ }
  (hG :  $\forall g \in G, \forall g' \in G, g \sqcap g' \in G$ ) (
  u u' : Valuation  $\Omega$ )
  (h :  $\forall g \in G, u g = u' g$ ) { $\iota$  : Type*}
  [DecidableEq  $\iota$ ]
  (s : Finset  $\iota$ ) (a :  $\iota \rightarrow \Omega$ ) (ha :  $\forall i \in$ 
  s, a i  $\in G$ ) :
  u (s.sup a) = u' (s.sup a)
3246
3247
3248
3249
3250
3251
3252
3253
3254
3255
3256 theorem Valuation.product_unique (v w :
  Valuation (LowerSet  $\mathbb{N}$ ))
3257
3258 {u u' : Valuation (LowerSet ( $\mathbb{N} \times \mathbb{N}$ ))}
3259 (hu :  $\forall A B : \text{LowerSet } \mathbb{N}, u (A \times^s B) =$ 
  v A * w B)
3260
3261 (hu' :  $\forall A B : \text{LowerSet } \mathbb{N}, u' (A \times^s B)$ 
  = v A * w B) : u = u'
3262
3263
3264 noncomputable def ultrafilterValuation (W
  : Ultrafilter  $\iota$ ) : Valuation (Set  $\iota$ )
3265
3266
3267 theorem rect_mem_iff { $\alpha \beta$  : Type*} {W :
  Ultrafilter ( $\alpha \times \beta$ )}
3268
3269 {S : Set  $\alpha$ } {T : Set  $\beta$ } :
3270 S  $\times^s$  T  $\in W \leftrightarrow S \in W.\text{map Prod.fst} \wedge T \in$ 
  W.map Prod.snd
3271
3272
3273 theorem product_valuation_not_unique :
   $\exists u u' : \text{Valuation (Set } (\mathbb{N} \times \mathbb{N}))$ ,
3274
  ( $\forall S T : \text{Set } \mathbb{N}, u (S \times^s T) = u' (S \times$ 
   $^s T) \wedge u \neq u'$ 
3275
3276
3277
3278 theorem fubini_fails_for_valuations :
   $\exists (v w : \text{Valuation (Set } \mathbb{N})) (u u' :$ 
  Valuation (Set ( $\mathbb{N} \times \mathbb{N}$ ))),
3279
  ( $\forall S T : \text{Set } \mathbb{N}, u (S \times^s T) = v S * w$ 
  T)  $\wedge$ 
3280
  ( $\forall S T : \text{Set } \mathbb{N}, u' (S \times^s T) = v S *$ 
  w T)  $\wedge u \neq u'$ 
3281
3282
3283
3284
3285
3286 noncomputable def Valuation.mixCountable {
   $\iota$  : Type*} (w :  $\iota \rightarrow \mathbb{R}_{\geq 0\infty}$ )
3287
  (hw :  $\sum' i, w i = 1$ ) (v :  $\iota \rightarrow$ 
  Valuation  $\Omega$ ) : Valuation  $\Omega$ 
3288
3289
3290
3291 theorem Valuation.
  mixCountable_mixCountable { $\iota$  : Type*} {
   $\kappa$  :  $\iota \rightarrow \text{Type*}$ }
3292
  (w :  $\iota \rightarrow \mathbb{R}_{\geq 0\infty}$ ) (hw :  $\sum' i, w i = 1$ )
  (u :  $\forall i, \kappa i \rightarrow \mathbb{R}_{\geq 0\infty}$ ) (hu :  $\forall i, \sum' j,$ 
  u i j = 1)
3293
  (v :  $\forall i, \kappa i \rightarrow \text{Valuation } \Omega$ ) :
  Valuation.mixCountable w hw
3294
3295
3296
3297
3298
3299
3300

```

```

3301      (fun i => Valuation.mixCountable (
3302        u i) (hu i) (v i))
3303    = Valuation.mixCountable (fun p :  $\Sigma$ 
3304      i,  $\kappa$  i => w p.1 * u p.1 p.2)
3305      (tsum_sigma_mul_eq_one w hw u hu
3306        ) (fun p => v p.1 p.2)
3307
3308  theorem Valuation.mixCountable_unique (v :
3309    Valuation  $\Omega$ ) :
3310    Valuation.mixCountable (fun _ : PUnit
3311      => 1) (by simp) (fun _ => v) = v
3312
3313  theorem Valuation.mixCountable_comm { $\iota$   $\kappa$  :
3314    Type*}
3315    (w :  $\iota \rightarrow \mathbb{R}_{\geq 0\infty}$ ) (hw :  $\sum' i, w i = 1$ ) (u
3316      :  $\kappa \rightarrow \mathbb{R}_{\geq 0\infty}$ ) (hu :  $\sum' j, u j = 1$ )
3317    (v :  $\iota \rightarrow \kappa \rightarrow \text{Valuation } \Omega$ ) :
3318    Valuation.mixCountable w hw (fun i =>
3319      Valuation.mixCountable u hu (v i))
3320    = Valuation.mixCountable u hu
3321      (fun j => Valuation.mixCountable
3322        w hw (fun i => v i j))
3323
3324
3325

```

B.9 Valuations Beyond Frames

Theorem 57.

```

3327  theorem exists_sharp_separating {a b :  $\Omega$ }
3328    (hab :  $\neg a \leq b$ ) :
3329     $\exists v : \text{Valuation } \Omega, v.\text{IsSharp} \wedge v a = 1$ 
3330       $\wedge v b = 0$ 
3331
3332

```

Definition 51.

```

3333  structure LValuation ( $\Omega : \text{Type}^*$ ) [Lattice
3334     $\Omega$ ] [BoundedOrder  $\Omega$ ] where
3335    toFun :  $\Omega \rightarrow \mathbb{R}_{\geq 0\infty}$ 
3336    map_bot' : toFun  $\perp = 0$ 
3337    map_top' : toFun  $\top = 1$ 
3338    mono' : Monotone toFun
3339    modular' :  $\forall a b, \text{toFun } a + \text{toFun } b$ 
3340      = toFun (a  $\sqcup$  b) + toFun (a  $\sqcap$ 
3341        b)
3342
3343

```

Proposition 52.

```

3344  def LValuation.comap (f :
3345    BoundedLatticeHom  $\Gamma$   $\Omega$ ) (v : LValuation
3346     $\Omega$ ) :
3347    LValuation  $\Gamma$ 
3348
3349  theorem LValuation.comap_id (v :
3350    LValuation  $\Omega$ ) :
3351    v.comap (BoundedLatticeHom.id  $\Omega$ ) = v
3352
3353  theorem LValuation.comap_comp (f :
3354    BoundedLatticeHom  $\Gamma$   $\Omega$ )
3355

```

```

(g : BoundedLatticeHom  $\Delta$   $\Gamma$ ) (v :
  LValuation  $\Omega$ ) :
  v.comap (f.comp g) = (v.comap f).comap
    g

```

Theorem 58.

```

def LValuation.IsSharp (v : LValuation  $\Omega$ )
  : Prop :=  $\forall a, v a = 0 \vee v a = 1$ 

theorem LValuation.exists_sharp_separating
  [DistribLattice  $\Omega$ ]
  {a b :  $\Omega$ } (hab :  $\neg a \leq b$ ) :
   $\exists v : \text{LValuation } \Omega, v.\text{IsSharp} \wedge v a =$ 
    1  $\wedge v b = 0$ 

```

Definition 59.

```

inductive M3 : Type
  | z | a | b | c | o

instance : Lattice M3 where
  -- z bottom, o top, a b c pairwise-
  -- incomparable atoms;
  -- any two distinct atoms join to o and
  -- meet to z

instance : BoundedOrder M3 where
  top := o
  bot := z

theorem M3.not_distributive :
   $\neg \forall x y w : M3, x \sqcap (y \sqcup w) = (x \sqcap y)$ 
     $\sqcup (x \sqcap w)$ 

```

Theorem 60.

```

theorem M3.no_sharp_valuation (v :
  LValuation M3) :  $\neg v.\text{IsSharp}$ 

```

Definition 61.

```

def IsTensorValuation (v :  $\Omega \rightarrow \mathbb{R}_{\geq 0\infty}$ ) (t :
   $\Omega \rightarrow \Omega \rightarrow \Omega$ ) : Prop :=
   $\forall a b, v a + v b = v (a \sqcup b) + v (t a b)$ 

```

Theorem 62.

```

theorem IsTensorValuation.collapse {v :  $\Omega$ 
   $\rightarrow \mathbb{R}_{\geq 0\infty}$ } {t :  $\Omega \rightarrow \Omega \rightarrow \Omega$ }
  (h : IsTensorValuation v t) {a :  $\Omega$ } (
    ha : v a  $\neq \top$ ) :
    v (t a a) = v a

```

Example 63.

```

theorem forced_constant_of_add_valuation {
  v :  $\mathbb{N} \rightarrow \mathbb{R}_{\geq 0\infty}$ }
  (h : IsTensorValuation v ( $\cdot + \cdot$ )) (
    hfin :  $\forall n, v n \neq \top$ ) (n :  $\mathbb{N}$ ) :

```

```

3411     v n = v 0
3412 Corollary 64.
3413
3414 theorem
3415   not_isTensorValuation_of_discriminates
3416   {v :  $\Omega \rightarrow \mathbb{R}_{\geq 0\infty}$ }
3417   {t :  $\Omega \rightarrow \Omega \rightarrow \Omega$ } {a :  $\Omega$ } (ha : v a  $\neq$  T)
3418   (hne : v (t a a)  $\neq$  v a) :
3419    $\neg$  IsTensorValuation v t
3420

```

B.10 Probability Calculi from Certain States

```

3421
3422
3423 def IsSharpVec (x : E  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) : Prop :=  $\forall$ 
3424   e, x e = 0  $\vee$  x e = 1
3425
3426 def Mix (S : Set (E  $\rightarrow \mathbb{R}_{\geq 0\infty}$ )) : Set (E  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) :=
3427   {x |  $\exists$  (n :  $\mathbb{N}$ ) (w : Fin n  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) (s :
3428     Fin n  $\rightarrow$  E  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ),
3429     ( $\forall$  i, s i  $\in$  S)  $\wedge \sum$  i, w i = 1  $\wedge$  x =  $\sum$ 
3430       i, w i • s i}
3431
3432
3433 theorem sharp_mem_Mix_iff {S : Set (E  $\rightarrow \mathbb{R}_{\geq 0\infty}$ )} (hS :  $\forall$  s  $\in$  S, IsSharpVec s)
3434   {x : E  $\rightarrow \mathbb{R}_{\geq 0\infty}$ } (hx : IsSharpVec x) : x
3435    $\in$  Mix S  $\leftrightarrow$  x  $\in$  S
3436
3437
3438 theorem Mix_Mix (S : Set (E  $\rightarrow \mathbb{R}_{\geq 0\infty}$ )) : Mix
3439   (Mix S) = Mix S
3440
3441
3442
3443 structure AffineLaw (E : Type*) [Fintype E]
3444   where
3445     a : E  $\rightarrow \mathbb{R}_{\geq 0\infty}$ 
3446     b : E  $\rightarrow \mathbb{R}_{\geq 0\infty}$ 
3447     k :  $\mathbb{R}_{\geq 0\infty}$ 
3448     k' :  $\mathbb{R}_{\geq 0\infty}$ 
3449
3450 def AffineLaw.Holds [Fintype E] (L :
3451   AffineLaw E) (x : E  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) : Prop :=
3452    $\sum$  e, L.a e * x e + L.k  $\leq \sum$  e, L.b e * x
3453     e + L.k'
3454
3455 theorem AffineLaw.holds_of_Mix [Fintype E]
3456   (L : AffineLaw E) {S : Set (E  $\rightarrow \mathbb{R}_{\geq 0\infty}$ )}
3457   (hL :  $\forall$  s  $\in$  S, L.Holds s) {x : E  $\rightarrow \mathbb{R}_{\geq 0\infty}$ }
3458   (hx : x  $\in$  Mix S) : L.Holds x
3459
3460 theorem Mix_eq_laws' {S : Set (E  $\rightarrow \mathbb{R}_{\geq 0\infty}$ )}
3461   (hS :  $\forall$  s  $\in$  S, IsSharpVec s) :
3462   Mix S = {x |  $\forall$  L : AffineLaw E, ( $\forall$  s  $\in$ 
3463     S, L.Holds s)  $\rightarrow$  L.Holds x}
3464
3465

```

```

3466 theorem frame_recipe :
3467   Set.range (fun v : Valuation (LowerSet
3468     P) => v.toFun)
3469   = Mix {f |  $\exists$  v : Valuation (LowerSet
3470     P), v.IsSharp  $\wedge$  f = v.toFun}
3471
3472 theorem Valuation.eq_deltaPoint_of_isSharp
3473   {v : Valuation (LowerSet P)} (hv : v.
3474     IsSharp) :
3475      $\exists$  p, v = deltaPoint p
3476
3477 structure FinLaw (E : Type*) where
3478   F : Finset E
3479   a : E  $\rightarrow \mathbb{R}_{\geq 0\infty}$ 
3480   b : E  $\rightarrow \mathbb{R}_{\geq 0\infty}$ 
3481   k :  $\mathbb{R}_{\geq 0\infty}$ 
3482   k' :  $\mathbb{R}_{\geq 0\infty}$ 
3483
3484 def FinLaw.Holds (L : FinLaw E) (x : E  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) : Prop :=
3485    $\sum$  e  $\in$  L.F, L.a e * x e + L.k  $\leq \sum$  e  $\in$  L.F
3486     , L.b e * x e + L.k'
3487
3488
3489 def restr (F : Finset E) (x : E  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) :
3490   F  $\rightarrow \mathbb{R}_{\geq 0\infty}$  := fun e => x e
3491
3492 theorem laws_iff_local_Mix {S : Set (E  $\rightarrow \mathbb{R}_{\geq 0\infty}$ )} (hS :  $\forall$  s  $\in$  S, IsSharpVec s)
3493   (x : E  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) :
3494   ( $\forall$  L : FinLaw E, ( $\forall$  s  $\in$  S, L.Holds s)
3495      $\rightarrow$  L.Holds x)  $\leftrightarrow$ 
3496      $\forall$  F : Finset E, restr F x  $\in$  Mix (
3497       restr F '' S)
3498
3499
3500 noncomputable def ofVec (s : Ev  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) :
3501   Cfg Ev := fun e => decide (s e = 1)
3502
3503 noncomputable def testFn {U : Set (Cfg Ev)}
3504   (hU : IsClopen U) : Cfg Ev  $\rightarrow^b \mathbb{R}_{\geq 0}$ 
3505
3506 def Cfg (Ev : Type) := Ev  $\rightarrow$  Bool
3507
3508 def Good (S : Set (Ev  $\rightarrow \mathbb{R}_{\geq 0\infty}$ )) (F : Finset
3509   Ev) ( $\sigma$  : Cfg Ev) : Prop :=
3510    $\exists$  s  $\in$  S,  $\forall$  e : F, s e.1 = vec  $\sigma$  e.1
3511
3512 def atom (e : Ev) : Set (Cfg Ev) := { $\sigma$  |  $\sigma$ 
3513   e = true}
3514
3515 def bad (S : Set (Ev  $\rightarrow \mathbb{R}_{\geq 0\infty}$ )) (F : Finset
3516   Ev) : Set (Cfg Ev) := { $\sigma$  |  $\neg$  Good S F  $\sigma$ 
3517   }
3518
3519
3520

```

```

3521 theorem laws_iff_barycenter {S : Set (Ev →
3522   ℝ≥0∞)} (hS : ∀ s ∈ S, IsSharpVec s)
3523   (x : Ev → ℝ≥0∞) :
3524   (∀ L : FinLaw Ev, (∀ s ∈ S, L.Holds s)
3525     → L.Holds x) ↔
3526     ∃ μ : ProbabilityMeasure (Cfg Ev), (
3527       ∀ F, (μ : Measure (Cfg Ev)) (bad
3528         S F) = 0) ∧
3529       ∀ e, x e = (μ : Measure (Cfg Ev))
3530         (atom e)
3531
3532 def flat (X : Type*) [Fintype X] : Set (X
3533   → ℝ≥0∞) := {x | ∑ a, x a ≤ 1}
3534
3535 noncomputable def app (t : X × Y → ℝ≥0∞) (
3536   x : X → ℝ≥0∞) : Y → ℝ≥0∞ :=
3537   fun b => ∑ a, x a * t (a, b)
3538
3539 def lin (X Y : Type*) [Fintype X] [Fintype
3540   Y] : Set (X × Y → ℝ≥0∞) :=
3541   {t | ∀ x ∈ flat X, app t x ∈ flat Y}
3542
3543 theorem mem_lin_iff [DecidableEq X] (t : X
3544   × Y → ℝ≥0∞) :
3545   t ∈ lin X Y ↔ ∀ a, ∑ b, t (a, b) ≤ 1
3546
3547 noncomputable def pairing (x y : E → ℝ≥0∞)
3548   : ℝ≥0∞ := ∑ e, x e * y e
3549
3550 def row (a : X) : X × Y → ℝ≥0∞ := fun ab
3551   => if ab.1 = a then 1 else 0
3552
3553 def orth (S : Set (E → ℝ≥0∞)) : Set (E → ℝ
3554   ≥0∞) := {y | ∀ x ∈ S, pairing x y ≤ 1}
3555
3556 structure IsPCS (P : Set (E → ℝ≥0∞)) :
3557   Prop where
3558     closed : orth (orth P) = P
3559     total : ∀ e, ∃ x ∈ P, x e ≠ 0
3560     bounded : ∀ e, ∃ M ≠ 0, ∀ x ∈ P, x e ≤ M
3561
3562 theorem isPCS_lin : IsPCS (lin X Y)
3563
3564 theorem isPCS_flat : IsPCS (flat X)
3565
3566 theorem isPCS_tensDual : IsPCS tensDual
3567
3568 theorem pcs_recipe {X Y : Type} [Fintype X
3569   ] [Fintype Y] [DecidableEq X] :
3570   lin X Y = Mix (Set.range (det : (X →
3571     Option Y) → X × Y → ℝ≥0∞))
3572
3573 def flatC (X : Type*) : X → X → Prop :=
3574   fun x y => x = y
3575
3576 def dualC {X : Type*} (c : X → X → Prop) :
3577   X → X → Prop := fun x y => x = y ∨ ¬ c
3578   x y
3579
3580 def tensC {X Y : Type*} (c : X → X → Prop)
3581   (d : Y → Y → Prop) : X × Y → X × Y →
3582   Prop :=
3583   fun p q => c p.1 q.1 ∧ d p.2 q.2
3584
3585 def arrowC {X Y : Type*} (c : X → X → Prop
3586   ) (d : Y → Y → Prop) : X × Y → X × Y →
3587   Prop :=
3588   dualC (tensC c (dualC d))
3589
3590 def IsClique {X : Type*} (c : X → X → Prop
3591   ) (B : X → Prop) : Prop := ∀ x y, B x →
3592   B y → c x y
3593
3594 def secondC : W × W → W × W → Prop :=
3595   dualC (tensC (arrowC (flatC Bool) (flatC
3596     Bool)) (arrowC (flatC Bool) (flatC
3597     Bool)))
3598
3599 noncomputable def throughAll (C : Finset W
3600   ) : Bool → Option Bool := fun a =>
3601   if h : ∃ p ∈ C, p.1 = a then some (
3602     Classical.choose h).2 else none
3603
3604 theorem sharp_lin_iff_clique {X Y : Type*}
3605   [Fintype X] [Fintype Y] [DecidableEq X
3606   ] [DecidableEq Y]
3607   {t : X × Y → ℝ≥0∞} (h01 : ∀ p, t p = 0
3608     ∨ t p = 1) :
3609   t ∈ lin X Y ↔ IsClique (arrowC (flatC
3610     X) (flatC Y)) (t · = 1)
3611
3612 def cov (f : X → Option Y) (g : X' →
3613   Option Y') (pq : (X × Y) × (X' × Y')) :
3614   Prop :=
3615   f pq.1.1 = some pq.1.2 ∧ g pq.2.1 = some
3616   pq.2.2
3617
3618 def cp {X Y : Type} (p q : X × Y) : Prop
3619   := p = q ∨ p.1 ≠ q.1
3620
3621 def cp2 (s t : (X × Y) × (X' × Y')) : Prop
3622   := cp s.1 t.1 ∧ cp s.2 t.2
3623
3624 noncomputable def thru {X Y : Type} [
3625   DecidableEq X] (C : Finset (X × Y)) : X
3626   → Option Y :=
3627   fun a => if h : ∃ p ∈ C, p.1 = a then
3628   some (Classical.choose h).2 else none
3629
3630

```

```

3631
3632 def GTree (c : X') (af : Y' → X) (pq : (X
3633   × Y) × (X' × Y')) : Prop :=
3634   pq.2.1 = c ∧ pq.1.1 = af pq.2.2
3635
3636 def tD (X Y X' Y' : Type) [Fintype X] [
3637   Fintype Y] [Fintype X'] [Fintype Y'] :
3638   Set ((X × Y) × (X' × Y') → ℝ≥0) :=
3639   {t | ∀ y ∈ lin X Y, ∀ z ∈ lin X' Y', ∑ p
3640     , ∑ q, y p * t (p, q) * z q ≤ 1}
3641
3642 def FTree (a : X) (cf : Y → X') (pq : (X ×
3643   Y) × (X' × Y')) : Prop :=
3644   pq.1.1 = a ∧ pq.2.1 = cf pq.1.2
3645
3646 theorem sharp_tD_iff [Nonempty X] [
3647   Nonempty X'] {z : (X × Y) × (X' × Y') →
3648     ℝ≥0}
3649   (h01 : ∀ pq, z pq = 0 ∨ z pq = 1) :
3650   z ∈ tD X Y X' Y' ↔
3651   (∃ a cf, ∀ pq, z pq = 1 → FTree a cf
3652     pq) ∨ (∃ c af, ∀ pq, z pq = 1 →
3653     GTree c af pq)
3654
3655 noncomputable def Core.val (K : Core A B C
3656   D) (a : A) (b : B) (c : C) (d : D) : ℝ
3657   ≥0 :=
3658   K.μ a * K.κ a b c * K.ρ a b c d
3659
3660 noncomputable def Core.mass (K : Core A B
3661   C D) : ℝ≥0 := ∑ a, K.μ a
3662
3663 theorem sharp_mem_sequentialG [Nonempty X]
3664   [Nonempty X'] {z : (X × Y) × (X' × Y')
3665     → ℝ≥0}
3666   (hz : z ∈ tD X Y X' Y') (h01 : ∀ u, z
3667     u = 0 ∨ z u = 1) : z ∈ sequentialG
3668     X Y X' Y'
3669
3670 structure Core (A B C D : Type) [Fintype A
3671   ] [Fintype C] where
3672   μ : A → ℝ≥0
3673   κ : A → B → C → ℝ≥0
3674   ρ : A → B → C → D → ℝ≥0
3675   hκ : ∀ a b, ∑ c, κ a b c ≤ 1
3676   hρ : ∀ a b c d, ρ a b c d ≤ 1
3677
3678 def sequentialG (X Y X' Y' : Type) [
3679   Fintype X] [Fintype X'] : Set ((X × Y)
3680   × (X' × Y') → ℝ≥0) :=
3681   {t | ∃ (F : Core X Y X' Y') (G : Core X'
3682     Y' X Y), F.mass + G.mass ≤ 1 ∧
3683
3684
3685
3686   t = fun pq => F.val pq.1.1 pq.1.2 pq
3687     .2.1 pq.2.2 + G.val pq.2.1 pq.2.2
3688     pq.1.1 pq.1.2}
3689
3690 theorem recipe_eq_sequentialG [Nonempty X]
3691   [Nonempty X'] :
3692   Mix {z | z ∈ tD X Y X' Y' ∧ ∀ u, z u =
3693     0 ∨ z u = 1} = sequentialG X Y X'
3694     Y'
3695
3696 def sharpSeqG (X Y X' Y' : Type) [Fintype
3697   X] [Fintype X'] : Set ((X × Y) × (X' ×
3698   Y') → ℝ≥0) :=
3699   {z | z ∈ sequentialG X Y X' Y' ∧ ∀ u, z
3700     u = 0 ∨ z u = 1}
3701
3702 theorem sequentialG_subset_Mix :
3703   sequentialG X Y X' Y' ⊆ Mix (sharpSeqG
3704     X Y X' Y')
3705
3706 noncomputable def compl' (m : A → ℝ≥0) :
3707   Option A → ℝ≥0
3708   | none => 1 - ∑ a, m a
3709   | some a => m a
3710
3711 abbrev ChoiceG (A B C D : Type) := (Unit →
3712   Option A) × (A × B → Option C) × (A ×
3713   B × C × D → Bool)
3714
3715 noncomputable def detCoreG [DecidableEq A]
3716   [DecidableEq C] (σ : ChoiceG A B C D)
3717   : Core A B C D
3718
3719 theorem Mix_sequentialG : Mix (sequentialG
3720   X Y X' Y') = sequentialG X Y X' Y'
3721
3722 def emb (s : W × W) : (X × Y) × (X' × Y')
3723   :=
3724   ((ex s.1.1, ey s.1.2), (ex' s.2.1, ey' s
3725     .2.2))
3726
3727 noncomputable def pentE (ex : Bool → X) (
3728   ey : Bool → Y) (ex' : Bool → X') (ey' :
3729   Bool → Y') :
3730   (X × Y) × (X' × Y') → ℝ≥0 :=
3731   fun u => if u ∈ pent.image (emb ex ey ex
3732     ' ey') then 2-1 else 0
3733
3734 theorem pentE_not_mix (hx : Function.
3735   Injective ex) (hy : Function.Injective
3736   ey)
3737   (hx' : Function.Injective ex') (hy' :
3738   Function.Injective ey')
3739
3740

```

```

3741 {n : ℕ} (w : Fin n → ℝ≥0∞) (hw : ∑ i,
3742   w i = 1) (z : Fin n → (X × Y) × (X'
3743     × Y')) → ℝ≥0∞
3744 (hz : ∀ i, z i ∈ tD X Y X' Y') (h01 :
3745   ∀ i u, z i u = 0 ∨ z i u = 1) :
3746   pentE ex ey ex' ey' ≠ ∑ i, w i • z i
3747
3748 theorem pentE_mem (hx : Function.Injective
3749   ex) (hy : Function.Injective ey)
3750   (hx' : Function.Injective ex') (hy' :
3751     Function.Injective ey') :
3752     pentE ex ey ex' ey' ∈ tD X Y X' Y'
3753
3754 noncomputable def idBool : Bool × Bool → ℝ
3755   ≥0∞ := fun ab => if ab.1 = ab.2 then 1
3756   else 0
3757
3758 noncomputable def det (f : X → Option Y) :
3759   X × Y → ℝ≥0∞
3760
3761 noncomputable def rowDist (t : X × Y → ℝ≥0
3762   ∞) (a : X) : Option Y → ℝ≥0∞
3763   | none => 1 - ∑ b, t (a, b)
3764   | some b => t (a, b)
3765
3766 noncomputable def tens (x : X → ℝ≥0∞) (y :
3767   Y → ℝ≥0∞) : X × Y → ℝ≥0∞ := fun ab =>
3768   x ab.1 * y ab.2
3769
3770 abbrev W := Bool × Bool
3771
3772 def pent : Finset (W × W) :=
3773   {((true, true), (true, true)), ((true,
3774     false), (false, false)),
3775     ((false, true), (true, false)), ((false
3776       , true), (false, true)),
3777     ((false, false), (false, false))}
3778
3779 noncomputable def pentElt : W × W → ℝ≥0∞
3780   := fun pq => if pq ∈ pent then 2-1 else
3781     0
3782
3783 def covers (f g : Bool → Option Bool) (pq
3784   : W × W) : Prop :=
3785   f pq.1.1 = some pq.1.2 ∧ g pq.2.1 = some
3786     pq.2.2
3787
3788 def compat (p q : W) : Prop := p = q ∨ p.1
3789   ≠ q.1
3790
3791 def compat2 (s t : W × W) : Prop := compat
3792   s.1 t.1 ∧ compat s.2 t.2
3793
3794
3795

```

```

def through (p q : W) : Bool → Option Bool
3796   := fun b => if b = p.1 then some p.2
3797   else some q.2
3798
3799
theorem pent_clique_le_two : ∀ B ∈ pent.
3800   powerset,
3801   (∀ s ∈ B, ∀ t ∈ B, compat2 s t) → B.
3802     card ≤ 2
3803
3804
theorem pent_indep_le_two : ∀ B ∈ pent.
3805   powerset,
3806   (∀ s ∈ B, ∀ t ∈ B, compat2 s t → s = t
3807     ) → B.card ≤ 2
3808
3809
structure FFirst where
3810   μ : Bool → ℝ≥0∞
3811   κ : Bool → Bool → Bool → ℝ≥0∞
3812   ρ : Bool → Bool → Bool → Bool → ℝ≥0∞
3813   hμ : μ true + μ false ≤ 1
3814   hκ : ∀ a b, κ a b true + κ a b false ≤ 1
3815   hρ : ∀ a b c d, ρ a b c d ≤ 1
3816
3817
structure GFirst where
3818   μ : Bool → ℝ≥0∞
3819   κ : Bool → Bool → Bool → ℝ≥0∞
3820   ρ : Bool → Bool → Bool → Bool → ℝ≥0∞
3821   hμ : μ true + μ false ≤ 1
3822   hκ : ∀ c d, κ c d true + κ c d false ≤ 1
3823   hρ : ∀ c d a b, ρ c d a b ≤ 1
3824
3825
noncomputable def FFirst.den (S : FFirst)
3826   : W × W → ℝ≥0∞ := fun pq =>
3827   S.μ pq.1.1 * S.κ pq.1.1 pq.1.2 pq.2.1 *
3828   S.ρ pq.1.1 pq.1.2 pq.2.1 pq.2.2
3829
3830
noncomputable def GFirst.den (S : GFirst)
3831   : W × W → ℝ≥0∞ := fun pq =>
3832   S.μ pq.2.1 * S.κ pq.2.1 pq.2.2 pq.1.1 *
3833   S.ρ pq.2.1 pq.2.2 pq.1.1 pq.1.2
3834
3835
noncomputable def copyStrategy : FFirst
3836   where
3837     μ a
3838
3839
def idleG : GFirst where
3840   μ _
3841
3842
def idleF : FFirst where
3843   μ _
3844
3845
def tensDual : Set (W × W → ℝ≥0∞) :=
3846   {t | ∀ y ∈ lin Bool Bool, ∀ z ∈ lin Bool
3847     Bool, ∑ p, ∑ q, y p * t (p, q) * z q
3848     ≤ 1}
3849
3850

```

```

3851
3852 def sequential : Set (W × W → ℝ≥0) :=
3853   {t | ∃ (α β : ℝ≥0) (F : FFirst) (G :
3854     GFirst), α + β ≤ 1 ∧ t = α • F.den +
3855     β • G.den}
3856
3857 theorem seq_pent_le_two {t : W × W → ℝ≥0}
3858   (ht : t ∈ sequential) : ∑ pq ∈ pent, t
3859   pq ≤ 2
3860
3861 theorem seq_pent_tight : ∃ t ∈ sequential,
3862   ∑ pq ∈ pent, t pq = 2
3863
3864 theorem pentElt_pent_sum : ∑ pq ∈ pent,
3865   pentElt pq = 5 * 2-1
3866
3867 theorem tensDual_pent_le {t : W × W → ℝ≥0}
3868   (ht : t ∈ tensDual) :
3869   2 * ∑ pq ∈ pent, t pq ≤ 5
3870
3871 theorem sharp_bang_mixed_zero {z : ℕ × ℕ →
3872   ℝ≥0} (hz : z ∈ bang) (h01 : ∀ m, z m
3873   = 0 ∨ z m = 1)
3874   {i j : ℕ} (hi : 1 ≤ i) (hj : 1 ≤ j) :
3875   z (i, j) = 0
3876
3877 def orthI {E : Type*} (S : Set (E → ℝ≥0))
3878   : Set (E → ℝ≥0) := {y | ∀ x ∈ S, ∑' e
3879   , x e * y e ≤ 1}
3880
3881 def gens : Set (ℕ × ℕ → ℝ≥0) := {u | ∃ x
3882   ∈ flat Bool, u = prom x}
3883
3884 noncomputable def half : Bool → ℝ≥0 :=
3885   fun _ => 2-1
3886
3887 noncomputable def prom (x : Bool → ℝ≥0) :
3888   ℕ × ℕ → ℝ≥0 := fun m => x true ^ m.1
3889   * x false ^ m.2
3890
3891 def bang : Set (ℕ × ℕ → ℝ≥0) := orthI (
3892   orthI gens)
3893
3894 theorem prom_half_not_mix : prom half ∈
3895   bang ∧
3896   prom half ∉ Mix {z | z ∈ bang ∧ ∀ m, z
3897   m = 0 ∨ z m = 1}
3898
3899 def excl (s t : (X × Y) × (X' × Y')) :
3900   Prop := s ≠ t ∧ cp2 s t
3901
3902 theorem tD_eq_QSTAB : tD X Y X' Y' = QSTAB
3903   (excl (X := X) (Y := Y) (X' := X') (Y'
3904   := Y'))
3905
3906 theorem recipe_eq_STAB :
3907   Mix {z | z ∈ tD X Y X' Y' ∧ ∀ u, z u =
3908   0 ∨ z u = 1}
3909   = STAB (excl (X := X) (Y := Y) (X'
3910   := X') (Y' := Y'))
3911
3912 def cyc {n : ℕ} (i j : ZMod n) : Prop := j
3913   = i + 1 ∨ i = j + 1
3914
3915 def IsCliqueG (C : Finset V) : Prop := ∀ u
3916   ∈ C, ∀ v ∈ C, u ≠ v → adj u v
3917
3918 def IsStableG (I : Finset V) : Prop := ∀ u
3919   ∈ I, ∀ v ∈ I, ¬ adj u v
3920
3921 noncomputable def ind (I : Finset V) : V →
3922   ℝ≥0 := fun v => if v ∈ I then 1 else
3923   0
3924
3925 structure Antihole (adj : V → V → Prop) (n
3926   : ℕ) where
3927   h : ZMod n → V
3928   inj : Function.Injective h
3929   adj_iff : ∀ i j, i ≠ j → (adj (h i) (h j)
3930   ) ↔ ¬ cyc i j)
3931
3932 def QSTAB : Set (V → ℝ≥0) := {x | ∀ C,
3933   IsCliqueG adj C → ∑ v ∈ C, x v ≤ 1}
3934
3935 def STAB : Set (V → ℝ≥0) := Mix {x | ∃ I,
3936   IsStableG adj I ∧ x = ind I}
3937
3938 structure Hole (adj : V → V → Prop) (n : ℕ
3939   ) where
3940   h : ZMod n → V
3941   inj : Function.Injective h
3942   adj_iff : ∀ i j, i ≠ j → (adj (h i) (h j)
3943   ) ↔ cyc i j)
3944
3945 theorem berge_of_stab_eq_qstab (heq : STAB
3946   adj = QSTAB adj) (k : ℕ) (hk : 2 ≤ k)
3947   [NeZero (2 * k + 1)] : IsEmpty (Hole
3948   adj (2 * k + 1)) ∧ IsEmpty (
3949   Antihole adj (2 * k + 1))
3950
3951 theorem berge_of_recipe_eq_pcs
3952   (heq : Mix {z | z ∈ tD X Y X' Y' ∧ ∀ u
3953   , z u = 0 ∨ z u = 1} = tD X Y X' Y'
3954   )
3955   (k : ℕ) (hk : 2 ≤ k) [NeZero (2 * k +
3956   1)] :
3957   IsEmpty (Hole (excl (X := X) (Y := Y)
3958   (X' := X') (Y' := Y')) (2 * k + 1))
3959   ∧
3960

```

```

3961     IsEmpty (Antihole (excl (X := X) (Y
3962       := Y) (X' := X') (Y' := Y')) (2 *
3963       k + 1))
3964

```

B.11 Multiplicative-Additive Formulas

```

3967 inductive Fm where
3968   | base (n : ℕ)
3969   | neg (A : Fm)
3970   | tens (A B : Fm)
3971   | wth (A B : Fm)
3972
3973 def web : Fm → Type
3974   | base n => Fin n
3975   | neg A => web A
3976   | tens A B => web A × web B
3977   | wth A B => web A ⊕ web B
3978
3979 def P : (A : Fm) → Set (web A → ℝ≥0∞)
3980   | base n => flat (Fin n)
3981   | neg A => orth (P A)
3982   | tens A B => orth (orth {z | ∃ x ∈ P A,
3983     ∃ y ∈ P B, z = tvec x y})
3984   | wth A B => {z | (fun a => z (Sum.inl a
3985     )) ∈ P A ∧ (fun b => z (Sum.inr b)) ∈
3986     P B}
3987
3988 def coh : (A : Fm) → web A → web A → Prop
3989   | base _ => fun x y => x = y
3990   | neg A => fun x y => x = y ∨ ¬ coh A x
3991     y
3992   | tens A B => fun s t => coh A s.1 t.1 ∧
3993     coh B s.2 t.2
3994   | wth A B => fun s t => match s, t with
3995     | Sum.inl a, Sum.inl a' => coh A a a'
3996     | Sum.inr b, Sum.inr b' => coh B b b'
3997     | _, _ => True
3998
3999 theorem isPCS_P : ∀ A : Fm, IsPCS (P A)
4000
4001 def par (A B : Fm) : Fm := neg (tens (neg
4002   A) (neg B))
4003 def plus (A B : Fm) : Fm := neg (wth (neg
4004   A) (neg B))
4005 def lolli (A B : Fm) : Fm := neg (tens A (
4006   neg B))
4007
4008 noncomputable def tvec {X Y : Type} (x : X
4009   → ℝ≥0∞) (y : Y → ℝ≥0∞) : X × Y → ℝ≥0∞
4010   :=
4011   fun ab => x ab.1 * y ab.2
4012
4013 noncomputable def svec {X Y : Type} (x : X
4014   → ℝ≥0∞) (y : Y → ℝ≥0∞) : X ⊕ Y → ℝ≥0∞
4015

```

```

4016   | Sum.inl a => x a
4017   | Sum.inr b => y b
4018
4019 noncomputable def pr (s t : E) : E → ℝ≥0∞
4020   := Pi.single s 1 + Pi.single t 1
4021
4022 theorem invariants : ∀ A : Fm,
4023   (∀ s : web A, Pi.single s 1 ∈ P A ∧ ∀
4024     x ∈ P A, x s ≤ 1) ∧
4025   (∀ s t : web A, s ≠ t → coh A s t → pr
4026     s t ∈ P A) ∧
4027   (∀ s t : web A, s ≠ t → ¬ coh A s t →
4028     ∀ x ∈ P A, x s + x t ≤ 1)
4029
4030 theorem single_mem_P (A : Fm) (s : web A)
4031   : Pi.single s 1 ∈ P A
4032
4033 theorem pair_mem_P (A : Fm) {s t : web A}
4034   (hst : s ≠ t) (h : coh A s t) : pr s t
4035   ∈ P A
4036
4037 theorem sharp_isClique (A : Fm) {z : web A
4038   → ℝ≥0∞} (hz : z ∈ P A)
4039   (h01 : ∀ s, z s = 0 ∨ z s = 1) : ∀ s t
4040     , z s = 1 → z t = 1 → coh A s t
4041
4042 theorem recipe_subset_P (A : Fm) :
4043   Mix {z | z ∈ P A ∧ ∀ s, z s = 0 ∨ z s
4044     = 1} ⊆ P A
4045
4046 theorem recipe_sharp_iff (A : Fm) {x : web
4047   A → ℝ≥0∞} (hx : ∀ s, x s = 0 ∨ x s =
4048     1) :
4049   x ∈ Mix {z | z ∈ P A ∧ ∀ s, z s = 0 ∨
4050     z s = 1} ↔ x ∈ P A
4051
4052 theorem Mix_subset_orth {S T : Set (E → ℝ≥
4053   0∞)} (h : S ⊆ orth T) : Mix S ⊆ orth T
4054
4055 theorem P_lolli_base (n m : ℕ) : P (lolli
4056   (base n) (base m)) = lin (Fin n) (Fin m
4057   )
4058
4059 def second (n m n' m' : ℕ) : Fm := neg (
4060   tens (lolli (base n) (base m)) (lolli (
4061     base n') (base m'))))
4062
4063 theorem P_second (n m n' m' : ℕ) :
4064   P (second n m n' m') = tD (Fin n) (Fin
4065     m) (Fin n') (Fin m')
4066
4067 def boolFin {k : ℕ} (hk : 2 ≤ k) : Bool →
4068   Fin k
4069

```

```

4071 theorem recipe_first_order (n m :  $\mathbb{N}$ ) :
4072   P (lolli (base n) (base m)) =
4073     ConstructiveProb.Mix (Set.range
4074       (det : (Fin n  $\rightarrow$  Option (Fin m))  $\rightarrow$ 
4075         Fin n  $\times$  Fin m  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ))
4076
4077 theorem recipe_second_order {n m n' m' :  $\mathbb{N}$ 
4078   } (hn : 0 < n) (hn' : 0 < n') :
4079   ConstructiveProb.Mix {z | z  $\in$  P (
4080     second n m n' m')  $\wedge \forall u, z u = 0 \vee$ 
4081     z u = 1}
4082   = sequentialG (Fin n) (Fin m) (Fin n
4083     ') (Fin m')
4084
4085 theorem recipe_second_order_ssubset {n m n
4086   ' m' :  $\mathbb{N}$ } (hn : 2  $\leq$  n) (hm : 2  $\leq$  m) (hn
4087   ' : 2  $\leq$  n')
4088   (hm' : 2  $\leq$  m') :
4089   ConstructiveProb.Mix {z | z  $\in$  P (
4090     second n m n' m')  $\wedge \forall u, z u = 0 \vee$ 
4091     z u = 1}
4092    $\subset$  P (second n m n' m')
4093
4094 def  $\varphi$  : W  $\times$  W  $\rightarrow$  web S2 := emb (boolFin
4095   le_rfl) (boolFin le_rfl) (boolFin
4096   le_rfl) (boolFin le_rfl)
4097
4098 abbrev S2 : Fm := second 2 2 2 2
4099
4100 abbrev TT : Fm := neg (tens S2 S2)
4101
4102 noncomputable def shannonF : Finset (web (
4103   tens S2 S2)) := shannon.image fun u =>
4104   ( $\varphi$  u.1,  $\varphi$  u.2)
4105
4106 theorem shannonF_clique :  $\forall u \in$  shannonF,
4107    $\forall v \in$  shannonF, coh TT u v
4108
4109 noncomputable def pentS2 : web S2  $\rightarrow \mathbb{R}_{\geq 0\infty}$ 
4110   :=
4111   pentE (boolFin le_rfl) (boolFin le_rfl)
4112   (boolFin le_rfl) (boolFin le_rfl)
4113
4114 theorem shannonF_not_mem : (fun u => if u
4115    $\in$  shannonF then (1 :  $\mathbb{R}_{\geq 0\infty}$ ) else 0)  $\notin$  P
4116   TT
4117
4118
4119 B.12 Quantum Event Logic
4120
4121 def vec : Fin 18  $\rightarrow$  Fin 4  $\rightarrow \mathbb{Z}$  := ![
4122   [-1, 1, 1, 1], ![0, 0, 0, 1], ![0, 0,
4123     1, 0], ![0, 0, 1, 1], ![0, 1, -1, 0],
4124     ![0, 1, 0, -1],
4125
4126   ![0, 1, 0, 0], ![1, -1, -1, 1], ![1, -1,
4127     0, 0], ![1, -1, 1, -1], ![1, 0, -1,
4128     0], ![1, 0, 0, -1],
4129     ![1, 0, 0, 1], ![1, 0, 1, 0], ![1, 1,
4130     -1, 1], ![1, 1, 0, 0], ![1, 1, 1,
4131     -1], ![1, 1, 1, 1]]
4132
4133 def bas : Fin 9  $\rightarrow$  Fin 4  $\rightarrow$  Fin 18 := ![
4134   [1, 2, 15, 8], [1, 6, 13, 10], [9, 7,
4135     15, 3], [9, 17, 10, 5], [2, 6, 12,
4136     11],
4137     [7, 17, 11, 4], [14, 16, 8, 3], [14,
4138     0, 13, 5], [16, 0, 12, 4]]
4139
4140 def dot (u v : Fin 4  $\rightarrow \mathbb{Z}$ ) :  $\mathbb{Z}$  :=  $\sum k, u k$ 
4141   * v k
4142
4143 theorem basis_orth :  $\forall b : \text{Fin } 9, \forall i j :$ 
4144   Fin 4,  $i \neq j \rightarrow \text{dot (vec (bas } b \ i)) (vec$ 
4145     (bas b j)) = 0
4146
4147 theorem occ_two :  $\forall v : \text{Fin } 18,$ 
4148   (univ.filter (fun p : Fin 9  $\times$  Fin 4 =>
4149     bas p.1 p.2 = v)).card = 2
4150
4151 theorem no_ks :  $\neg \exists f : \text{Fin } 18 \rightarrow \text{Bool}, \forall b$ 
4152   , (univ.filter (fun i => f (bas b i))).
4153   card = 1
4154
4155 def certain : Set (Fin 18  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) :=
4156   {s | IsSharpVec s  $\wedge \forall b, \sum i, s$  (bas b i
4157     ) = 1}
4158
4159 theorem no_certain_state : certain =  $\emptyset$ 
4160
4161 theorem Mix_certain_empty : Mix certain =
4162    $\emptyset$ 
4163
4164 theorem parseval ( $\psi : \text{Fin } 4 \rightarrow \mathbb{R}$ ) (b : Fin
4165   9) :
4166    $\sum i, (\sum k, \psi k * \text{vec (bas } b \ i) k) ^ 2$ 
4167   /  $\sum k, ((\text{vec (bas } b \ i) k : \mathbb{R})) ^ 2$ 
4168   =  $\sum k, \psi k ^ 2$ 
4169
4170 def states : Set (Fin 18  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) := {x |  $\forall$ 
4171   b,  $\sum i, x$  (bas b i) = 1}
4172
4173 noncomputable def born ( $\psi : \text{Fin } 4 \rightarrow \mathbb{R}$ ) :
4174   Fin 18  $\rightarrow \mathbb{R}_{\geq 0\infty}$  := fun v =>
4175   ENNReal.ofReal (( $\sum k, \psi k * \text{vec } v \ k$ ) ^ 2
4176     /  $\sum k, ((\text{vec } v \ k : \mathbb{R})) ^ 2$ )
4177
4178 theorem born_mem { $\psi : \text{Fin } 4 \rightarrow \mathbb{R}$ } (h $\psi$  :  $\sum k$ 
4179   ,  $\psi k ^ 2 = 1$ ) : born  $\psi \in$  states
4180

```

```

4181
4182 theorem born_not_mix { $\psi$  : Fin 4  $\rightarrow$   $\mathbb{R}$ } (h $\psi$  :
4183    $\sum k, \psi k^2 = 1$ ) :
4184   born  $\psi \in \text{states} \wedge \text{born } \psi \notin \text{Mix certain}$ 
4185

```

B.13 Finite Frames without Choice

```

4186
4187 def below (a :  $\alpha$ ) : Finset  $\alpha$  := Finset.
4188   univ.filter ( $\cdot \leq a$ )
4189
4190
4191 def ind (j x :  $\alpha$ ) :  $\mathbb{N}$  := if j  $\leq$  x then 1
4192   else 0
4193
4194 def JoinPrime (j :  $\alpha$ ) : Prop :=  $\forall x y : \alpha,$ 
4195   j  $\leq$  x  $\sqcup$  y  $\rightarrow$  j  $\leq$  x  $\vee$  j  $\leq$  y
4196
4197 theorem exists_joinPrime_aux :  $\forall (n : \mathbb{N})$  (
4198   a b :  $\alpha$ ), (below a).card = n  $\rightarrow$   $\neg a \leq b$ 
4199    $\rightarrow$ 
4200    $\exists j, j \leq a \wedge \neg j \leq b \wedge \text{JoinPrime } j$ 
4201
4202 theorem exists_sharp_separating_fin [
4203   BoundedOrder  $\alpha$ ] {a b :  $\alpha$ } (hab :  $\neg a \leq$ 
4204   b) :
4205    $\exists v : \alpha \rightarrow \mathbb{N}, (\forall x, v x = 0 \vee v x = 1)$ 
4206      $\wedge \text{Monotone } v \wedge v \perp = 0 \wedge v \top = 1 \wedge$ 
4207      $(\forall x y, v x + v y = v (x \sqcup y) + v (x$ 
4208        $\sqcap y)) \wedge v a = 1 \wedge v b = 0$ 
4209
4210 theorem em_of_sharp_complement_fin [
4211   BoundedOrder  $\alpha$ ] (a c :  $\alpha$ )
4212   (h :  $\forall v : \alpha \rightarrow \mathbb{N}, (\forall x, v x = 0 \vee v x$ 
4213      $= 1) \rightarrow \text{Monotone } v \rightarrow v \perp = 0 \rightarrow v \top =$ 
4214     1  $\rightarrow$ 
4215      $(\forall x y, v x + v y = v (x \sqcup y) + v (x$ 
4216        $\sqcap y)) \rightarrow v a + v c = 1$ ) :
4217   a  $\sqcup$  c =  $\top$ 
4218

```

B.14 Choice-Free Finite Results

```

4219
4220
4221 def wt (b : Bool) (m : Nat) : Nat := if b
4222   then m else 0
4223
4224 def mem : Op  $\rightarrow$  Bool  $\rightarrow$  Bool
4225   | bot, _ => false
4226   | halts, b => b
4227   | top, _ => true
4228
4229 def le (o o' : Op) : Bool := mem o true  $\leq$ 
4230   mem o' true  $\wedge$  mem o false  $\leq$  mem o'
4231   false
4232
4233 def inf : Op  $\rightarrow$  Op  $\rightarrow$  Op
4234   | bot, _ => bot
4235

```

```

4236   | _, bot => bot
4237   | halts, _ => halts
4238   | top, o => o
4239
4240 def val (wT wF : Nat) (o : Op) : Nat := wt
4241   (mem o true) wT + wt (mem o false) wF
4242
4243 def inClosure (x : Bool) : Bool := [bot,
4244   halts, top].all fun o => !mem o x ||
4245   mem o true
4246
4247 def frontier (x : Bool) : Bool :=
4248   inClosure x  $\wedge$  !mem halts x
4249
4250 inductive Op | bot | halts | top
4251   deriving DecidableEq
4252
4253 def neg : Op  $\rightarrow$  Op
4254   | bot => top
4255   | halts => bot
4256   | top => bot
4257
4258 theorem neg_spec :  $\forall o o' : \text{Op}, (\text{inf } o' o$ 
4259   = bot)  $\leftrightarrow$  le o' (neg o) = true
4260
4261 theorem neg_halts : neg halts = bot
4262
4263 theorem frontier_iff :  $\forall x : \text{Bool},$ 
4264   frontier x = true  $\leftrightarrow$  x = false
4265
4266 theorem slack_eq (wT wF : Nat) :
4267   val wT wF top = val wT wF halts + val
4268   wT wF (neg halts) + wt (frontier
4269   false) wF
4270
4271 def fs : (n : Nat)  $\rightarrow$  (Fin n  $\rightarrow$  Nat)  $\rightarrow$  Nat
4272   | 0, _ => 0
4273   | n + 1, f => f <0, by omega> + fs n (
4274     fun i => f <i.val + 1, by omega>)
4275
4276 def allFin : (n : Nat)  $\rightarrow$  (Fin n  $\rightarrow$  Bool)  $\rightarrow$ 
4277   Bool
4278   | 0, _ => true
4279   | n + 1, p => p <0, by omega>  $\wedge$  allFin
4280     n (fun i => p <i.val + 1, by omega>)
4281
4282 def ext (ops : List (Fin n  $\rightarrow$  Bool)) (U :
4283   Fin n  $\rightarrow$  Bool) (x : Fin n) : Bool :=
4284   ops.any fun 0 => 0 x  $\wedge$  allFin n (fun y
4285     => !(0 y  $\wedge$  U y))
4286
4287 def bd (ops : List (Fin n  $\rightarrow$  Bool)) (U :
4288   Fin n  $\rightarrow$  Bool) (x : Fin n) : Bool :=
4289   !U x  $\wedge$  !ext ops U x
4290

```

```

4291
4292 theorem sub_ext {ops : List (Fin n → Bool)
4293   } {U : Fin n → Bool} {O : Fin n → Bool}
4294   (h0 : 0 ∈ ops) (hd : ∀ y, (! (0 y && U
4295     y)) = true) {x : Fin n} (hx : 0 x =
4296     true) :
4297     ext ops U x = true
4298
4299 theorem slack_eq_boundary_fin (ops : List
4300   (Fin n → Bool)) (U : Fin n → Bool) (μ :
4301   Fin n → Nat) :
4302   fs n μ = fs n (fun x => wt (U x) (μ x)
4303     ) + fs n (fun x => wt (ext ops U x)
4304     (μ x)) +
4305   fs n (fun x => wt (bd ops U x) (μ x)
4306     )
4307
4308 def isPre : List Bool → List Bool → Bool
4309 | [], _ => true
4310 | _ :: _, [] => false
4311 | a :: u, b :: v => a == b && isPre u v
4312
4313 def Incomp (u v : List Bool) : Prop :=
4314   isPre u v = false ∧ isPre v u = false
4315
4316 def PF : List (List Bool) → Prop
4317 | [] => True
4318 | w :: S => (∀ v ∈ S, Incomp w v) ∧ PF S
4319
4320 def tails (b : Bool) : List (List Bool) →
4321   List (List Bool)
4322 | [] => []
4323 | [] :: S => tails b S
4324 | (a :: t) :: S => if a = b then t ::
4325   tails b S else tails b S
4326
4327 def ksum (L : Nat) : List (List Bool) →
4328   Nat
4329 | [] => 0
4330 | w :: S => 2 ^ (L - w.length) + ksum L
4331   S
4332
4333 theorem kraft : ∀ (L : Nat) (S : List (
4334   List Bool)), PF S → (∀ w ∈ S, w.length
4335   ≤ L) →
4336   ksum L S ≤ 2 ^ L
4337
4338 def Sat {n : Nat} (r : Row n) (w : Fin n →
4339   Int) : Prop :=
4340   0 ≤ dot n r.c w ∧ (r.s = true → 0 < dot
4341     n r.c w)
4342
4343 def comb {n : Nat} (p q : Int) (r1 r2 :
4344   Row n) : Row n :=
4345
4346   (fun i => p * r1.c i + q * r2.c i, (
4347     decide (0 < p) && r1.s) || (decide (0
4348     < q) && r2.s))
4349
4350 def zeroS (n : Nat) : Row n := (fun _ =>
4351   0, true)
4352
4353 def lastc (r : Row (n + 1)) : Int := r.c (
4354   Fin.last n)
4355
4356 def proj (r : Row (n + 1)) : Row n := (fun
4357   i => r.c i.castSucc, r.s)
4358
4359 def posR (rows : List (Row (n + 1))) :=
4360   rows.filter fun r => decide (0 < lastc
4361     r)
4362
4363 def negR (rows : List (Row (n + 1))) :=
4364   rows.filter fun r => decide (lastc r <
4365     0)
4366
4367 def zerR (rows : List (Row (n + 1))) :=
4368   rows.filter fun r => decide (lastc r =
4369     0)
4370
4371 def pair (p q : Row (n + 1)) : Row n :=
4372   proj (comb (-lastc q) (lastc p) p q)
4373
4374 def step (rows : List (Row (n + 1))) :
4375   List (Row n) :=
4376   (zerR rows).map proj ++ (posR rows).
4377   flatMap fun p => (negR rows).map fun
4378   q => pair p q
4379
4380 def extd (D W : Int) (w' : Fin n → Int) :
4381   Fin (n + 1) → Int :=
4382   fun i => if h : i.val < n then D * w' {i
4383     .val, h} else W
4384
4385 def hrow (μ : Fin N → Int) (ν α : Int) (β
4386   : Fin m → Int) : Fin (N + 1) → Int :=
4387   fun i => if h : i.val < N then μ {i.val,
4388     h} + α + d * dot m β (S {i.val, h})
4389   else ν - α - dot m β y
4390
4391 def hullRows : List (Row (N + 1)) :=
4392   (List.finRange N).map (fun j => (hrow S
4393     y d (fun k => if k = j then 1 else 0)
4394     0 0 (fun _ => 0), false)) ++
4395   [(hrow S y d (fun _ => 0) 1 0 (fun _ =>
4396     0), true),
4397     (hrow S y d (fun _ => 0) 0 1 (fun _ =>
4398     0), false),
4399     (hrow S y d (fun _ => 0) 0 (-1) (fun _
4400     => 0), false)] ++
4401   (List.finRange m).flatMap (fun e =>

```

```

4401   [⟨hrow S y d (fun _ => 0) 0 0 (fun k
4402     => if k = e then 1 else 0), false⟩,
4403     ⟨hrow S y d (fun _ => 0) 0 0 (fun k
4404       => if k = e then -1 else 0), false
4405     ⟩)]
4406
4407   structure Row (n : Nat) where
4408     c : Fin n → Int
4409     s : Bool
4410
4411   inductive Der {n : Nat} (rows : List (Row
4412     n)) : Row n → Prop
4413   | base {r : Row n} : r ∈ rows → Der rows
4414     r
4415   | comb {r1 r2 : Row n} (p q : Int) : 0 ≤
4416     p → 0 ≤ q → Der rows r1 → Der rows r
4417     2 →
4418     Der rows (comb p q r1 r2)
4419
4420   theorem fm : ∀ (n : Nat) (rows : List (Row
4421     n)), (∃ w, ∀ r ∈ rows, Sat r w) ∨ Der
4422     rows (zeroS n)
4423
4424   theorem complete (hd : 0 < d)
4425     (hlaw : ∀ (c : Fin m → Int) (k : Int),
4426       (∀ j, k ≤ dot m c (S j)) → k * d ≤
4427       dot m c y) :
4428     ∃ (lam : Fin N → Int) (t : Int), 0 < t
4429       ∧ (∀ j, 0 ≤ lam j) ∧
4430       dot N (fun _ => 1) lam = t ∧ ∀ e, d
4431         * dot N lam (fun j => S j e) = t
4432         * y e
4433
4434   theorem sound (hd : 0 < d) (lam : Fin N →
4435     Int) (t : Int) (ht : 0 < t) (hl : ∀ j,
4436     0 ≤ lam j)
4437     (hsum : dot N (fun _ => 1) lam = t) (
4438       hx : ∀ e, d * dot N lam (fun j => S
4439         j e) = t * y e)
4440     (c : Fin m → Int) (k : Int) (hc : ∀ j,
4441       k ≤ dot m c (S j)) : k * d ≤ dot m
4442       c y
4443
4444   def union {A B : Nat → Nat → Bool} (a :
4445     EOD A) (b : EOD B) : EOD (fun m m' => A
4446       m m' || B m m') where
4447     N
4448
4449   def inter {A B : Nat → Nat → Bool} (a :
4450     EOD A) (b : EOD B) : EOD (fun m m' => A
4451       m m' && B m m') where
4452     N

```

```

def compl {A : Nat → Nat → Bool} (a : EOD
  A) : EOD (fun m m' => !A m m') where
  N
def val (b : Bool) : Nat := if b then 1
  else 0
structure EC (S : Nat → Bool) where
  N : Nat
  v : Bool
  spec : ∀ m, N ≤ m → S m = v
def rect {S T : Nat → Bool} (s : EC S) (t
  : EC T) : EOD (fun m m' => S m && T m')
  where
  N
def tri : EOD (fun m m' => decide (m < m'))
  ) where
  N
structure EOD (A : Nat → Nat → Bool) where
  N : Nat
  lt : Bool
  eq : Bool
  gt : Bool
  spec : ∀ m m', N ≤ m → N ≤ m' →
    A m m' = if m < m' then lt else if m =
      m' then eq else gt
def L {A : Nat → Nat → Bool} (w : EOD A) :
  Bool := w.lt
/-- The iterated limit with `m → ∞` first.
  -/
def R {A : Nat → Nat → Bool} (w : EOD A) :
  Bool := w.gt
theorem L_modular {A B} (a : EOD A) (b :
  EOD B) :
  val (L a) + val (L b) = val (L (union
    a b)) + val (L (inter a b))
theorem rect_L {S T} (s : EC S) (t : EC T)
  : L (rect s t) = (s.v && t.v)
theorem rect_R {S T} (s : EC S) (t : EC T)
  : R (rect s t) = (s.v && t.v)
theorem products_differ : L tri ≠ R tri

```

B.15 Valued Frames and the Valuation Monoid

```

noncomputable def ValF : Frmop ⇒ Type _

```

```

4511 abbrev SpVal := ValF.Elements
4512
4513 inductive Gen : Multiset P → Multiset P →
4514   Prop
4515   | modular (a b : P) : Gen ({a} + {b}) ({
4516     a ⊔ b} + {a ⊓ b})
4517   | bot : Gen {⊥} 0
4518
4519 def con : AddCon (Multiset P) := addConGen
4520   (Gen P)
4521
4522 abbrev V := (con P).Quotient
4523
4524 def ι (a : P) : V P := (con P).mk' {a}
4525
4526 structure Modular (f : P → M) : Prop where
4527   bot : f ⊥ = 0
4528   modular : ∀ a b, f a + f b = f (a ⊔ b) +
4529     f (a ⊓ b)
4530
4531 def lift (f : P → M) (hf : Modular f) : V
4532   P →+ M
4533
4534 theorem lift_ι (f : P → M) (hf : Modular f
4535   ) (a : P) : lift f hf (ι a) = f a
4536
4537 theorem hom_ext {h h' : V P →+ M} (H : ∀ a
4538   , h (ι a) = h' (ι a)) : h = h'
4539
4540 theorem modular_of_hom (h : V P →+ M) :
4541   Modular (fun a => h (ι a))
4542
4543 noncomputable def lvaluation_equiv {P :
4544   Type*} [Lattice P] [BoundedOrder P] :
4545   LValuation P ≈
4546     {h : V P →+ ℝ≥0 // h (ι ⊤) = 1 ∧
4547       Monotone fun a => h (ι a)}
4548
4549 theorem exists_flag (x : V P) : ∃ l : List
4550   P, l.Pairwise (· ≤ ·) ∧ x = (l.map ι).
4551   sum
4552
4553 theorem M3_flag_not_unique : ι M3.a + ι M3
4554   .o = ι M3.c + ι M3.o
4555
4556 def m3sep : M3 → ℝ≥0
4557   | .z => 0
4558   | .c => 0
4559   | _ => 1
4560
4561 theorem m3sep_modular : Modular m3sep
4562
4563 theorem M3_not_cancel : ι M3.a + ι M3.b =
4564   ι M3.a + ι M3.c ∧ ι M3.b ≠ ι M3.c
4565

```

B.16 A Language for the Second-Order Type

```

4566 inductive PEnd
4567   | halt
4568   | diverge
4569   | choose (k : Nat) (w : Fin k → ℝ≥0) (
4570     hw : ∑ i, w i ≤ 1) (p : Fin k → PEnd)
4571   | call (c : C) (k : D → PEnd)
4572
4573 inductive POne (C D : Type)
4574   | diverge
4575   | choose (k : Nat) (w : Fin k → ℝ≥0) (
4576     hw : ∑ i, w i ≤ 1) (p : Fin k → POne
4577     C D)
4578   | call (c : C) (k : D → PEnd)
4579
4580 inductive P0 (X Y X' Y' : Type)
4581   | diverge
4582   | choose (k : Nat) (w : Fin k → ℝ≥0) (
4583     hw : ∑ i, w i ≤ 1) (p : Fin k → P0 X
4584     Y X' Y')
4585   | callF (a : X) (k : Y → POne X' Y')
4586   | callG (c : X') (k : Y' → POne X Y)
4587
4588 noncomputable def PEnd.den : PEnd → ℝ≥0
4589   | .halt => 1
4590   | .diverge => 0
4591   | .choose _ w _ p => ∑ i, w i * (p i).
4592     den
4593
4594 noncomputable def POne.den {C D : Type} [
4595   DecidableEq C] : POne C D → C × D → ℝ≥0
4596   ∞
4597   | .diverge => fun _ => 0
4598   | .choose _ w _ p => fun cd => ∑ i, w i
4599     * (p i).den cd
4600   | .call c k => fun cd => if cd.1 = c
4601     then (k cd.2).den else 0
4602
4603 noncomputable def P0.den : P0 X Y X' Y' →
4604   (X × Y) × (X' × Y') → ℝ≥0
4605   | .diverge => fun _ => 0
4606   | .choose _ w _ p => fun u => ∑ i, w i *
4607     (p i).den u
4608   | .callF a k => fun u => if u.1.1 = a
4609     then (k u.1.2).den u.2 else 0
4610   | .callG c k => fun u => if u.2.1 = c
4611     then (k u.2.2).den u.1 else 0
4612
4613 theorem P0.sound : ∀ p : P0 X Y X' Y', p.
4614   den ∈ sequentialG X Y X' Y'
4615
4616 theorem P0.den_mem_recipe [Nonempty X] [
4617   Nonempty X'] (p : P0 X Y X' Y') :
4618

```

```

4621     p.den ∈ Mix {z | z ∈ tD X Y X' Y' ∧ ∀
4622       u, z u = 0 ∨ z u = 1}
4623
4624   def PEnd.ofBool (b : Bool) : PEnd := if b
4625     then .halt else .diverge
4626
4627   theorem P0.sharp_definable [Nonempty X] [
4628     Nonempty X'] {z : (X × Y) × (X' × Y') →
4629     ℝ≥0∞}
4630     (hz : z ∈ tD X Y X' Y') (h01 : ∀ u, z
4631       u = 0 ∨ z u = 1) : ∃ p : P0 X Y X'
4632       Y', p.den = z
4633
4634
4635

```

B.17 The Boolean Case

```

4636   noncomputable def comp (m : Bool → ℝ≥0∞) :
4637     Option Bool → ℝ≥0∞
4638     | none => 1 - (m true + m false)
4639     | some b => m b
4640
4641   noncomputable def bern (r : ℝ≥0∞) : Bool →
4642     ℝ≥0∞
4643     | true => r
4644     | false => 1 - r
4645
4646   abbrev Choice := (Unit → Option Bool) × (
4647     Bool × Bool → Option Bool) × (Bool ×
4648     Bool × Bool × Bool → Bool)
4649
4650   noncomputable def detCore (σ : Choice) (a
4651     b c d : Bool) : ℝ≥0∞ :=
4652     (if σ.1 () = some a then 1 else 0) * (if
4653       σ.2.1 (a, b) = some c then 1 else 0)
4654     *
4655     (if σ.2.2 (a, b, c, d) = true then 1
4656       else 0)
4657
4658   noncomputable def detF (σ : Choice) :
4659     FFirst where
4660     μ a
4661
4662   noncomputable def detG (σ : Choice) :
4663     GFirst where
4664     μ c
4665
4666   def sharpSeq : Set (W × W → ℝ≥0∞) := {z |
4667     z ∈ sequential ∧ ∀ pq, z pq = 0 ∨ z pq
4668     = 1}
4669
4670   structure Beh where
4671     μ : Bool → ℝ≥0∞
4672     κ : Bool → Bool → Bool → ℝ≥0∞
4673     ρ : Bool → Bool → Bool → Bool → ℝ≥0∞
4674     hκ : ∀ a b, κ a b true + κ a b false ≤ 1
4675

```

```

4676   hρ : ∀ a b c d, ρ a b c d ≤ 1
4677
4678   noncomputable def Beh.val (B : Beh) (a b c
4679     d : Bool) : ℝ≥0∞ := B.μ a * B.κ a b c
4680     * B.ρ a b c d
4681
4682   noncomputable def Beh.mass (B : Beh) : ℝ≥0
4683     ∞ := B.μ true + B.μ false
4684
4685   def seqU : Set (W × W → ℝ≥0∞) :=
4686     {t | ∃ BF BG : Beh, BF.mass + BG.mass ≤
4687       1 ∧
4688       t = fun pq => BF.val pq.1.1 pq.1.2 pq
4689         .2.1 pq.2.2 + BG.val pq.2.1 pq.2.2
4690         pq.1.1 pq.1.2}
4691
4692   theorem FFirst.den_mem_Mix (F : FFirst) :
4693     F.den ∈ Mix sharpSeq
4694
4695   theorem sequential_subset_Mix : sequential
4696     ⊆ Mix sharpSeq
4697
4698   theorem Mix_sequential : Mix sequential =
4699     sequential
4700
4701   theorem recipe_second_order_eq_sequential'
4702     :
4703     Mix {z | z ∈ tensDual ∧ ∀ pq, z pq = 0
4704       ∨ z pq = 1} = sequential
4705
4706   abbrev W2 := (W × W) × (W × W)
4707
4708   noncomputable def tens2 (t t' : W × W → ℝ
4709     ≥0∞) : W2 → ℝ≥0∞ := fun u => t u.1 * t'
4710     u.2
4711
4712   def gensTT : Set (W2 → ℝ≥0∞) := {u | ∃ t ∈
4713     tensDual, ∃ t' ∈ tensDual, u = tens2 t
4714     t'}
4715
4716   def PTT : Set (W2 → ℝ≥0∞) := orth (orth
4717     gensTT)
4718
4719   def PTTdual : Set (W2 → ℝ≥0∞) := orth
4720     gensTT
4721
4722   def cohTT : W2 → W2 → Prop := tensC
4723     secondC secondC
4724
4725   def cohTTdual : W2 → W2 → Prop := dualC
4726     cohTT
4727
4728   def shannon : Finset W2 :=
4729     {((true, true), (true, true)), ((true,
4730       true), (true, true))},

```

```

4731      (((false, true), (false, true)), ((
4732        false, true), (true, false))),
4733      (((false, true), (true, false)), ((
4734        false, false), (false, false))),
4735      (((true, false), (false, false)), ((
4736        false, true), (false, true))),
4737      (((false, false), (false, false)), ((
4738        true, false), (false, false)))}
4739
4740 theorem shannon_clique : IsClique
4741   cohTTdual ( $\cdot \in$  shannon)
4742
4743 theorem shannon_not_mem :
4744   (fun u => if u  $\in$  shannon then (1 :  $\mathbb{R}_{\geq 0}$ )
4745     else 0)  $\notin$  PTTdual
4746
4747 theorem recipe_eq_denotations [Nonempty X]
4748   [Nonempty X'] :
4749   Mix {z | z  $\in$  tD X Y X' Y'  $\wedge \forall u, z u =$ 
4750     0  $\vee z u = 1$ } = Set.range (P0.den (
4751     X := X) (Y := Y)
4752     (X' := X') (Y' := Y'))
4753

```

B.18 Coordinate Retracts

```

4754
4755 noncomputable def push (j : E  $\rightarrow$  F) (x : E
4756    $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) : F  $\rightarrow \mathbb{R}_{\geq 0\infty}$  :=
4757   fun a =>  $\sum d$ , if j d = a then x d else 0
4758
4759 structure Retract (D A : Fm) where
4760   j : web D  $\rightarrow$  web A
4761   inj : Function.Injective j
4762   push_mem :  $\forall x \in P D$ , push j x  $\in P A$ 
4763   pull_mem :  $\forall z \in P A$ , (fun d => z (j d))
4764      $\in P D$ 
4765
4766 def recipe (A : Fm) : Set (web A  $\rightarrow \mathbb{R}_{\geq 0\infty}$ )
4767   :=
4768   ConstructiveProb.Mix {z | z  $\in P A \wedge \forall s$ ,
4769     z s = 0  $\vee z s = 1$ }
4770
4771 theorem Retract.recipe_ne {D A : Fm} (R :
4772   Retract D A) (h : recipe D  $\neq$  P D) :
4773   recipe A  $\neq$  P A
4774
4775 theorem pull_push {j : E  $\rightarrow$  F} (hj :
4776   Function.Injective j) (x : E  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) :
4777   (fun d => push j x (j d)) = x
4778
4779 theorem pairing_push (j : E  $\rightarrow$  F) (u : E  $\rightarrow$ 
4780    $\mathbb{R}_{\geq 0\infty}$ ) (z : F  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) :
4781   pairing (push j u) z = pairing u (fun
4782     d => z (j d))
4783

```

```

4784 def Retract.refl (D : Fm) : Retract D D
4785
4786 def Retract.trans {D A C : Fm} (R :
4787   Retract D A) (S : Retract A C) :
4788   Retract D C
4789
4790 def Retract.neg {D A : Fm} (R : Retract D
4791   A) : Retract (Fm.neg D) (Fm.neg A)
4792
4793 def Retract.negneg (D : Fm) : Retract D (
4794   Fm.neg (Fm.neg D))
4795
4796 def Retract.tensL (D B : Fm) (b : web B) :
4797   Retract D (Fm.tens D B)
4798
4799 def Retract.tensR (D B : Fm) (b : web B) :
4800   Retract D (Fm.tens B D)
4801
4802 def Retract.wthL (D B : Fm) : Retract D (
4803   Fm.wth D B)
4804
4805 def Retract.wthR (D B : Fm) : Retract D (
4806   Fm.wth B D)
4807
4808 theorem push_tensL {X Y : Type} [Fintype X]
4809   [DecidableEq X] [DecidableEq Y] (b :
4810   Y)
4811   (x : X  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) : push (fun d => (d, b)
4812     ) x = tvec x (Pi.single b 1)
4813
4814 theorem push_tensR {X Y : Type} [Fintype X]
4815   [DecidableEq X] [DecidableEq Y] (b :
4816   Y)
4817   (x : X  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) : push (fun d => (b, d)
4818     ) x = tvec (Pi.single b 1) x
4819
4820 theorem push_comp [Fintype F] (j1 : E  $\rightarrow$  F)
4821   (j2 : F  $\rightarrow$  G) (x : E  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) :
4822   push (fun d => j2 (j1 d)) x = push j2
4823     (push j1 x)
4824
4825 theorem push_id [DecidableEq E] (x : E  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) :
4826   push (fun d : E => d) x = x
4827
4828 def NE : Fm  $\rightarrow$  Prop
4829   | base n => 0 < n
4830   | Fm.neg A => NE A
4831   | tens A B => NE A  $\wedge$  NE B
4832   | wth A B => NE A  $\wedge$  NE B
4833
4834 theorem NE.point :  $\forall A : Fm$ , NE A  $\rightarrow$ 
4835   Nonempty (web A)
4836   | base _, h => <<0, h>>
4837   | Fm.neg A, h => NE.point A h
4838

```

```

4841 | tens A B, h => by
4842 | obtain ⟨a⟩ := NE.point A h.1
4843 | obtain ⟨b⟩ := NE.point B h.2
4844 | exact ⟨(a, b)⟩
4845 | with A _, h => by
4846 | obtain ⟨a⟩ := NE.point A h.1
4847 | exact ⟨Sum.inl a⟩
4848
4849 /-- `Occ D p A`: `D` occurs in `A`,
4850 | positively if `p` and negatively
4851 | otherwise. -/
4852 inductive Occ (D : Fm) : Bool → Fm → Prop
4853 | self : Occ D true D
4854 | neg {p : Bool} {A : Fm} : Occ D p A →
4855 | Occ D (!p) (Fm.neg A)
4856 | tensL {p : Bool} {A : Fm} (B : Fm) :
4857 | Occ D p A → Occ D p (tens A B)
4858 | tensR {p : Bool} {A : Fm} (B : Fm) :
4859 | Occ D p A → Occ D p (tens B A)
4860 | withL {p : Bool} {A : Fm} (B : Fm) :
4861 | Occ D p A → Occ D p (with A B)
4862 | withR {p : Bool} {A : Fm} (B : Fm) :
4863 | Occ D p A → Occ D p (with B A)
4864
4865 /-- `D` or `D^⊥`, according to polarity.
4866 -/
4867 def pol : Bool → Fm → Fm
4868 | true, D => D
4869 | false, D => Fm.neg D
4870
4871 /-- **Occurrences give retracts.** -/
4872 theorem occ_retract {D : Fm} : ∀ {p : Bool}
4873 | {A : Fm}, Occ D p A → NE.A →
4874 | Nonempty (Retract (pol p D) A)
4875 | _, _, Occ.self, _ => ⟨Retract.refl D⟩
4876 | _, _, Occ.neg (p := p) h, hA => by
4877 | obtain ⟨R⟩ := occ_retract h hA
4878 | cases p
4879 | · exact ⟨(Retract.negneg D).trans R.
4880 | neg⟩
4881 | · exact ⟨R.neg⟩
4882 | _, _, Occ.tensL B h, hA => by
4883 | obtain ⟨R⟩ := occ_retract h hA.1
4884 | obtain ⟨b⟩ := NE.point B hA.2
4885 | exact ⟨R.trans (Retract.tensL _ B b)⟩
4886 | _, _, Occ.tensR B h, hA => by
4887 | obtain ⟨R⟩ := occ_retract h hA.2
4888 | obtain ⟨b⟩ := NE.point B hA.1
4889 | exact ⟨R.trans (Retract.tensR _ B b)⟩
4890 | _, _, Occ.withL B h, hA => by
4891 | obtain ⟨R⟩ := occ_retract h hA.1
4892 | exact ⟨R.trans (Retract.withL _ B)⟩
4893 | _, _, Occ.withR B h, hA => by
4894 | obtain ⟨R⟩ := occ_retract h hA.2
4895
4896 exact ⟨R.trans (Retract.withR _ B)⟩
4897
4898 /-- The second-order core `(n → m) ⊗ (n' →
4899 | m')`. -/
4900 abbrev core (n m n' m' : ℕ) : Fm := tens (
4901 | lolli (base n) (base m)) (lolli (base n
4902 | ') (base m'))
4903
4904 /-- **The gap at every formula with a
4905 | negative occurrence of the core.** -/
4906 theorem gap_of_occ {n m n' m' : ℕ} (hn : 2
4907 | ≤ n) (hm : 2 ≤ m) (hn' : 2 ≤ n') (hm'
4908 | : 2 ≤ m')
4909 | {A : Fm} (hocc : Occ (core n m n' m')
4910 | false A) (hA : NE.A) : recipe A ⊂ P
4911 | A
4912
4913 example : recipe (lolli (core 2 2 2 2) (
4914 | base 2)) ⊂ P (lolli (core 2 2 2 2) (
4915 | base 2))
4916
4917 def partialCertain : Set (Fin 18 → ℝ≥0∞)
4918 :=
4919 | {s | IsSharpVec s ∧ ∀ b, ∑ i, s (bas b i
4920 | ) ≤ 1}
4921
4922 theorem sum_bases (f : Fin 18 → ℝ≥0∞) :
4923 | ∑ b : Fin 9, ∑ i : Fin 4, f (bas b i)
4924 | = 2 * ∑ v, f v
4925
4926 theorem partial_sum_le_four {s : Fin 18 →
4927 | ℝ≥0∞} (hs : s ∈ partialCertain) : ∑ v,
4928 | s v ≤ 4
4929
4930 theorem mix_sum_le_four {x : Fin 18 → ℝ≥0∞}
4931 | (hx : x ∈ Mix partialCertain) : ∑ v,
4932 | x v ≤ 4
4933
4934 theorem born_total {ψ : Fin 4 → ℝ} (hψ : ∑
4935 | k, ψ k ^ 2 = 1) : 2 * ∑ v, born ψ v =
4936 | 9
4937
4938 theorem born_not_mix_partial {ψ : Fin 4 →
4939 | ℝ} (hψ : ∑ k, ψ k ^ 2 = 1) :
4940 | born ψ ∉ Mix partialCertain
4941
4942 def S4 : Finset (Fin 18) := {0, 1, 3, 11}
4943
4944 theorem S4_stable : ∀ b : Fin 9, (Finset.
4945 | univ.filter (fun i : Fin 4 => bas b i ∈
4946 | S4)).card ≤ 1
4947
4948 noncomputable def ind4 : Fin 18 → ℝ≥0∞ :=
4949 | fun v => if v ∈ S4 then 1 else 0
4950

```

```

4951 theorem ind4_mem : ind4 ∈ partialCertain ∧
4952   Σ v, ind4 v = 4
4953
4954
4955
4956

```

4955 C The Aczél Development

4957 This appendix sketches the from-scratch construction behind
 4958 the product-rule half of Theorem 30. The target shape is
 4959 Aczél's associativity theorem [Aczél 1966]: a suitably regular
 4960 associative $F : \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ regruates to multiplication,
 4961 $G(F x y) = G(x) \cdot G(y)$ for all x, y and a strictly monotone
 4962 G . `mathlib` has no associativity representation, no Hölder-
 4963 style embedding of ordered semigroups on intervals, and no
 4964 Cauchy functional equation under monotonicity (only under
 4965 continuity).

4966 **Hypotheses.** A Scale bundles: associativity of F , *divisi-*
 4967 *bility* $\forall c \exists x, F x x = c$, positivity on the cone $x < F x c$, joint
 4968 order-preservation, joint continuity, a unit u , and identity
 4969 at $-\infty$, meaning $F x z \rightarrow x$ as $z \rightarrow -\infty$. Order-preservation
 4970 is required by the additivity proof below. Divisibility yields
 4971 the 2^n -th roots used in the dyadic construction below. With-
 4972 out it the generator is not definable. It is assumed directly
 4973 in the bundle, though its interval form follows from the in-
 4974 termediate value theorem (`exists_diag_eq`). Non-vacuity:
 4975 *log-sum-exp*, $F x y = \log(e^x + e^y)$ with $u = 0$, satisfies every
 4976 field of the bundle (`logSumExpScale`), with intended gener-
 4977 ator globally `exp`.
 4978

4979 **Construction.** Write $a^{(n)}$ for n right-associated copies
 4980 of a under F (`Fpow`). Associativity alone makes $n \mapsto a^{(n)}$
 4981 a semigroup morphism from $(\mathbb{N}, +)$. Commutativity is not
 4982 assumed, and follows instead as a corollary of the gener-
 4983 ator. Unboundedness of the iterates is derived: were they
 4984 bounded, monotone convergence would give a fixed point
 4985 $F L a = L$, contradicting positivity, and unboundedness yields
 4986 the Archimedean property. Divisibility yields a tower of 2^n -
 4987 th roots $u \succ u^{1/2} \succ u^{1/4} \succ \dots$ (`droot`), where 2^ℓ copies of
 4988 the finer root compose under F to one copy of the coarser
 4989 root. Counting the number of copies of the n -th root not
 4990 exceeding b (`dcount`, well defined by the Archimedean prop-
 4991 erty) gives dyadic approximants $g_n(b) = \text{dcount}(b, n)/2^n$.
 4992 The refinement inequality $2N + 1 \leq N' \leq 2N + 2$, where N
 4993 and N' are the counts at consecutive refinement levels n and
 4994 $n + 1$, makes (g_n) monotone and Cauchy, and the generator
 4995 is its limit, $g(b) = \bigsqcup_n g_n(b)$.
 4996

4997 **Results (all sorry-free, standard axioms).** On
 4998 the cone $\{x \mid u \leq x\}$ we prove three things. Normal-
 4999 ization: $g(u) = 1$. Additivity: $g(F x y) = g(x) + g(y)$,
 5000 since $\text{dcount } x + \text{dcount } y + 1 \leq \text{dcount}(F x y) \leq$
 5001 $\text{dcount } x + \text{dcount } y + 3$ and the 2^{-n} error term vanishes.
 5002 And strict monotonicity, proved without any continuity of
 5003 g : a root inserted between $x < y$ contributes a full extra
 5004 block of 2^{n-M} copies, M being the refinement level of the
 5005

5006 inserted root. Exponentiating gives the multiplicative form,
 5007 a strictly monotone G with $G(F x y) = G(x) G(y)$ on the
 5008 cone (`exists_mul_generator`), the conclusion of Aczél's
 5009 theorem restricted to the cone (`aczelStatement_cone`).
 5010 The reorientation $\bar{G} = \exp(-g)$ gives the $[0, 1]$ -valued form
 5011 used in Section 6.
 5012

5013 **Remaining gap.** The constructed g is discontinuous at
 5014 the unit (it is 0 below u), so matching the classical statement
 5015 verbatim needs the off-cone extension (a group completion)
 5016 and continuity of the extension. This is open. It is not vac-
 5017 uous: the log-sum-exp example admits a global continuous
 5018 generator (`hasOrderedGenerator_logSumExp`). Of Hölder's
 5019 classical hypotheses, the Archimedean property is proved,
 5020 and is what enables the embedding into $\mathbb{R}$. The interval form
 5021 of divisibility follows from the intermediate value theorem.
 5022 Only the embedding's global regularity remains.
 5023
 5024
 5025
 5026
 5027
 5028
 5029
 5030
 5031
 5032
 5033
 5034
 5035
 5036
 5037
 5038
 5039
 5040
 5041
 5042
 5043
 5044
 5045
 5046
 5047
 5048
 5049
 5050
 5051
 5052
 5053
 5054
 5055
 5056
 5057
 5058
 5059
 5060